

Developer Guide for Intel® oneAPI Math Kernel Library Linux*

Contents

Chapter 1: Developer Guide for Intel® oneAPI Math Kernel Library for Linux*

Getting Help and Support	7
What's New	7
Notational Conventions	8
Related Information	9
Getting Started	9
Shared Library Versioning.....	9
CMake Config for oneMKL.....	9
Checking Your Installation	79
Setting Environment Variables	80
Scripts to Set Environment Variables	80
Modulefiles to Set Environment Variables	81
Automating the Process	81
Using the CMake Config File	82
Compiler Support.....	82
Using Code Examples	82
Before You Begin Using the Intel® oneAPI Math Kernel Library (oneMKL) ..	82
Structure of the Intel® oneAPI Math Kernel Library (oneMKL)	84
Architecture Support	84
High-Level Directory Structure	84
Layered Model Concept	86
Linking Your Application with the Intel® oneAPI Math Kernel Library	87
Linking Quick Start.....	87
Using the -qmkl Compiler Option.....	87
Using the -qmkl-ilp64 Compiler Option.....	88
Using the Single Dynamic Library	88
Selecting Libraries to Link with	89
Using the Link-line Advisor	90
Using the Command-Line Link Tool	90
Linking Examples	91
Linking on Intel(R) 64 Architecture Systems	91
Linking in Detail	92
Listing Libraries on a Link Line	93
Dynamically Selecting the Interface and Threading Layer	93
Linking with Interface Libraries	94
Linking with Threading Libraries.....	97
Linking with Computational Libraries	99
Linking with Compiler Run-time Libraries.....	99
Linking with System Libraries	99
Building Custom Shared Objects.....	100
Using the Custom Shared Object Builder	100
Composing a List of Functions	102
Specifying Function Names.....	102
Distributing Your Custom Shared Object.....	103
Managing Performance and Memory.....	103
Improving Performance with Threading	103
OpenMP* Threaded Functions and Problems	104

Functions Threaded with Intel® Threading Building Blocks.....	105
Avoiding Conflicts in the Execution Environment.....	106
Techniques to Set the Number of Threads	107
Setting the Number of Threads Using an OpenMP* Environment Variable	108
Changing the Number of OpenMP* Threads at Run Time	108
Using Additional Threading Control	111
Calling oneMKL Functions from Multi-threaded Applications.....	116
Using Intel® Hyper-Threading Technology	117
Managing Multi-core Performance.....	118
Managing Performance with Heterogeneous Cores	119
Improving Performance for Small Size Problems	120
Using MKL_DIRECT_CALL in C Applications.....	120
Using MKL_DIRECT_CALL in Fortran Applications.....	121
Limitations of the Direct Call	122
Other Tips and Techniques to Improve Performance	122
Coding Techniques	122
Improving oneMKL Performance on Specific Processors	123
Operating on Denormals	124
Using Memory Functions	124
Memory Leaks in Intel® oneAPI Math Kernel Library (oneMKL)	124
Redefining Memory Functions	124
Language-Specific Usage Options	125
Using Language-Specific Interfaces with Intel® oneAPI Math Kernel Library (oneMKL)	125
Interface Libraries and Modules	125
Fortran 95 Interfaces to LAPACK and BLAS	127
Compiler-dependent Functions and Fortran 90 Modules	127
Mixed-Language Programming with the Intel Math Kernel Library	127
Calling LAPACK, BLAS, and CBLAS Routines from C/C++ Language Environments.....	128
Using Complex Types in C/C++	129
Calling BLAS Functions That Return the Complex Values in C/C++ + Code	130
Obtaining Numerically Reproducible Results	132
Getting Started with Conditional Numerical Reproducibility	133
Specifying Code Branches	135
Reproducibility Conditions	136
Setting the Environment Variable for Conditional Numerical Reproducibility	138
Code Examples	138
Coding Tips	140
Example of Data Alignment	140
Using Predefined Preprocessor Symbols for Intel® oneMKL Version- Dependent Compilation	141
Querying oneMATH Specification Version Compliance	143
Managing Output.....	144
Using oneMKL Verbose Mode	144
Version Information Line	146
Call Description Line.....	147
Working with the Intel® Math Kernel Library Cluster Edition Software.....	151
Linking with Intel® Math Kernel Library Cluster Edition Software	151
Setting the Number of OpenMP* Threads	152
Using Shared Libraries	153

Setting Environment Variables on a Cluster.....	153
Interaction with the Message-Passing Interface	153
Using a Custom Message-Passing Interface.....	154
Examples of Linking for Clusters.....	155
Examples for Linking a C Application	155
Examples for Linking a Fortran Application	156
Managing Behavior of the Intel® oneAPI Math Kernel Library with Environment Variables	157
Managing Behavior of Function Domains with Environment Variables ...	157
Setting the Default Mode of Vector Math with an Environment Variable	157
Managing Performance of the Cluster Fourier Transform Functions	158
Managing Invalid Input Checking in LAPACKE Functions	160
Instruction Set–Specific Dispatching on Intel® Architectures	160
Configuring Your Integrated Development Environment to Link with Intel® oneAPI Math Kernel Library (oneMKL)	162
Configuring the Eclipse* IDE CDT to Link with Intel® oneAPI Math Kernel Library (oneMKL).....	162
Intel® Math Kernel Library Benchmarks.....	163
Intel Optimized LINPACK Benchmark for Linux*.....	163
Contents.....	163
Running the Software	164
Known Limitations.....	165
Intel® Distribution for LINPACK* Benchmark and Intel® Optimized HPL- AI* Benchmark.....	165
Overview	165
Overview	166
Contents.....	166
Building the Intel® Distribution for LINPACK* Benchmark and the Intel® Optimized HPL-AI* Benchmark for a Customized MPI Implementation	167
Building the Netlib HPL from Source Code	167
Configuring Parameters	168
Ease-of-use Command-Line Parameters	169
Running the Intel® Distribution for LINPACK* Benchmark and the Intel® Optimized HPL-AI* Benchmark.....	169
Heterogeneous Support in the Intel® Distribution for LINPACK* Benchmark	170
Environment Variables.....	172
Improving Performance of Your Cluster	173
Intel® Optimized High Performance Conjugate Gradient Benchmark.....	173
Overview of the Intel Optimized HPCG	173
Versions of the Intel® CPU Optimized HPCG	174
Versions of the Intel® GPU Optimized HPCG	175
Getting Started with Intel® CPU Optimized HPCG	176
Getting Started with Intel® GPU Optimized HPCG.....	176
Choosing the Best Configuration and Problem Sizes for CPUs	177
Choosing the Best HPCG Configuration for GPUs.....	178
Appendix A: Intel® oneAPI Math Kernel Library (oneMKL) Language Interfaces Support.....	178
Language Interfaces Support, by Function Domain	178
Include Files	179
Appendix B: Support for Third-Party Interfaces.....	181
FFTW Interface Support	181

Appendix C: Directory Structure In Detail	181
Detailed Structure of the Intel(R) 64 Architecture Directories	182
Static Libraries	182
Dynamic Libraries	185
Notices and Disclaimers.....	190

Developer Guide for Intel® oneAPI Math Kernel Library for Linux*

1

For more documentation on this and other products, visit the [oneAPI Documentation Library](#).

Intel® Math Kernel Library is now Intel® oneAPI Math Kernel Library (oneMKL). Documentation for older versions of Intel® Math Kernel Library is available for download only. For a list of available documentation downloads by product version, see these pages:

- [Download Documentation for Intel® Parallel Studio XE](#)
- [Download Documentation for Intel® System Studio](#)

What's New

NOTE

Starting with the 2024.0 release, macOS is no longer supported in Intel® oneAPI Toolkits and components.

Several Intel-led open-source developer-tool projects will continue supporting macOS on Apple Silicon including oneAPI Threading Building Blocks (oneTBB) and Intel® Implicit SPMD Program Compiler, and we welcome the opportunity to work with contributors to expand support to additional tools in the future.

All macOS content will be removed from technical documentation in the 2024.1 release. If you need to keep a copy of the documentation, download it from the [Downloadable Documentation](#) site.

Intel® oneAPI Math Kernel Library (oneMKL) is a computing math library of highly optimized, extensively threaded routines for applications that require maximum performance. The library provides Fortran and C programming language interfaces. oneMKL C language interfaces can be called from applications written in either C or C++, as well as in any other language that can reference a C interface.

oneMKL provides comprehensive functionality support in these major areas of computation:

- BLAS (level 1, 2, and 3) and LAPACK linear algebra routines, offering vector, vector-matrix, and matrix-matrix operations.
- ScaLAPACK distributed processing linear algebra routines, as well as the Basic Linear Algebra Communications Subprograms (BLACS) and the Parallel Basic Linear Algebra Subprograms (PBLAS).
- oneMKL PARDISO (a direct sparse solver based on Parallel Direct Sparse Solver PARDISO*), an iterative sparse solver, and supporting sparse BLAS (level 1, 2, and 3) routines for solving sparse systems of equations, as well as a distributed version of oneMKL PARDISO solver provided for use on clusters.
- Fast Fourier transform (FFT) functions in one, two, or three dimensions with support for mixed radices (not limited to sizes that are powers of 2), as well as distributed versions of these functions provided for use on clusters.
- Vector Mathematics (VM) routines for optimized mathematical operations on vectors.
- Vector Statistics (VS) routines, which offer high-performance vectorized random number generators (RNG) for several probability distributions, convolution and correlation routines, and summary statistics functions.
- Data Fitting Library, which provides capabilities for spline-based approximation of functions, derivatives and integrals of functions, and search.
- Extended Eigensolver, a shared memory programming (SMP) version of an eigensolver based on the Feast Eigenvalue Solver.

For detailed function descriptions, including calling syntax, see:

- [Intel® oneAPI Math Kernel Library Developer Reference - C](#)
- [Intel® oneAPI Math Kernel Library Developer Reference - Fortran](#)

Intel® oneAPI Math Kernel Library (oneMKL) is optimized for performance on Intel processors. oneMKL also runs on non-Intel x86-compatible processors.

For Windows* and Linux* systems based on Intel® 64 Architecture, oneMKL also includes support for the Intel® Many Integrated Core Architecture (Intel® MIC Architecture) and provides libraries to help you port your applications to Intel MIC Architecture.

This document explains different aspects of Intel® oneAPI Math Kernel Library (oneMKL) usage and helps you perform multiple tasks related to programming with Intel® oneAPI Math Kernel Library (oneMKL), in particular:

- [Run code examples.](#)
- [Set environment variables.](#)
- [Link your application](#), including [linking with Intel MKL cluster domains](#).
- [Improve performance with threading.](#)
- Benchmark your cluster with [Intel® Distribution for LINPACK* Benchmark](#) or [Intel® Optimized High Performance Conjugate Gradient Benchmark](#).
- [Obtain numerically reproducible results.](#)

NOTE

oneMKL provides limited input validation to minimize the performance overheads. It is your responsibility when using oneMKL to ensure that input data has the required format and does not contain invalid characters. These can cause unexpected behavior of the library. Examples of the inputs that may result in unexpected behavior:

- Not-a-number (NaN) and other special floating point values
- Large inputs may lead to accumulator overflow

As the oneMKL API accepts raw pointers, it is your application's responsibility to validate the buffer sizes before passing them to the library. The library requires subroutine and function parameters to be valid before being passed. While some oneMKL routines do limited checking of parameter errors, your application should check for NULL pointers, for example.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.intel.com/PerformanceIndex.

Notice revision #20201201

Getting Help and Support

Intel provides a support web site that contains a rich repository of self help information, including getting started tips, known product issues, product errata, license information, user forums, and more. Visit the Intel® oneAPI Math Kernel Library (oneMKL) support website at <http://www.intel.com/software/products/support/>.

What's New

For more documentation on this and other products, visit the [oneAPI Documentation Library](#).

Intel® Math Kernel Library is now Intel® oneAPI Math Kernel Library (oneMKL). Documentation for older versions of Intel® Math Kernel Library is available for download only. For a list of available documentation downloads by product version, see these pages:

- [Download Documentation for Intel® Parallel Studio XE](#)

- [Download Documentation for Intel® System Studio](#)

This Developer Guide documents Intel® oneAPI Math Kernel Library.

The Developer Guide has been updated to reflect enhancements to the product, improvements, and error corrections.

Notational Conventions

The following term is used in reference to the operating system.

Linux* This term refers to information that is valid on all supported Linux* operating systems.

The following notations are used to refer to Intel® oneAPI Math Kernel Library (oneMKL) directories.

<parent directory> The installation directory that includes the Intel® oneAPI Math Kernel Library (oneMKL) directory; for example, the directory for the Intel® oneAPI Base Toolkit.

<mkl directory> The main directory where Intel® oneAPI Math Kernel Library (oneMKL) is installed:
<mkl directory>=<parent directory>/mkl.
 Replace this placeholder with the specific pathname in the configuring, linking, and building instructions.

The following font conventions are used in this document.

Italic Italic is used for emphasis and also indicates document names in body text, for example:
 see *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference*.

Monospace lowercase Indicates filenames, directory names, and pathnames, for example: `./benchmarks/linpack`

Monospace lowercase mixed with uppercase Indicates:

- Commands and command-line options, for example,
`icx myprog.c -L$MKLPATH -I$MKLINCLUDE -lmkl -liomp5 -lpthread`
- Filenames, directory names, and pathnames, for example,
- C/C++ code fragments, for example,
`a = new double [SIZE*SIZE];`

UPPERCASE MONOSPACE Indicates system variables, for example, `$MKLPATH`.

Monospace italic Indicates a parameter in discussions, for example, *lda*.
 When enclosed in angle brackets, indicates a placeholder for an identifier, an expression, a string, a symbol, or a value, for example, *<mkl directory>*.
 Substitute one of these items for the placeholder.

[items] Square brackets indicate that the items enclosed in brackets are optional.

{ item | item } Braces indicate that only one of the items listed between braces should be selected. A vertical bar (|) separates the items.

Related Information

To reference how to use the library in your application, use this guide in conjunction with the following documents:

- The *Intel® oneAPI Math Kernel Library Developer Reference*, which provides *reference* information on routine functionalities, parameter descriptions, interfaces, calling syntaxes, and return values.
- The *Intel® oneAPI Math Kernel Library forLinux* OS Release Notes*.

Web versions of these documents are available in the Intel® Software Documentation Library at <https://www.intel.com/content/www/us/en/develop/documentation.html>.

Getting Started

Shared Library Versioning

Intel® oneAPI Math Kernel Library (oneMKL) adds shared library versioning for all operating systems and platforms, as opposed to not using any library versioning up to Intel® Math Kernel Library (Intel® MKL) 2020 Update 4.

This new feature:

- Allows applications to work correctly in an environment with multiple oneMKL and/or Intel® MKL packages installed
- Communicates clearly when incompatible changes are made, and an application should be rebuilt
- Allows you to link against a specific version of shared libraries

The starting version for shared libraries is “1” and any change that break backward compatibility will result in an increment to this number. Intel expects to make this change as seldom as possible and inform customers about it at least 24 months in advance.

The product version “2021.1” is now decoupled from the library version, meaning that “2021.2” can ship with shared libraries versioned as “1”. This means that the libraries shipped in “2021.2” are backward compatible with libraries shipped in “2021.1”.

Changes to the link-line:

- On Linux*, no changes are required to the link-line because symbolic links are provided with the old names, which point to the new library that contains the version information. The symbolic link name is also the `soname` of that library.
 - For example, `libmkl_core.so -> libmkl_core.so.<version>`
 - Using `-lmkl_core` will still work as before, ensuring backward compatibility with Intel® MKL 2020 line-up (including Intel® Parallel Studio and Intel® System Studio distributions).
- On Windows*, import libraries used in the link-line do not contain any version information, as before, but point to the new DLL, which contains the version information.
 - For example, `mkl_core_dll.lib` has the same name as before and requires no change to the link-line. The linker, however, resolves this to the new `mkl_core.<version>.dll` instead of the older `mkl_core.dll`.

CMake Config for oneMKL

If you want to integrate oneMKL into your CMake projects, starting with the Intel® oneAPI Math Kernel Library (oneMKL) 2021.3 release, `MKLConfig.cmake` is provided as part of the package and installation. `MKLConfig.cmake` supports all oneMKL configurations, compilers, and runtimes, as the oneMKL product itself. Help/usage is provided in the top section of `MKLConfig.cmake`.

Example

NOTE The following example demonstrates how to link with oneMKL using the C/Fortran API. For the oneMKL SYCL functionality, use the `MKL::MKL_DPCPP` target instead of `MKL::MKL`. For more information about how to link cluster functionality, enable the OpenMP Offload feature, specify the link type and threading type, and so forth, refer to the input and output parameters listed in the comments at the top of the `MKLConfig.cmake` file.

```
my_test/
|_ build/          <-- Out-of-source build directory
|_ CMakeLists.txt  <-- User side project's CMakeLists.txt
|_ app.c           <-- Source file that uses oneMKL API
```

app.c

```
#include <stdio.h>
#include "mkl.h"

int main()
{
    MKLVersion mkl_version;
    mkl_get_version(&mkl_version);

    printf("You are using oneMKL %d.%d\n", mkl_version.MajorVersion, mkl_version.UpdateVersion);

    return 0;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

project(oneMKL_Example LANGUAGES C)

find_package(MKL CONFIG REQUIRED PATHS $ENV{MKLROOT})

add_executable(myapp app.c)
target_link_libraries(myapp PUBLIC MKL::MKL)
```

Command line

```
# Source the oneAPI setvars.sh/oneapi-vars.sh script beforehand
build$ cmake .. -DCMAKE_C_COMPILER=icx
build$ cmake --build .
```

If your application requires a customized setting, the following `CMakeLists.txt` demonstrates how to achieve a more fine-grained control.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.13)

project(oneMKL_Example LANGUAGES C)

find_package(MKL CONFIG REQUIRED PATHS $ENV{MKLROOT})
message(STATUS "Imported oneMKL targets: ${MKL_IMPORTED_TARGETS}")

add_executable(myapp app.c)
target_compile_options(myapp PUBLIC $<TARGET_PROPERTY:MKL::MKL,INTERFACE_COMPILE_OPTIONS>)
```

```
target_include_directories(myapp PUBLIC
$<TARGET_PROPERTY:MKL::MKL,INTERFACE_INCLUDE_DIRECTORIES>)
target_link_libraries(myapp PUBLIC $<LINK_ONLY:MKL::MKL>)
```

NOTE

When the Ninja build system is in use, Ninja 1.10.2 or later is required for Fortran support.

CBLAS Example**mkl_example.h**

```
/*
*****
* Copyright 1999-2022 Intel Corporation.
*
* This software and the related documents are Intel copyrighted materials, and
* your use of them is governed by the express license under which they were
* provided to you (License). Unless the License provides otherwise, you may not
* use, modify, copy, publish, distribute, disclose or transmit this software or
* the related documents without Intel's prior written permission.
*
* This software and the related documents are provided as is, with no express
* or implied warranties, other than those that are expressly stated in the
* License.
*****
*/

/*
! Content:
! Intel(R) oneAPI Math Kernel Library (oneMKL) examples interface
!*****
*/

#ifndef _MKL_EXAMPLE_H_
#define _MKL_EXAMPLE_H_

#include "mkl_types.h"
#include "mkl_cblas.h"
#include <stdio.h>

#ifdef __cplusplus
extern "C" {
#endif /* __cplusplus */

#define COMMENTS ':'
#define MAX_STRING_LEN 512

#define MIN(a,b) ((a) > (b)) ? (b) : (a)
#define MAX(a,b) ((a) > (b)) ? (a) : (b)
#define ABS(a) ((a) < 0) ? -(a) : (a)

#define GENERAL_MATRIX 0
#define UPPER_MATRIX 1
#define LOWER_MATRIX -1

#define FULLPRINT 0
#define SHORTPRINT 1

#ifdef MKL_ILP64
```

```

#define INT_FORMAT "%lld"
#else
#define INT_FORMAT "%d"
#endif

/*
 * =====
 * Prototypes for example program functions
 * =====
 */

int GetIntegerParameters(FILE*, ...);
int GetIntegerParameters8(FILE*, ...);
int GetIntegerParameters16(FILE*, ...);
int GetIntegerParameters32(FILE*, ...);
int GetScalarsH(FILE*, ...);
int GetScalarsD(FILE*, ...);
int GetScalarsS(FILE*, ...);
int GetScalarsC(FILE*, ...);
int GetScalarsZ(FILE*, ...);
int GetCblasCharParameters(FILE *in_file, ...);
MKL_INT MaxValue(MKL_INT, MKL_INT*);
MKL_INT GetVectorI(FILE*, MKL_INT, MKL_INT*);
MKL_INT GetVectorI32(FILE*, MKL_INT, MKL_INT32*);
MKL_INT GetVectorS(FILE*, MKL_INT, float*, MKL_INT);
MKL_INT GetVectorC(FILE*, MKL_INT, MKL_Complex8*, MKL_INT);
MKL_INT GetVectorD(FILE*, MKL_INT, double*, MKL_INT);
MKL_INT GetVectorZ(FILE*, MKL_INT, MKL_Complex16*, MKL_INT);
MKL_INT GetArrayI8(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, MKL_INT8*, MKL_INT*);
MKL_INT GetArrayI16(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, MKL_INT16*, MKL_INT*);
MKL_INT GetArrayI32(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, MKL_INT32*, MKL_INT*);
MKL_INT GetArrayBF16(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, MKL_BF16*, MKL_INT*);
MKL_INT GetArrayH(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, MKL_F16*, MKL_INT*);
MKL_INT GetArrayS(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, float*, MKL_INT*);
MKL_INT GetArrayD(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, double*, MKL_INT*);
MKL_INT GetArrayC(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, MKL_Complex8*, MKL_INT*);
MKL_INT GetArrayZ(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT*, MKL_INT*, MKL_Complex16*, MKL_INT*);
MKL_INT GetBandArrayS(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT, MKL_INT, MKL_INT, float*, MKL_INT);
MKL_INT GetBandArrayD(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT, MKL_INT, MKL_INT, double*,
MKL_INT);
MKL_INT GetBandArrayC(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT, MKL_INT, MKL_INT, MKL_Complex8*,
MKL_INT);
MKL_INT GetBandArrayZ(FILE*, CBLAS_LAYOUT*, MKL_INT, MKL_INT, MKL_INT, MKL_INT, MKL_Complex16*,
MKL_INT);
MKL_INT GetValuesI8(FILE*, MKL_INT8*, MKL_INT, MKL_INT, MKL_INT);
MKL_INT GetValuesI16(FILE*, MKL_INT16*, MKL_INT, MKL_INT, MKL_INT);
MKL_INT GetValuesI32(FILE*, MKL_INT32*, MKL_INT, MKL_INT, MKL_INT);
MKL_INT GetValuesI(FILE *, MKL_INT*, MKL_INT, MKL_INT);
MKL_INT GetValuesBF16(FILE *, MKL_BF16*, MKL_INT, MKL_INT, MKL_INT);
MKL_INT GetValuesH(FILE *, MKL_F16*, MKL_INT, MKL_INT, MKL_INT);
MKL_INT GetValuesC(FILE *, MKL_Complex8*, MKL_INT, MKL_INT, MKL_INT);
MKL_INT GetValuesZ(FILE *, MKL_Complex16*, MKL_INT, MKL_INT, MKL_INT);
MKL_INT GetValuesD(FILE *, double*, MKL_INT, MKL_INT, MKL_INT);
MKL_INT GetValuesS(FILE *, float*, MKL_INT, MKL_INT, MKL_INT);
float GetValueH(MKL_F16);
void PrintParameters( char*, ... );
void PrintVectorI(MKL_INT, MKL_INT*, char*);
void PrintVectorI32(MKL_INT, MKL_INT32*, char*);

```

```

void PrintVectorS(int, MKL_INT, float*, MKL_INT, char*);
void PrintVectorC(int, MKL_INT, MKL_Complex8*, MKL_INT, char*);
void PrintVectorD(int, MKL_INT, double*, MKL_INT, char*);
void PrintVectorZ(int, MKL_INT, MKL_Complex16*, MKL_INT, char*);
void PrintArrayI8(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, MKL_INT8*, MKL_INT*, char*);
void PrintArrayI16(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, MKL_INT16*, MKL_INT*, char*);
void PrintArrayI32(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, MKL_INT32*, MKL_INT*, char*);
void PrintArrayBF16(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, MKL_BF16*, MKL_INT*, char*);
void PrintArrayH(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, MKL_F16*, MKL_INT*, char*);
void PrintArrayS(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, float*, MKL_INT*, char*);
void PrintArrayD(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, double*, MKL_INT*, char*);
void PrintArrayC(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, MKL_Complex8*, MKL_INT*, char*);
void PrintArrayZ(CBLAS_LAYOUT*, int, int, MKL_INT*, MKL_INT*, MKL_Complex16*, MKL_INT*, char*);
void PrintBandArrayS(CBLAS_LAYOUT*, int, MKL_INT, MKL_INT, MKL_INT, MKL_INT, float*, MKL_INT,
char*);
void PrintBandArrayD(CBLAS_LAYOUT*, int, MKL_INT, MKL_INT, MKL_INT, MKL_INT, double*, MKL_INT,
char*);
void PrintBandArrayC(CBLAS_LAYOUT*, int, MKL_INT, MKL_INT, MKL_INT, MKL_INT, MKL_Complex8*,
MKL_INT, char*);
void PrintBandArrayZ(CBLAS_LAYOUT*, int, MKL_INT, MKL_INT, MKL_INT, MKL_INT, MKL_Complex16*,
MKL_INT, char*);

#ifdef __cplusplus
}
#endif /* __cplusplus */

#endif /* _MKL_EXAMPLE_H_ */

```

common_func.c

```

/*****
 * Copyright 1999-2022 Intel Corporation.
 *
 * This software and the related documents are Intel copyrighted materials, and
 * your use of them is governed by the express license under which they were
 * provided to you (License). Unless the License provides otherwise, you may not
 * use, modify, copy, publish, distribute, disclose or transmit this software or
 * the related documents without Intel's prior written permission.
 *
 * This software and the related documents are provided as is, with no express
 * or implied warranties, other than those that are expressly stated in the
 * License.
 *****/

/*
! Content:
!
! *****/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <stdarg.h>
#include <math.h>

#include "mkl_example.h"
#include "mkl_cblas.h"

```

```

MKL_INT MaxValue(MKL_INT n, MKL_INT *x)
{
    MKL_INT    i, indmax;

    indmax = x[0];
    for (i = 1; i < n; i++)
        if (indmax < x[i]) indmax = x[i];
    return indmax;
} /* MaxValue */

typedef union {
    float float_part;
    MKL_BF16 int_part[2];
} conv_union_bf16;

typedef union {
    MKL_F16 raw;
    struct {
        unsigned int frac : 10;
        unsigned int exp  : 5;
        unsigned int sign : 1;
    } bits;
} conv_union_f16;

typedef union {
    float raw;
    struct {
        unsigned int frac : 23;
        unsigned int exp  : 8;
        unsigned int sign : 1;
    } bits;
} conv_union_f32;

static float b2f(MKL_BF16 src) {
    conv_union_bf16 conv;
    conv.int_part[0] = 0;
    conv.int_part[1] = src;
    return conv.float_part;
}

static MKL_BF16 f2b(float src) {
    conv_union_bf16 conv;
    conv.float_part = src;
    return conv.int_part[1];
}

static float h2f(MKL_F16 x) {
    conv_union_f16 src;
    conv_union_f32 dst;

    src.raw = x;
    dst.raw = 0;
    dst.bits.sign = src.bits.sign;

    if (src.bits.exp == 0x01fU) {
        dst.bits.exp = 0xffU;
        if (src.bits.frac > 0) {
            dst.bits.frac = ((src.bits.frac | 0x200U) << 13);
        }
    }
}

```

```

    }
} else if (src.bits.exp > 0x00U) {
    dst.bits.exp = src.bits.exp + ((1 << 7) - (1 << 4));
    dst.bits.frac = (src.bits.frac << 13);
} else {
    unsigned int v = (src.bits.frac << 13);

    if (v > 0) {
        dst.bits.exp = 0x71;
        while ((v & 0x800000UL) == 0) {
            dst.bits.exp--;
            v <<= 1;
        }
        dst.bits.frac = v;
    }
}

return dst.raw;
}

static MKL_F16 f2h(float x) {
    conv_union_f32 src;
    conv_union_f16 dst;

    src.raw = x;
    dst.raw = 0;
    dst.bits.sign = src.bits.sign;

    if (src.bits.exp == 0xffU) {
        dst.bits.exp = 0x01fU;
        dst.bits.frac = (src.bits.frac >> 13);
        if (src.bits.frac > 0) dst.bits.frac |= 0x200U;
    } else if (src.bits.exp >= 0x08fU) {
        dst.bits.exp = 0x01fU;
        dst.bits.frac = 0x000U;
    } else if (src.bits.exp >= 0x071U) {
        dst.bits.exp = src.bits.exp + ((1 << 4) - (1 << 7));
        dst.bits.frac = (src.bits.frac >> 13);
    } else if (src.bits.exp >= 0x067U) {
        dst.bits.exp = 0x000;
        if (src.bits.frac > 0) {
            dst.bits.frac = (((1U << 23) | src.bits.frac) >> 14);
        } else {
            dst.bits.frac = 1;
        }
    }
}

return dst.raw;
}

float GetValueH(MKL_F16 x)
{
    return h2f( x );
} /* GetValueH */

MKL_INT GetVectorI(FILE *in_file, MKL_INT n, MKL_INT *x)
{
    return GetValuesI( in_file, x, 0, n );
}

```

```

} /* GetVectorI */

MKL_INT GetVectorI16(FILE *in_file, MKL_INT n, MKL_INT16 *x)
{
    return GetValuesI16( in_file, x, 1, 0, n );
} /* GetVectorI16 */

MKL_INT GetVectorI32(FILE *in_file, MKL_INT n, MKL_INT32 *x)
{
    return GetValuesI32( in_file, x, 1, 0, n );
} /* GetVectorI32 */

MKL_INT GetVectorS(FILE *in_file, MKL_INT n, float *x, MKL_INT incx)
{
    return GetValuesS( in_file, x, 1, 0, (1+(n-1)*ABS(incx)) );
} /* GetVectorS */

MKL_INT GetVectorD(FILE *in_file, MKL_INT n, double *x, MKL_INT incx)
{
    return GetValuesD( in_file, x, 1, 0, (1+(n-1)*ABS(incx)) );
} /* GetVectorD */

MKL_INT GetVectorC(FILE *in_file, MKL_INT n, MKL_Complex8 *x, MKL_INT incx)
{
    return GetValuesC( in_file, x, 1, 0, (1+(n-1)*ABS(incx)) );
} /* GetVectorC */

MKL_INT GetVectorZ(FILE *in_file, MKL_INT n, MKL_Complex16 *x, MKL_INT incx)
{
    return GetValuesZ( in_file, x, 1, 0, (1+(n-1)*ABS(incx)) );
} /* GetVectorZ */

MKL_INT GetArrayI8(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag, MKL_INT *m, MKL_INT *n,
                   MKL_INT8 *a, MKL_INT *lda)
{
    MKL_INT i, j, number;
    MKL_INT8 *addr;

    if( *layout == CblasRowMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI8( in_file, addr, 1, 0, *n );
                if( number != *n ) return 1;
            } /* for */
        } else if( flag == UPPER_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI8( in_file, addr, 1, i, *n-i );
                if( number != *n-i ) return 1;
            } /* for */
        } else if( flag == LOWER_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI8( in_file, addr, 1, 0, i+1 );
                if( number != i+1 ) return 1;
            } /* for */
        }
    }
}

```



```

    } /* if */
} else if( *layout == CblasColMajor ) {
    if( flag == GENERAL_MATRIX ) {
        for( j = 0; j < (*n); j++ ) {
            addr = a + j*(*lda);
            number = GetValuesI8( in_file, addr, 1, 0, *m );
            if( number != *m ) return 1;
        } /* for */
    } else if( flag == UPPER_MATRIX ) {
        for( j = 0; j < (*n); j++ ) {
            addr = a + j*(*lda);
            number = GetValuesI8( in_file, addr, 1, 0, j+1 );
            if( number != j+1 ) return 1;
        } /* for */
    } else if( flag == LOWER_MATRIX ) {
        for( j = 0; j < (*n); j++ ) {
            addr = a + j*(*lda);
            number = GetValuesI8( in_file, addr, 1, j, *m-j );
            if( number != *m-j ) return 1;
        } /* for */
    } /* if */
} /* if */
return 0;
} /* GetArrayI8 */

MKL_INT GetArrayI16(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag, MKL_INT *m, MKL_INT *n,
                   MKL_INT16 *a, MKL_INT *lda)
{
    MKL_INT i, j , number;
    MKL_INT16 *addr;

    if( *layout == CblasRowMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI16( in_file, addr, 1, 0, *n );
                if( number != *n ) return 1;
            } /* for */
        } else if( flag == UPPER_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI16( in_file, addr, 1, i, *n-i );
                if( number != *n-i ) return 1;
            } /* for */
        } else if( flag == LOWER_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI16( in_file, addr, 1, 0, i+1 );
                if( number != i+1 ) return 1;
            } /* for */
        } /* if */
    } else if( *layout == CblasColMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesI16( in_file, addr, 1, 0, *m );
                if( number != *m ) return 1;
            } /* for */
        }
    }
}

```

```

    } else if( flag == UPPER_MATRIX ) {
        for( j = 0; j < (*n); j++ ) {
            addr = a + j*(*lda);
            number = GetValuesI16( in_file, addr, 1, 0, j+1 );
            if( number != j+1 ) return 1;
        } /* for */
    } else if( flag == LOWER_MATRIX ) {
        for( j = 0; j < (*n); j++ ) {
            addr = a + j*(*lda);
            number = GetValuesI16( in_file, addr, 1, j, *m-j );
            if( number != *m-j ) return 1;
        } /* for */
    } /* if */
} /* if */
return 0;
} /* GetArrayI16 */

MKL_INT GetArrayI32(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag, MKL_INT *m, MKL_INT *n,
                   MKL_INT32 *a, MKL_INT *lda)
{
    MKL_INT i, j , number;
    MKL_INT32 *addr;

    if( *layout == CblasRowMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI32( in_file, addr, 1, 0, *n );
                if( number != *n ) return 1;
            } /* for */
        } else if( flag == UPPER_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI32( in_file, addr, 1, i, *n-i );
                if( number != *n-i ) return 1;
            } /* for */
        } else if( flag == LOWER_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesI32( in_file, addr, 1, 0, i+1 );
                if( number != i+1 ) return 1;
            } /* for */
        } /* if */
    } else if( *layout == CblasColMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesI32( in_file, addr, 1, 0, *m );
                if( number != *m ) return 1;
            } /* for */
        } else if( flag == UPPER_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesI32( in_file, addr, 1, 0, j+1 );
                if( number != j+1 ) return 1;
            } /* for */
        } else if( flag == LOWER_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {

```

```

        addr = a + j*(*lda);
        number = GetValuesI32( in_file, addr, 1, j, *m-j );
        if( number != *m-j ) return 1;
    } /* for */
} /* if */
} /* if */
return 0;
} /* GetArrayI32 */

MKL_INT GetArrayBF16(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag, MKL_INT *m, MKL_INT *n,
                    MKL_BF16 *a, MKL_INT *lda)
{
    MKL_INT i, j, number;
    MKL_BF16 *addr;

    if( *layout == CblasRowMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesBF16( in_file, addr, 1, 0, *n );
                if( number != *n ) return 1;
            } /* for */
        } else if( flag == UPPER_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesBF16( in_file, addr, 1, i, *n-i );
                if( number != *n-i ) return 1;
            } /* for */
        } else if( flag == LOWER_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesBF16( in_file, addr, 1, 0, i+1 );
                if( number != i+1 ) return 1;
            } /* for */
        } /* if */
    } else if( *layout == CblasColMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesBF16( in_file, addr, 1, 0, *m );
                if( number != *m ) return 1;
            } /* for */
        } else if( flag == UPPER_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesBF16( in_file, addr, 1, 0, j+1 );
                if( number != j+1 ) return 1;
            } /* for */
        } else if( flag == LOWER_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesBF16( in_file, addr, 1, j, *m-j );
                if( number != *m-j ) return 1;
            } /* for */
        } /* if */
    } /* if */
    return 0;
} /* GetArrayBF16 */

```

```

MKL_INT GetArrayH(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag, MKL_INT *m, MKL_INT *n,
                  MKL_F16 *a, MKL_INT *lda)
{
    MKL_INT    i, j, number;
    MKL_F16    *addr;

    if( *layout == CblasRowMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( i = 0; i < (*m); i++ ) {
                addr = a + i*(*lda);
                number = GetValuesH( in_file, addr, 1, 0, *n );
                if( number != *n ) return 1;
            } /* for */
        } else if( flag == UPPER_MATRIX ) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesH( in_file, addr, 1, i, *n-i );
                if( number != *n-i ) return 1;
            } /* for */
        } else if( flag == LOWER_MATRIX ) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesH( in_file, addr, 1, 0, i+1 );
                if( number != i+1 ) return 1;
            } /* for */
        } /* if */
    } else if( *layout == CblasColMajor ) {
        if( flag == GENERAL_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesH( in_file, addr, 1, 0, *m );
                if( number != *m ) return 1;
            } /* for */
        } else if( flag == UPPER_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesH( in_file, addr, 1, 0, j+1 );
                if( number != j+1 ) return 1;
            } /* for */
        } else if( flag == LOWER_MATRIX ) {
            for( j = 0; j < (*n); j++ ) {
                addr = a + j*(*lda);
                number = GetValuesH( in_file, addr, 1, j, *m-j );
                if( number != *m-j ) return 1;
            } /* for */
        } /* if */
    } /* if */
    return 0;
} /* GetArrayH */

MKL_INT GetArrayS(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag, MKL_INT *m, MKL_INT *n,
                  float *a, MKL_INT *lda)
{
    MKL_INT    i, j, number;
    float      *addr;

    if( *layout == CblasRowMajor ) {

```

```

    if( flag == GENERAL_MATRIX ) {
        for( i = 0; i < (*m); i++ ) {
            addr = a + i*(*lda);
            number = GetValuesS( in_file, addr, 1, 0, *n );
            if( number != *n ) return 1;
        } /* for */
    } else if( flag == UPPER_MATRIX ) {
        for (i = 0; i < (*m); i++) {
            addr = a + i*(*lda);
            number = GetValuesS( in_file, addr, 1, i, *n-i );
            if( number != *n-i ) return 1;
        } /* for */
    } else if( flag == LOWER_MATRIX ) {
        for (i = 0; i < (*m); i++) {
            addr = a + i*(*lda);
            number = GetValuesS( in_file, addr, 1, 0, i+1 );
            if( number != i+1 ) return 1;
        } /* for */
    } /* if */
} else if( *layout == CblasColMajor ) {
    if( flag == GENERAL_MATRIX ) {
        for( j = 0; j < (*n); j++ ) {
            addr = a + j*(*lda);
            number = GetValuesS( in_file, addr, 1, 0, *m );
            if( number != *m ) return 1;
        } /* for */
    } else if( flag == UPPER_MATRIX ) {
        for( j = 0; j < (*n); j++ ) {
            addr = a + j*(*lda);
            number = GetValuesS( in_file, addr, 1, 0, j+1 );
            if( number != j+1 ) return 1;
        } /* for */
    } else if( flag == LOWER_MATRIX ) {
        for( j = 0; j < (*n); j++ ) {
            addr = a + j*(*lda);
            number = GetValuesS( in_file, addr, 1, j, *m-j );
            if( number != *m-j ) return 1;
        } /* for */
    } /* if */
} /* if */
return 0;
} /* GetArrayS */

MKL_INT GetArrayD(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag, MKL_INT *m, MKL_INT *n,
    double *a, MKL_INT *lda)
{
    MKL_INT i, j, number;
    double *addr;

    if (*layout == CblasRowMajor) {
        if (flag == GENERAL_MATRIX) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesD( in_file, addr, 1, 0, *n );
                if( number != *n ) return 1;
            } /* for */
        } else if (flag == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {

```

```

        addr = a + i*(*lda);
        number = GetValuesD( in_file, addr, 1, i, *n-i );
        if( number != *n-i ) return 1;
    } /* for */
} else if (flag == LOWER_MATRIX) {
    for (i = 0; i < (*m); i++) {
        addr = a + i*(*lda);
        number = GetValuesD( in_file, addr, 1, 0, i+1 );
        if( number != i+1 ) return 1;
    } /* for */
} /* if */
} else if (*layout == CblasColMajor) {
    if (flag == GENERAL_MATRIX) {
        for (j = 0; j < (*n); j++) {
            addr = a + j*(*lda);
            number = GetValuesD( in_file, addr, 1, 0, *m );
            if( number != *m ) return 1;
        } /* for */
    } else if (flag == UPPER_MATRIX) {
        for (j = 0; j < (*n); j++) {
            addr = a + j*(*lda);
            number = GetValuesD( in_file, addr, 1, 0, j+1 );
            if( number != j+1 ) return 1;
        } /* for */
    } else if (flag == LOWER_MATRIX) {
        for (j = 0; j < (*n); j++) {
            addr = a + j*(*lda);
            number = GetValuesD( in_file, addr, 1, j, *m-j );
            if( number != *m-j ) return 1;
        } /* for */
    } /* if */
} /* if */
return 0;
} /* GetArrayD */

MKL_INT GetArrayC(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag, MKL_INT *m, MKL_INT *n,
                  MKL_Complex8 *a, MKL_INT *lda)
{
    MKL_INT      i, j, number;
    MKL_Complex8 *addr;

    if (*layout == CblasRowMajor) {
        if (flag == GENERAL_MATRIX) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesC( in_file, addr, 1, 0, *n );
                if( number != *n ) return 1;
            } /* for */
        } else if (flag == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesC( in_file, addr, 1, i, *n-i );
                if( number != *n-i ) return 1;
            } /* for */
        } else if (flag == LOWER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesC( in_file, addr, 1, 0, i+1 );

```

```

        if( number != i+1 ) return 1;
    } /* for */
} /* if */
} else if (*layout == CblasColMajor) {
    if (flag == GENERAL_MATRIX) {
        for (j = 0; j < (*n); j++) {
            addr = a + j*(*lda);
            number = GetValuesC( in_file, addr, 1, 0, *m );
            if( number != *m ) return 1;
        } /* for */
    } else if (flag == UPPER_MATRIX) {
        for (j = 0; j < (*n); j++) {
            addr = a + j*(*lda);
            number = GetValuesC( in_file, addr, 1, 0, j+1 );
            if( number != j+1 ) return 1;
        } /* for */
    } else if (flag == LOWER_MATRIX) {
        for (j = 0; j < (*n); j++) {
            addr = a + j*(*lda);
            number = GetValuesC( in_file, addr, 1, j, *m-j );
            if( number != *m-j ) return 1;
        } /* for */
    } /* if */
} /* if */
return 0;
} /* GetArrayC */

MKL_INT GetArrayZ(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT flag,
                  MKL_INT *m, MKL_INT *n, MKL_Complex16 *a, MKL_INT *lda)
{
    MKL_INT      i, j, number;
    MKL_Complex16 *addr;

    if (*layout == CblasRowMajor) {
        if (flag == GENERAL_MATRIX) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesZ( in_file, addr, 1, 0, *n );
                if( number != *n ) return 1;
            } /* for */
        } else if (flag == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesZ( in_file, addr, 1, i, *n-i );
                if( number != *n-i ) return 1;
            } /* for */
        } else if (flag == LOWER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                addr = a + i*(*lda);
                number = GetValuesZ( in_file, addr, 1, 0, i+1 );
                if( number != i+1 ) return 1;
            } /* for */
        } /* if */
    } else if (*layout == CblasColMajor) {
        if (flag == GENERAL_MATRIX) {
            for (j = 0; j < (*n); j++) {
                addr = a + j*(*lda);
                number = GetValuesZ( in_file, addr, 1, 0, *m );

```

```

        if( number != *m ) return 1;
    } /* for */
} else if (flag == UPPER_MATRIX) {
    for (j = 0; j < (*n); j++) {
        addr = a + j*(*lda);
        number = GetValuesZ( in_file, addr, 1, 0, j+1 );
        if( number != j+1 ) return 1;
    } /* for */
} else if (flag == LOWER_MATRIX) {
    for (j = 0; j < (*n); j++) {
        addr = a + j*(*lda);
        number = GetValuesZ( in_file, addr, 1, j, *m-j );
        if( number != *m-j ) return 1;
    } /* for */
} /* if */
} /* if */
return 0;
} /* GetArrayZ */

MKL_INT GetBandArrayS(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT kl, MKL_INT ku,
                    MKL_INT m, MKL_INT n, float *a, MKL_INT lda)
{
    MKL_INT i, j;
    MKL_INT kl1, kul, i_start, j_start, j_end, ku_rows, kl_rows, number;
    float *addr, *addr1;

    if (*layout == CblasRowMajor) {
        for( i = 0; i < MIN( m, n ); i++ ) {
            addr = a + i*lda;
            kl1 = ( i - kl <= 0 ) ? i : kl;
            kul = ( i + ku >= n ) ? MAX(0,n-i-1) : ku;
            j_start = kl - kl1;
            j_end = j_start + kl1 + kul;
            addr1 = addr + j_start;
            number = GetValuesS( in_file, addr1, 1, 0, j_end-j_start+1 );
            if( number != j_end-j_start+1 ) return 1;
        }
        for( i = MIN( m, n ); i < MIN( m, MIN( m, n ) + kl ); i++ ) {
            addr = a + i*lda;
            kl1 = n - i + kl;
            j_start = ( kl > n ) ? kl - n : n - kl;
            j_end = j_start + kl1 - 1;
            addr1 = addr + j_start;
            number = GetValuesS( in_file, addr1, 1, 0, j_end-j_start+1 );
            if( number != j_end-j_start+1 ) return 1;
        }
    } else if (*layout == CblasColMajor) {
        i_start = (ku > n) ? ku - n + 1 : 0;
        ku_rows = (ku > n) ? n - 1 : ku;
        j_start = ku_rows;
        for( i = 0; i < ku_rows; i++ ) {
            j = j_start*lda; addr1 = a+i*i_start;
            number = GetValuesS( in_file, addr1, lda, j, n-ku_rows );
            if( number != n-ku_rows ) return 1;
            j_start--;
        }
    }

    j_end = MIN(m,n);

```



```

    addr1 = a + ku;
    number = GetValuesS( in_file, addr1, lda, 0, j_end );
    if( number != j_end ) return 1;

    kl_rows = ( kl <= m-1 ) ? kl : m - 1;
    for ( i = 1; i < kl_rows+1; i++ ) {
        kl1 = ( i+j_end <= m ) ? j_end : m - i;
        addr1 = a + ku + i;
        number = GetValuesS( in_file, addr1, lda, 0, kl1 );
        if( number != kl1 ) return 1;
    }
}
return 0;
} /* GetBandArrayS */

MKL_INT GetBandArrayD(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT kl, MKL_INT ku,
                    MKL_INT m, MKL_INT n, double *a, MKL_INT lda)
{
    MKL_INT    i, j;
    MKL_INT    kl1, kul, i_start, j_start, j_end, kl_rows, ku_rows, number;
    double     *addr, *addr1;

    if (*layout == CblasRowMajor) {
        for( i = 0; i < MIN( m, n ); i++ ) {
            addr = a + i*lda;
            kl1 = ( i - kl <= 0 ) ? i : kl;
            kul = ( i + ku >= n ) ? MAX(0,n-i-1) : ku;
            j_start = kl - kl1;
            j_end = j_start + kl1 + kul;
            addr1 = addr + j_start;
            number = GetValuesD( in_file, addr1, 1, 0, j_end-j_start+1 );
            if( number != j_end-j_start+1 ) return 1;
        }
        for( i = MIN( m, n ); i < MIN( m, MIN( m, n ) + kl ); i++ ) {
            addr = a + i*lda;
            kl1 = n - i + kl;
            j_start = ( kl > n ) ? kl - n : n - kl;
            j_end = j_start + kl1 - 1;
            addr1 = addr + j_start;
            number = GetValuesD( in_file, addr1, 1, 0, j_end-j_start+1 );
            if( number != j_end-j_start+1 ) return 1;
        }
    } else if (*layout == CblasColMajor) {
        i_start = (ku > n) ? ku - n + 1 : 0;
        ku_rows = (ku > n) ? n - 1 : ku;
        j_start = ku_rows;
        for( i = 0; i < ku_rows; i++ ) {
            j = j_start*lda; addr1 = a+i+i_start;
            number = GetValuesD( in_file, addr1, lda, j, n-ku_rows );
            if( number != n-ku_rows ) return 1;
            j_start--;
        }

        j_end = MIN(m,n);
        addr1 = a+ku;
        number = GetValuesD( in_file, addr1, lda, 0, j_end );
        if( number != j_end ) return 1;
    }
}

```

```

        kl_rows = ( kl <= m-1 ) ? kl : m - 1;
        for ( i = 1; i < kl_rows+1; i++ ) {
            k11 = ( i+j_end <= m ) ? j_end : m - i;
            addr1 = a+ku+i;
            number = GetValuesD( in_file, addr1, lda, 0, k11 );
            if( number != k11 ) return 1;
        }
    } /* if */
    return 0;
} /* GetBandArrayD */

MKL_INT GetBandArrayC(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT kl, MKL_INT ku,
                    MKL_INT m, MKL_INT n, MKL_Complex8 *a, MKL_INT lda)
{
    MKL_INT      i, j;
    MKL_INT      k11, ku1, i_start, j_start, j_end, ku_rows, kl_rows, number;
    MKL_Complex8 *addr, *addr1;

    if (*layout == CblasRowMajor) {
        for( i = 0; i < MIN( m, n ); i++ ) {
            addr = a + i*lda;
            k11 = ( i - kl <= 0 ) ? i : kl;
            ku1 = ( i + ku >= n ) ? MAX(0,n-i-1) : ku;
            j_start = kl - k11;
            j_end = j_start + k11 + ku1;
            addr1 = addr + j_start;
            number = GetValuesC( in_file, addr1, 1, 0, j_end-j_start+1 );
            if( number != j_end-j_start+1 ) return 1;
        }
        for( i = MIN( m, n ); i < MIN( m, MIN( m, n ) + kl ); i++ ) {
            addr = a + i*lda;
            k11 = n - i + kl;
            j_start = ( kl > n ) ? kl - n : n - kl;
            j_end = j_start + k11 - 1;
            addr1 = addr + j_start;
            number = GetValuesC( in_file, addr1, 1, 0, j_end-j_start+1 );
            if( number != j_end-j_start+1 ) return 1;
        }
    } else if (*layout == CblasColMajor) {
        i_start = (ku > n) ? ku - n + 1 : 0;
        ku_rows = (ku > n) ? n - 1 : ku;
        j_start = ku_rows;
        for( i = 0; i < ku_rows; i++ ) {
            j = j_start*lda; addr1 = a+i+i_start;
            number = GetValuesC( in_file, addr1, lda, j, n-ku_rows );
            if( number != n-ku_rows ) return 1;
            j_start--;
        }

        j_end = MIN(m,n);
        addr1 = a+ku;
        number = GetValuesC( in_file, addr1, lda, 0, j_end );
        if( number != j_end ) return 1;

        kl_rows = ( kl <= m-1 ) ? kl : m - 1;
        for ( i = 1; i < kl_rows+1; i++ ) {
            k11 = ( i+j_end <= m ) ? j_end : m - i;
            addr1 = a+ku+i;

```

```

        number = GetValuesC( in_file, addr1, lda, 0, kl1 );
        if( number != kl1 ) return 1;
    }
} /* if */
return 0;
} /* GetBandArrayC */

MKL_INT GetBandArrayZ(FILE *in_file, CBLAS_LAYOUT*layout, MKL_INT kl, MKL_INT ku,
                    MKL_INT m, MKL_INT n, MKL_Complex16 *a, MKL_INT lda)
{
    MKL_INT      i, j;
    MKL_INT      kl1, kul, i_start, j_start, j_end, ku_rows, kl_rows, number;
    MKL_Complex16 *addr, *addr1;

    if (*layout == CblasRowMajor) {
        for( i = 0; i < MIN( m, n ); i++ ) {
            addr = a + i*lda;
            kl1 = ( i - kl <= 0 ) ? i : kl;
            kul = ( i + ku >= n ) ? MAX(0,n-i-1) : ku;
            j_start = kl - kl1;
            j_end = j_start + kl1 + kul;
            addr1 = addr + j_start;
            number = GetValuesZ( in_file, addr1, 1, 0, j_end-j_start+1 );
            if( number != j_end-j_start+1 ) return 1;
        }
        for( i = MIN( m, n ); i < MIN( m, MIN( m, n ) + kl ); i++ ) {
            addr = a + i*lda;
            kl1 = n - i + kl;
            j_start = ( kl > n ) ? kl - n : n - kl;
            j_end = j_start + kl1 - 1;
            addr1 = addr + j_start;
            number = GetValuesZ( in_file, addr1, 1, 0, j_end-j_start+1 );
            if( number != j_end-j_start+1 ) return 1;
        }
    } else if (*layout == CblasColMajor) {
        i_start = (ku > n) ? ku - n + 1 : 0;
        ku_rows = (ku > n) ? n - 1 : ku;
        j_start = ku_rows;
        for( i = 0; i < ku_rows; i++ ) {
            j = j_start*lda; addr1 = a+i+i_start;
            number = GetValuesZ( in_file, addr1, lda, j, n-ku_rows );
            if( number != n-ku_rows ) return 1;
            j_start--;
        }

        j_end = MIN(m,n);
        addr1 = a+ku;
        number = GetValuesZ( in_file, addr1, lda, 0, j_end );
        if( number != j_end ) return 1;

        kl_rows = ( kl <= m-1 ) ? kl : m - 1;
        for ( i = 1; i < kl_rows+1; i++ ) {
            kl1 = ( i+j_end <= m ) ? j_end : m - i;
            addr1 = a+ku+i;
            number = GetValuesZ( in_file, addr1, lda, 0, kl1 );
            if( number != kl1 ) return 1;
        }
    } /* if */
}

```

```

        return 0;
    } /* GetBandArrayZ */

MKL_INT GetValuesI( FILE *in_file, MKL_INT *in_array, MKL_INT begin, MKL_INT max_numbers )
{
    MKL_INT i, counter=0;
    int     value;
    char     buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%d", &value) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i]=(MKL_INT)value;
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
    }
    return counter;
}

MKL_INT GetValuesI8( FILE *in_file, MKL_INT8 *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )
{
    MKL_INT i, counter=0;
    int value_int;
    char     buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%d", &value_int) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i*ld]=(MKL_INT8)value_int;
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
    }
    return counter;
}

```

```

MKL_INT GetValuesI16( FILE *in_file, MKL_INT16 *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )
{
    MKL_INT i, counter=0;
    MKL_INT16    value;
    char    buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%hd", &value) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i*ld]=(MKL_INT16)value;
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
    }
    return counter;
}

MKL_INT GetValuesI32( FILE *in_file, MKL_INT32 *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )
{
    MKL_INT i, counter=0;
    MKL_INT32    value;
    char    buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%d", &value) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i*ld]=(MKL_INT32)value;
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
    }
    return counter;
}

```

```

MKL_INT GetValuesBF16( FILE *in_file, MKL_BF16 *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )
{
    MKL_INT i, counter=0;
    float    temp;
    char     buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%f", &temp) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i*ld]=f2b(temp);
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
    }
    return counter;
} /* GetValuesBF16 */

```

```

MKL_INT GetValuesH( FILE *in_file, MKL_F16 *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )
{
    MKL_INT i, counter=0;
    float    temp;
    char     buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%f", &temp) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i*ld]=f2h(temp);
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
    }
    return counter;
} /* GetValuesH */

```

```

MKL_INT GetValuesD( FILE *in_file, double *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )

```

```

{
    MKL_INT i, counter=0;
    double temp;
    char buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%lf", &temp) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i*ld]=temp;
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
    }
    return counter;
} /* GetValuesD */

MKL_INT GetValuesS( FILE *in_file, float *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )
{
    MKL_INT i, counter=0;
    float temp;
    char buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%f", &temp) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i*ld]=temp;
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
    }
    return counter;
} /* GetValuesS */

MKL_INT GetValuesC( FILE *in_file, MKL_Complex8 *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )
{
    MKL_INT i, counter=0;

```

```

float    re, im;
char     seps[]=" ,()\t";
char     buf[MAX_STRING_LEN], *str;

do {
    fgets( buf, MAX_STRING_LEN, in_file );
    str = strtok( buf, seps );
    if( str == NULL ){
        printf( "\n File format is inappropriate\n");
        return 0;
    }
} while ( *str == COMMENTS );
for( i = 0; i < max_numbers; i++ ) {
    if ( *str==COMMENTS ) break;
    if( sscanf( str, "%f", &re) != 1 || (str = strtok( NULL,seps)) == NULL ||
        sscanf( str, "%f", &im) != 1 ){
        printf( "\n File format is inappropriate\n" );
        return 0;
    }
    in_array[begin+i*ld].real=re;
    in_array[begin+i*ld].imag=im;
    counter++;
    if ( (str = strtok( NULL, seps )) == NULL ) break;
}
return counter;
} /* GetValuesC */

MKL_INT GetValuesZ( FILE *in_file, MKL_Complex16 *in_array, MKL_INT ld, MKL_INT begin, MKL_INT
max_numbers )
{
    MKL_INT i, counter=0;
    double  re, im;
    char     seps[]=" ,()\t";
    char     buf[MAX_STRING_LEN], *str;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, seps );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    for( i = 0; i < max_numbers; i++ ) {
        if ( *str==COMMENTS ) break;
        if( sscanf( str, "%lf", &re) != 1 || (str = strtok( NULL,seps))== NULL ||
            sscanf( str, "%lf", &im) != 1 ){
            printf( "\n File format is inappropriate\n" );
            return 0;
        }
        in_array[begin+i*ld].real=re;
        in_array[begin+i*ld].imag=im;
        counter++;
        if ( (str = strtok( NULL, seps )) == NULL ) break;
    }
    return counter;
} /* GetValuesZ */

```



```

int GetIntegerParameters8(FILE *in_file, ...)
{
    int      counter=0;
    char     buf[MAX_STRING_LEN], *str;
    MKL_INT8 *param;
    va_list  ap;
    int      value;

    do {
        fgets(buf, MAX_STRING_LEN, in_file);
        str = strtok( buf, " " );
        if( str == NULL ){
            printf("\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start(ap, in_file);
    param = va_arg(ap, MKL_INT8 *);

    while ( 1 ) {
        if( *str == COMMENTS ) break;
        if( sscanf(str, "%d", &value ) != 1 ){
            printf("\n File format is inappropriate\n");
            va_end( ap );
            return 0;
        }
        *param = (MKL_INT8)value;
        counter++;
        if( (str = strtok( NULL, " " )) == NULL ) break;
        param = va_arg( ap, MKL_INT8 * );
    }
    va_end( ap );
    return counter;
} /* GetIntegerParameters16 */

int GetIntegerParameters16(FILE *in_file, ...)
{
    int      counter=0;
    char     buf[MAX_STRING_LEN], *str;
    MKL_INT16 *param;
    va_list  ap;
    MKL_INT16 value;

    do {
        fgets(buf, MAX_STRING_LEN, in_file);
        str = strtok( buf, " " );
        if( str == NULL ){
            printf("\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start(ap, in_file);
    param = va_arg(ap, MKL_INT16 *);

    while ( 1 ) {
        if( *str == COMMENTS ) break;
        if( sscanf(str, "%hd", &value ) != 1 ){
            printf("\n File format is inappropriate\n");

```

```

        va_end( ap );
        return 0;
    }
    *param = (MKL_INT16)value;
    counter++;
    if( (str = strtok( NULL, " " )) == NULL ) break;
    param = va_arg( ap, MKL_INT16 * );
}
va_end( ap );
return counter;
} /* GetIntegerParameters16 */

int GetIntegerParameters32(FILE *in_file, ...)
{
    int        counter=0;
    char        buf[MAX_STRING_LEN], *str;
    MKL_INT32 *param;
    va_list     ap;
    MKL_INT32     value;

    do {
        fgets(buf, MAX_STRING_LEN, in_file);
        str = strtok( buf, " " );
        if( str == NULL ){
            printf("\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start(ap, in_file);
    param = va_arg(ap, MKL_INT32 *);

    while ( 1 ) {
        if( *str == COMMENTS ) break;
        if( sscanf(str, "%d", &value ) != 1 ){
            printf("\n File format is inappropriate\n");
            va_end( ap );
            return 0;
        }
        *param = (MKL_INT32)value;
        counter++;
        if( (str = strtok( NULL, " " )) == NULL ) break;
        param = va_arg( ap, MKL_INT32 * );
    }
    va_end( ap );
    return counter;
} /* GetIntegerParameters32 */

int GetIntegerParameters(FILE *in_file, ...)
{
    int        counter=0;
    char        buf[MAX_STRING_LEN], *str;
    MKL_INT *param;
    va_list     ap;
    int        value;

    do {
        fgets(buf, MAX_STRING_LEN, in_file);
        str = strtok( buf, " " );

```

```

        if( str == NULL ){
            printf("\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start(ap, in_file);
    param = va_arg(ap, MKL_INT *);

    while ( 1 ) {
        if( *str == COMMENTS ) break;
        if( sscanf(str, "%d", &value ) != 1 ){
            printf("\n File format is inappropriate\n");
            va_end( ap );
            return 0;
        }
        *param = (MKL_INT)value;
        counter++;
        if( (str = strtok( NULL, " " )) == NULL ) break;
        param = va_arg( ap, MKL_INT * );
    }
    va_end( ap );
    return counter;
} /* GetIntegerParameters */

int GetCblasCharParameters(FILE *in_file, ...)
{
    int      counter=0;
    char      buf[MAX_STRING_LEN], *str;
    int      *param;
    va_list  ap;

    do {
        fgets(buf, MAX_STRING_LEN, in_file);
        str = strtok( buf, " " );
        if( str == NULL ){
            printf("\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start(ap, in_file);
    param = va_arg(ap, int *);

    while ( 1 ) {
        if( *str == COMMENTS ) break;
        if( sscanf(str, "%d", param ) != 1 ){
            printf("\n File format is inappropriate\n");
            va_end( ap );
            return 0;
        }
        counter++;
        if( (str = strtok( NULL, " " )) == NULL ) break;
        param = va_arg( ap, int * );
    }
    va_end( ap );
    return counter;
} /* GetCblasCharParameters */

int GetScalarsH( FILE *in_file, ... )

```

```

{
    int      counter=0;
    char     buf[MAX_STRING_LEN], *str;
    MKL_F16 *param;
    va_list ap;
    float    t;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start( ap, in_file );
    param = va_arg( ap, MKL_F16 * );
    while ( 1 ) {
        if ( *str == COMMENTS ) break;
        if( sscanf( str, "%f", &t ) != 1 ){
            printf( "\n File format is inappropriate\n" );
            va_end( ap );
            return 0;
        }
    }
    *param = f2h(t);
    counter++;
    if ( (str = strtok( NULL, " " )) == NULL ) break;
    param = va_arg( ap, MKL_F16 * );
}
va_end( ap );
return counter;
} /* GetScalarsH */

int GetScalarsD( FILE *in_file, ... )
{
    int      counter=0;
    char     buf[MAX_STRING_LEN], *str;
    double   *param;
    va_list ap;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start( ap, in_file );
    param = va_arg( ap, double * );
    while ( 1 ) {
        if ( *str == COMMENTS ) break;
        if( sscanf( str, "%lf", param ) != 1 ){
            printf( "\n File format is inappropriate\n" );
            va_end( ap );
            return 0;
        }
    }
    counter++;
}

```

```

        if ( (str = strtok( NULL, " " )) == NULL ) break;
        param = va_arg( ap, double * );
    }
    va_end( ap );
    return counter;
} /* GetScalarsD */

int GetScalarsS( FILE *in_file, ... )
{
    int      counter=0;
    char      buf[MAX_STRING_LEN], *str;
    float     *param;
    va_list ap;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, " " );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start( ap, in_file );
    param = va_arg( ap, float * );
    while ( 1 ) {
        if ( *str == COMMENTS ) break;
        if( sscanf( str, "%f", param ) != 1 ){
            printf( "\n File format is inappropriate\n" );
            va_end( ap );
            return 0;
        }
        counter++;
        if ( (str = strtok( NULL, " " )) == NULL ) break;
        param = va_arg( ap, float * );
    }
    va_end( ap );
    return counter;
} /* GetScalarsS */

int GetScalarsC( FILE *in_file, ... )
{
    int      counter=0;
    char      buf[MAX_STRING_LEN], *str;
    char      seps[]=" ,()\\t";
    float      re = 0.f, im = 0.f;
    MKL_Complex8 *param;
    va_list ap;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, seps );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start( ap, in_file );
    param = va_arg( ap, MKL_Complex8 * );

```

```

while ( 1 ) {
    if ( *str == COMMENTS ) break;
    if( sscanf( str, "%f", &re) != 1 || (str = strtok( NULL,seps))== NULL ||
        sscanf( str, "%f", &im) != 1 ){
        printf( "\n File format is inappropriate %f %f\n", re, im );
        va_end( ap );
        return 0;
    }
    param->real = re;
    param->imag = im;
    counter++;
    if ( (str = strtok( NULL, seps )) == NULL ) break;
    param = va_arg( ap, MKL_Complex8 * );
}
va_end( ap );
return counter;
} /* GetScalarsC */

int GetScalarsZ( FILE *in_file, ... )
{
    int          counter=0;
    char          buf[MAX_STRING_LEN], *str;
    char          seps[]=" ,()\t";
    double        re = 0., im = 0.;
    MKL_Complex16 *param;
    va_list       ap;

    do {
        fgets( buf, MAX_STRING_LEN, in_file );
        str = strtok( buf, seps );
        if( str == NULL ){
            printf( "\n File format is inappropriate\n");
            return 0;
        }
    } while ( *str == COMMENTS );
    va_start( ap, in_file );
    param = va_arg( ap, MKL_Complex16 * );
    while ( 1 ) {
        if ( *str == COMMENTS ) break;
        if( sscanf( str, "%lf", &re) != 1 || (str = strtok( NULL,seps)) == NULL ||
            sscanf( str, "%lf", &im) != 1 ){
            printf( "\n File format is inappropriate %f %f\n", re, im );
            va_end( ap );
            return 0;
        }
        param->real = re;
        param->imag = im;
        counter++;
        if ( (str = strtok( NULL, seps )) == NULL ) break;
        param = va_arg( ap, MKL_Complex16 * );
    }
    va_end( ap );
    return counter;
} /* GetScalarsZ */

void PrintParameters( char *names, ... )
{
    char          *p, *str, str1[MAX_STRING_LEN], buf[MAX_STRING_LEN];

```

```

va_list      ap;
char         seps[]=" ,";
MKL_INT      itmp;
MKL_UINT     i;

printf("\n      ");
va_start( ap, names );
strcpy(buf, names);
str = strtok( buf, seps );
if( str == NULL ){
    printf( "\n You must determine the parameters names\n");
    return;
}
do {
    strcpy(str1, str);
    p = str1;
    for( i = 0; i < strlen(str1); i++ ) {
        *p = (char) tolower(str1[i]);
        p++;
    }
    if ( strcmp( str1, "layout" ) == 0 ) {
        itmp = va_arg( ap, CBLAS_LAYOUT );
        if ( itmp == CblasRowMajor )
            printf("LAYOUT = CblasRowMajor  ");
        else if ( itmp == CblasColMajor )
            printf("LAYOUT = CblasColMajor  ");
    } else if ( strcmp( str1, "side" ) == 0 ) {
        itmp = va_arg( ap, CBLAS_SIDE );
        if ( itmp == CblasLeft )
            printf("SIDE = CblasLeft  ");
        else if ( itmp == CblasRight )
            printf("SIDE = CblasRight  ");
    } else if ( strcmp( str1, "uplo" ) == 0 ) {
        itmp = va_arg( ap, CBLAS_UPLO );
        if ( itmp == CblasUpper )
            printf("UPLO = CblasUpper  ");
        else if ( itmp == CblasLower )
            printf("UPLO = CblasLower  ");
    } else if ( strcmp( str1, "diag" ) == 0 ) {
        itmp = va_arg( ap, CBLAS_DIAG );
        if ( itmp == CblasUnit )
            printf("DIAG=CblasUnit  ");
        else if ( itmp == CblasNonUnit )
            printf("DIAG = CblasNonUnit  ");
    } else if ( ( strcmp( str1, "trans" ) == 0 ) ||
                ( strcmp( str1, "transa" ) == 0 ) ||
                ( strcmp( str1, "transb" ) == 0 ) ) {
        itmp = va_arg( ap, CBLAS_TRANSPOSE );
        if ( itmp == CblasNoTrans )
            printf("%s = CblasNoTrans  ", str);
        else if ( itmp == CblasTrans )
            printf("%s = CblasTrans  ", str);
        else if ( itmp == CblasConjTrans )
            printf("%s = CblasConjTrans  ", str);
    } else if ( strcmp( str1, "offsetc" ) == 0 ) {
        itmp = va_arg( ap, CBLAS_OFFSET );
        if ( itmp == CblasFixOffset ) {
            printf("OFFSETC = CblasFixOffset ");

```

```

        } else if ( itmp == CblasRowOffset ) {
            printf("OFFSETC = CblasRowOffset ");
        } else if ( itmp == CblasColOffset ) {
            printf("OFFSETC = CblasColOffset ");
        }
    }
} while ( (str = strtok( NULL, seps )) != NULL );
va_end( ap );
return;
} /* PrintParameters */

void PrintVectorI(MKL_INT n, MKL_INT *x, char *name)
{
    MKL_INT i;

    printf("\n      VECTOR %s\n      ", name);
    for (i = 0; i < n; i++)
        printf("      INT_FORMAT, *(x+i));
    return;
} /* PrintVectorI */

void PrintVectorI16(MKL_INT n, MKL_INT16 *x, char *name)
{
    MKL_INT i;

    printf("\n      VECTOR %s\n      ", name);
    for (i = 0; i < n; i++)
        printf("      %d", *(x+i));
    return;
} /* PrintVectorI16 */

void PrintVectorI32(MKL_INT n, MKL_INT32 *x, char *name)
{
    MKL_INT i;

    printf("\n      VECTOR %s\n      ", name);
    for (i = 0; i < n; i++)
        printf("      %d", *(x+i));
    return;
} /* PrintVectorI32 */

void PrintVectorS(int flag, MKL_INT n, float *x, MKL_INT incx, char *name)
{
    MKL_INT i;

    switch(flag) {
    case 0:
        printf("\n      VECTOR %s  INC%s=" INT_FORMAT "\n      ",
            name, name, incx);
        break;
    case 1:
        printf("\n      VECTOR %s\n      ", name);
        break;
    default:
        break;
    } /* switch */
    for (i = 0; i < (1+(n-1)*ABS(incx)); i++)
        printf("%6.2f ", *(x+i));

```



```

        return;
    } /* PrintVectorS */

void PrintVectorD(int flag, MKL_INT n, double *x, MKL_INT incx, char *name)
{
    MKL_INT i;

    switch(flag) {
    case 0:
        printf("\n          VECTOR %s   INC%s=" INT_FORMAT "\n          ",
               name, name, incx);
        break;
    case 1:
        printf("\n          VECTOR %s\n          ", name);
        break;
    default:
        break;
    } /* switch */
    for (i = 0; i < (1+(n-1)*ABS(incx)); i++)
        printf("%8.3f  ", *(x+i));
    return;
} /* PrintVectorD */

void PrintVectorC(int flag, MKL_INT n, MKL_Complex8 *x, MKL_INT incx, char *name)
{
    MKL_INT i;

    switch(flag) {
    case 0:
        printf("\n          VECTOR %s   INC%s=" INT_FORMAT "\n          ",
               name, name, incx);
        break;
    case 1:
        printf("\n          VECTOR %s\n          ", name);
        break;
    default:
        break;
    } /* switch */
    for (i = 0; i < (1+(n-1)*ABS(incx)); i++)
        printf("(%6.2f,%6.2f)  ", (x+i)->real, (x+i)->imag);
    return;
} /* PrintVectorC */

void PrintVectorZ(int flag, MKL_INT n, MKL_Complex16 *x, MKL_INT incx, char *name)
{
    MKL_INT i;

    switch(flag) {
    case 0:
        printf("\n          VECTOR %s   INC%s=" INT_FORMAT "\n          ",
               name, name, incx);
        break;
    case 1:
        printf("\n          VECTOR %s\n          ", name);
        break;
    default:
        break;
    } /* switch */

```

```

        for (i = 0; i < (1+(n-1)*ABS(incx)); i++)
            printf("(%.2f,%.2f)  ", (x+i)->real, (x+i)->imag);
        return;
} /* PrintVectorZ */

void PrintArrayI8(CBLAS_LAYOUT*layout, int flag1, int flag2, MKL_INT *m, MKL_INT *n, MKL_INT8 *a,
                 MKL_INT *lda, char *name)
{
    MKL_INT i, j;
    MKL_INT8 *addr;

    switch(flag1) {
    case 0:
        printf("\n      ARRAY %s  LD%s=" INT_FORMAT, name, name, *lda);
        break;
    case 1:
        printf("\n      ARRAY %s", name);
        break;
    default:
        break;
    } /* switch */
    if (*layout == CblasRowMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n      ");
                addr = a + i*(*lda);
                for (j = 0; j < *n; j++)
                    printf("%d ", (int) *(addr+j));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n      ");
                addr = a + i*(*lda);
                for (j=0; j < i; j++)
                    printf("      ");
                for (j = i; j < *n; j++)
                    printf("%d ", (int) *(addr+j));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n      ");
                addr = a + i*(*lda);
                for (j = 0; j <= i; j++)
                    printf("%d ", (int) *(addr+j));
            } /* for */
        } /* if */
    } else if (*layout == CblasColMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n      ");
                addr = a + i;
                for (j = 0; j < *n; j++)
                    printf("%d ", (int) *(addr+j*(*lda)));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n      ");
                addr = a + i;

```

```

        for (j=0; j < i; j++)
            printf("          ");
        for (j = i; j < *n; j++)
            printf("%d ", (int)*(addr+j*(*lda)));
    } /* for */
} else if (flag2 == LOWER_MATRIX) {
    for (i = 0; i < (*m); i++) {
        printf("\n          ");
        addr = a + i;
        for (j = 0; j <= i; j++)
            printf("%d ", (int) *(addr+j*(*lda)));
    } /* for */
} /* if */
} /* if */
return;
} /* PrintArrayI8 */

void PrintArrayI16(CBLAS_LAYOUT*layout, int flag1, int flag2, MKL_INT *m, MKL_INT *n, MKL_INT16
*a,
                MKL_INT *lda, char *name)
{
    MKL_INT i, j;
    MKL_INT16 *addr;

    switch(flag1) {
    case 0:
        printf("\n          ARRAY %s   LD%s=" INT_FORMAT, name, name, *lda);
        break;
    case 1:
        printf("\n          ARRAY %s", name);
        break;
    default:
        break;
    } /* switch */
    if (*layout == CblasRowMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j = 0; j < *n; j++)
                    printf("%hd ", *(addr+j));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j=0; j < i; j++)
                    printf("          ");
                for (j = i; j < *n; j++)
                    printf("%hd ", *(addr+j));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j = 0; j <= i; j++)
                    printf("%hd ", *(addr+j));
            } /* for */
        }
    }
}

```

```

        } /* if */
    } else if (*layout == CblasColMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i;
                for (j = 0; j < *n; j++)
                    printf("%hd ", *(addr+j*(*lda)));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n          ");
                addr = a + i;
                for (j=0; j < i; j++)
                    printf("          ");
                for (j = i; j < *n; j++)
                    printf("%hd ", *(addr+j*(*lda)));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n          ");
                addr = a + i;
                for (j = 0; j <= i; j++)
                    printf("%hd ", *(addr+j*(*lda)));
            } /* for */
        } /* if */
    } /* if */
    return;
} /* PrintArrayI16 */

void PrintArrayI32(CBLAS_LAYOUT*layout, int flag1, int flag2, MKL_INT *m, MKL_INT *n, MKL_INT32
*a,
                MKL_INT *lda, char *name)
{
    MKL_INT i, j;
    MKL_INT32 *addr;

    switch(flag1) {
    case 0:
        printf("\n          ARRAY %s   LD%s=" INT_FORMAT, name, name, *lda);
        break;
    case 1:
        printf("\n          ARRAY %s", name);
        break;
    default:
        break;
    } /* switch */
    if (*layout == CblasRowMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j = 0; j < *n; j++)
                    printf("%d ", *(addr+j));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n          ");

```

```

        addr = a + i*(*lda);
        for (j=0; j < i; j++)
            printf("          ");
        for (j = i; j < *n; j++)
            printf("%d ", *(addr+j));
    } /* for */
} else if (flag2 == LOWER_MATRIX) {
    for (i = 0; i < *m; i++) {
        printf("\n          ");
        addr = a + i*(*lda);
        for (j = 0; j <= i; j++)
            printf("%d ", *(addr+j));
    } /* for */
} /* if */
} else if (*layout == CblasColMajor) {
    if (flag2 == GENERAL_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n          ");
            addr = a + i;
            for (j = 0; j < *n; j++)
                printf("%d ", *(addr+j*(*lda)));
        } /* for */
    } else if (flag2 == UPPER_MATRIX) {
        for (i = 0; i < (*m); i++) {
            printf("\n          ");
            addr = a + i;
            for (j=0; j < i; j++)
                printf("          ");
            for (j = i; j < *n; j++)
                printf("%d ", *(addr+j*(*lda)));
        } /* for */
    } else if (flag2 == LOWER_MATRIX) {
        for (i = 0; i < (*m); i++) {
            printf("\n          ");
            addr = a + i;
            for (j = 0; j <= i; j++)
                printf("%d ", *(addr+j*(*lda)));
        } /* for */
    } /* if */
} /* if */
return;
} /* PrintArrayI32 */

void PrintArrayBF16(CBLAS_LAYOUT*layout, int flag1, int flag2, MKL_INT *m, MKL_INT *n, MKL_BF16
*a,
                MKL_INT *lda, char *name)
{
    MKL_INT i, j;
    MKL_BF16 *addr;

    switch(flag1) {
    case 0:
        printf("\n          ARRAY %s   LD%s=" INT_FORMAT, name, name, *lda);
        break;
    case 1:
        printf("\n          ARRAY %s", name);
        break;
    default:

```

```

        break;
    } /* switch */
    if (*layout == CblasRowMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j = 0; j < *n; j++)
                    printf("%6.2f  ", b2f(*(addr+j)));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j=0; j < i; j++)
                    printf("          ");
                for (j = i; j < *n; j++)
                    printf("%6.2f  ", b2f(*(addr+j)));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j = 0; j <= i; j++)
                    printf("%6.2f  ", b2f(*(addr+j)));
            } /* for */
        } /* if */
    } else if (*layout == CblasColMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i;
                for (j = 0; j < *n; j++)
                    printf("%6.2f  ", b2f(*(addr+j*(*lda))));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n          ");
                addr = a + i;
                for (j=0; j < i; j++)
                    printf("          ");
                for (j = i; j < *n; j++)
                    printf("%6.2f  ", b2f(*(addr+j*(*lda))));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n          ");
                addr = a + i;
                for (j = 0; j <= i; j++)
                    printf("%6.2f  ", b2f(*(addr+j*(*lda))));
            } /* for */
        } /* if */
    } /* if */
    return;
} /* PrintArrayBF16 */

void PrintArrayH(CBLAS_LAYOUT*layout, int flag1, int flag2, MKL_INT *m, MKL_INT *n, MKL_F16 *a,
                MKL_INT *lda, char *name)

```

```

{
    MKL_INT i, j;
    MKL_F16 *addr;

    switch(flag1) {
    case 0:
        printf("\n      ARRAY %s      LD%s=" INT_FORMAT, name, name, *lda);
        break;
    case 1:
        printf("\n      ARRAY %s", name);
        break;
    default:
        break;
    } /* switch */
    if (*layout == CblasRowMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n      ");
                addr = a + i*(*lda);
                for (j = 0; j < *n; j++)
                    printf("%6.2f  ", h2f(*(addr+j)));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n      ");
                addr = a + i*(*lda);
                for (j=0; j < i; j++)
                    printf("      ");
                for (j = i; j < *n; j++)
                    printf("%6.2f  ", h2f(*(addr+j)));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n      ");
                addr = a + i*(*lda);
                for (j = 0; j <= i; j++)
                    printf("%6.2f  ", h2f(*(addr+j)));
            } /* for */
        } /* if */
    } else if (*layout == CblasColMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n      ");
                addr = a + i;
                for (j = 0; j < *n; j++)
                    printf("%6.2f  ", h2f(*(addr+j*(*lda))));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n      ");
                addr = a + i;
                for (j=0; j < i; j++)
                    printf("      ");
                for (j = i; j < *n; j++)
                    printf("%6.2f  ", h2f(*(addr+j*(*lda))));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < (*m); i++) {

```

```

        printf("\n          ");
        addr = a + i;
        for (j = 0; j <= i; j++)
            printf("%6.2f ", h2f(*(addr+j*(*lda))));
    } /* for */
} /* if */
} /* if */
return;
} /* PrintArrayH */

void PrintArrayS(CBLAS_LAYOUT*layout, int flag1, int flag2, MKL_INT *m, MKL_INT *n, float *a,
                MKL_INT *lda, char *name)
{
    MKL_INT i, j;
    float *addr;

    switch(flag1) {
    case 0:
        printf("\n          ARRAY %s  LD%s=" INT_FORMAT, name, name, *lda);
        break;
    case 1:
        printf("\n          ARRAY %s", name);
        break;
    default:
        break;
    } /* switch */
    if (*layout == CblasRowMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j = 0; j < *n; j++)
                    printf("%6.2f ", *(addr+j));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < (*m); i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j=0; j < i; j++)
                    printf("          ");
                for (j = i; j < *n; j++)
                    printf("%6.2f ", *(addr+j));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j = 0; j <= i; j++)
                    printf("%6.2f ", *(addr+j));
            } /* for */
        } /* if */
    } else if (*layout == CblasColMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i;
                for (j = 0; j < *n; j++)
                    printf("%6.2f ", *(addr+j*(*lda)));
            }
        }
    }
}

```



```

        } /* for */
    } else if (flag2 == UPPER_MATRIX) {
        for (i = 0; i < (*m); i++) {
            printf("\n          ");
            addr = a + i;
            for (j=0; j < i; j++)
                printf("          ");
            for (j = i; j < *n; j++)
                printf("%6.2f ", *(addr+j*(*lda)));
        } /* for */
    } else if (flag2 == LOWER_MATRIX) {
        for (i = 0; i < (*m); i++) {
            printf("\n          ");
            addr = a + i;
            for (j = 0; j <= i; j++)
                printf("%6.2f ", *(addr+j*(*lda)));
        } /* for */
    } /* if */
} /* if */
return;
} /* PrintArrayS */

void PrintArrayD(CBLAS_LAYOUT*layout, int flag1, int flag2, MKL_INT *m, MKL_INT *n, double *a,
                MKL_INT *lda, char *name)
{
    MKL_INT i, j;
    double *addr;

    switch(flag1) {
    case 0:
        printf("\n          ARRAY %s  LD%s=" INT_FORMAT,
               name, name, *lda);
        break;
    case 1:
        printf("\n          ARRAY %s", name);
        break;
    default:
        break;
    } /* switch */
    if (*layout == CblasRowMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j = 0; j < *n; j++)
                    printf("%8.3f ", *(addr+j));
            } /* for */
        } else if (flag2 == UPPER_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);
                for (j=0; j < i; j++)
                    printf("          ");
                for (j = i; j < *n; j++)
                    printf("%8.3f ", *(addr+j));
            } /* for */
        } else if (flag2 == LOWER_MATRIX) {
            for (i = 0; i < *m; i++) {

```

```

        printf("\n          ");
        addr = a + i*(*lda);
        for (j = 0; j <= i; j++)
            printf("%8.3f  ", *(addr+j));
    } /* for */
} /* if */
} else if (*layout == CblasColMajor) {
    if (flag2 == GENERAL_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n          ");
            addr = a + i;
            for (j = 0; j < *n; j++)
                printf("%8.3f  ", *(addr+j*(*lda)));
        } /* for */
    } else if (flag2 == UPPER_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n          ");
            addr = a + i;
            for (j=0; j < i; j++)
                printf("          ");
            for (j = i; j < *n; j++)
                printf("%8.3f  ", *(addr+j*(*lda)));
        } /* for */
    } else if (flag2 == LOWER_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n          ");
            addr = a + i;
            for (j = 0; j <= i; j++)
                printf("%8.3f  ", *(addr+j*(*lda)));
        } /* for */
    } /* if */
} /* if */
return;
} /* PrintArrayD */

void PrintArrayC(CBLAS_LAYOUT*layout, int flag1, int flag2, MKL_INT *m, MKL_INT *n, MKL_Complex8
*a,
                MKL_INT *lda, char *name)
{
    MKL_INT i, j;
    MKL_Complex8 *addr;

    switch(flag1) {
    case 0:
        printf("\n          ARRAY %s  LD%s=" INT_FORMAT,
               name, name, *lda);
        break;
    case 1:
        printf("\n          ARRAY %s", name);
        break;
    default:
        break;
    } /* switch */
    if (*layout == CblasRowMajor) {
        if (flag2 == GENERAL_MATRIX) {
            for (i = 0; i < *m; i++) {
                printf("\n          ");
                addr = a + i*(*lda);

```

```

        for (j = 0; j < *n; j++)
            printf("(%.2f,%.2f)  ", (addr+j)->real, (addr+j)->imag);
    } /* for */
} else if (flag2 == UPPER_MATRIX) {
    for (i = 0; i < *m; i++) {
        printf("\n          ");
        addr = a + i*(*lda);
        for (j=0; j < i; j++)
            printf("          ");
        for (j = i; j < *n; j++)
            printf("(%.2f,%.2f)  ", (addr+j)->real, (addr+j)->imag);
    } /* for */
} else if (flag2 == LOWER_MATRIX) {
    for (i = 0; i < *m; i++) {
        printf("\n          ");
        addr = a + i*(*lda);
        for (j = 0; j <= i; j++)
            printf("(%.2f,%.2f)  ", (addr+j)->real, (addr+j)->imag);
    } /* for */
} /* if */
} else if(*layout == CblasColMajor) {
    if (flag2 == GENERAL_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n          ");
            addr = a + i;
            for (j = 0; j < *n; j++)
                printf("(%.2f,%.2f)  ", (addr+j*(*lda))->real,
                    (addr+j*(*lda))->imag);
        } /* for */
    } else if (flag2 == UPPER_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n          ");
            addr = a + i;
            for (j=0; j < i; j++)
                printf("          ");
            for (j = i; j < *n; j++)
                printf("(%.2f,%.2f)  ", (addr+j*(*lda))->real,
                    (addr+j*(*lda))->imag);
        } /* for */
    } else if (flag2 == LOWER_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n          ");
            addr = a + i;
            for (j = 0; j <= i; j++)
                printf("(%.2f,%.2f)  ", (addr+j*(*lda))->real,
                    (addr+j*(*lda))->imag);
        } /* for */
    } /* if */
} /* if */
return;
} /* PrintArrayC */

void PrintArrayZ(CBLAS_LAYOUT*layout, int flag1, int flag2,
                MKL_INT *m, MKL_INT *n, MKL_Complex16 *a, MKL_INT *lda, char *name)
{
    MKL_INT    i, j;
    MKL_Complex16 *addr;

```

```

if (flag1 == 0)
    printf("\n      ARRAY %s      LD%s=" INT_FORMAT, name, name, *lda);
else
    printf("\n      ARRAY %s", name);

if (*layout == CblasRowMajor) {
    if (flag2 == GENERAL_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n      ");
            addr = a + i*(*lda);
            for (j = 0; j < *n; j++)
                printf("(%.2f,%.2f)  ", (addr+j)->real,
                    (addr+j)->imag);
        } /* for */
    } else if (flag2 == UPPER_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n      ");
            addr = a + i*(*lda);
            for (j=0; j < i; j++)
                printf("                ");
            for (j = i; j < *n; j++)
                printf("(%.2f,%.2f)  ", (addr+j)->real,
                    (addr+j)->imag);
        } /* for */
    } else if (flag2 == LOWER_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n      ");
            addr = a + i*(*lda);
            for (j = 0; j <= i; j++)
                printf("(%.2f,%.2f)  ", (addr+j)->real,
                    (addr+j)->imag);
        } /* for */
    } /* if */
} else if (*layout == CblasColMajor) {
    if (flag2 == GENERAL_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n      ");
            addr = a + i;
            for (j = 0; j < *n; j++)
                printf("(%.2f,%.2f)  ", (addr+j*(*lda))->real,
                    (addr+j*(*lda))->imag);
        } /* for */
    } else if (flag2 == UPPER_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n      ");
            addr = a + i;
            for (j=0; j < i; j++)
                printf("                ");
            for (j = i; j < *n; j++)
                printf("(%.2f,%.2f)  ", (addr+j*(*lda))->real,
                    (addr+j*(*lda))->imag);
        } /* for */
    } else if (flag2 == LOWER_MATRIX) {
        for (i = 0; i < *m; i++) {
            printf("\n      ");
            addr = a + i;
            for (j = 0; j <= i; j++)
                printf("(%.2f,%.2f)  ", (addr+j*(*lda))->real,

```

```

        (addr+j*(*lda))->imag);
    } /* for */
  } /* if */
} /* if */
return;
} /* PrintArrayZ */

void PrintBandArraysS(CBLAS_LAYOUT*layout, int flag,
    MKL_INT kl, MKL_INT ku, MKL_INT m, MKL_INT n,
    float *a, MKL_INT lda, char *name)
{
    MKL_INT i, j, l;
    float *addr, *addr1;
    MKL_INT kl1, ku1, i_start, j_start, j_end, kl_rows, ku_rows;

    if (flag == 0)
        printf("\n      ARRAY %s  LD%s=" INT_FORMAT "  KL=" INT_FORMAT "  KU=" INT_FORMAT,
            name, name, lda, kl, ku);
    else
        printf("\n      ARRAY %s  LD%s=" INT_FORMAT, name, name, lda);

    if (*layout == CblasRowMajor) {
        for( i = 0; i < MIN( m, n ); i++ ) {
            printf("\n      ");
            addr = a + i*lda;
            kl1 = ( i - kl <= 0 ) ? i : kl;
            ku1 = ( i + ku >= n ) ? MAX(0,n-i-1) : ku;
            j_start = kl - kl1;
            j_end = j_start + kl1 + ku1;
            addr1 = addr + j_start;
            for( l=0; l < j_start; l++ )
                printf("      ");
            for( l=0; l < j_end-j_start+1; l++ )
                printf("%6.2f ", *(addr1+l));
        }
        for( i = MIN( m, n ); i < MIN( m, MIN( m, n ) + kl ); i++ ) {
            printf("\n      ");
            addr = a + i*lda;
            kl1 = n - i + kl;
            j_start = ( kl > n ) ? kl - n : n - kl;
            j_end = j_start + kl1 - 1;
            addr1 = addr + j_start;
            for( l=0; l < j_start; l++ )
                printf("      ");
            for( l=0; l < j_end-j_start+1; l++ )
                printf("%6.2f ", *(addr1+l));
        }
    } else if (*layout == CblasColMajor) {
        i_start = (ku > n) ? ku - n + 1 : 0;
        ku_rows = (ku > n) ? n - 1 : ku;
        j_start = ku_rows;
        for( i = 0; i < ku_rows; i++ ) {
            printf("\n      ");
            j = j_start*lda; addr1 = a + i + i_start;
            for( l=0; l < j_start; l++ )
                printf("      ");
            for( l=0; l < n-ku_rows; l++ )
                printf("%6.2f ", *(addr1+j+l*lda) );
        }
    }
}

```

```

        j_start--;
    }

    j_end = MIN(m,n);
    addr1 = a + ku;
    printf("\n      ");
    for( l=0; l < j_end; l++ )
        printf("%6.2f ", *(addr1+l*lda) );

    kl_rows = ( kl <= m-1 ) ? kl : m - 1;
    for ( i = 1; i < kl_rows+1; i++ ) {
        printf("\n      ");
        kll = ( i+j_end <= m ) ? j_end : m - i;
        addr1 = a + ku + i;
        for( l=0; l < kll; l++ )
            printf("%6.2f ", *(addr1+l*lda) );
    }
} /* if */
return;
} /* PrintBandArrayS */

void PrintBandArrayD(CBLAS_LAYOUT*layout, int flag,
                    MKL_INT kl, MKL_INT ku, MKL_INT m, MKL_INT n,
                    double *a, MKL_INT lda, char *name)
{
    MKL_INT i, j, l;
    double *addr, *addr1;
    MKL_INT kll, kul, i_start, j_start, j_end, kl_rows, ku_rows;

    if (flag == 0)
        printf("\n      ARRAY %s  LD%s=" INT_FORMAT "  KL=" INT_FORMAT "  KU=" INT_FORMAT,
               name, name, lda, kl, ku);
    else
        printf("\n      ARRAY %s  LD%s=" INT_FORMAT, name, name, lda);

    if (*layout == CblasRowMajor) {
        for( i = 0; i < MIN( m, n ); i++ ) {
            printf("\n      ");
            addr = a + i*lda;
            kll = ( i - kl <= 0 ) ? i : kl;
            kul = ( i + ku >= n ) ? MAX(0,n-i-1) : ku;
            j_start = kl - kll;
            j_end = j_start + kll + kul;
            addr1 = addr + j_start;
            for( l=0; l < j_start; l++ )
                printf("      ");
            for( l=0; l < j_end-j_start+1; l++ )
                printf("%6.2f ", *(addr1+l));
        }
        for( i = MIN( m, n ); i < MIN( m, MIN( m, n ) + kl ); i++ ) {
            printf("\n      ");
            addr = a + i*lda;
            kll = n - i + kl;
            j_start = ( kl > n ) ? kl - n : n - kl;
            j_end = j_start + kll - 1;
            addr1 = addr + j_start;
            for( l=0; l < j_start; l++ )
                printf("      ");

```

```

        for( l=0; l < j_end-j_start+1; l++ )
            printf("%6.2f ", *(addr1+l));
    }
} else if (*layout == CblasColMajor) {
    i_start = (ku > n) ? ku - n + 1 : 0;
    ku_rows = (ku > n) ? n - 1 : ku;
    j_start = ku_rows;
    for( i = 0; i < ku_rows; i++ ) {
        printf("\n      ");
        j = j_start*lda; addr1 = a + i + i_start;
        for( l=0; l < j_start; l++ )
            printf("      ");
        for( l=0; l < n-ku_rows; l++ )
            printf("%6.2f ", *(addr1+j+l*lda) );
        j_start--;
    }

    j_end = MIN(m,n);
    addr1 = a + ku;
    printf("\n      ");
    for( l=0; l < j_end; l++ )
        printf("%6.2f ", *(addr1+l*lda) );

    kl_rows = ( kl <= m-1 ) ? kl : m - 1;
    for ( i = 1; i < kl_rows+1; i++ ) {
        printf("\n      ");
        kll = ( i+j_end <= m ) ? j_end : m - i;
        addr1 = a + ku + i;
        for( l=0; l < kll; l++ )
            printf("%6.2f ", *(addr1+l*lda) );
    }
} /* if */
return;
} /* PrintBandArrayD */

void PrintBandArrayC(CBLAS_LAYOUT*layout, int flag,
                    MKL_INT kl, MKL_INT ku, MKL_INT m, MKL_INT n,
                    MKL_Complex8 *a, MKL_INT lda, char *name)
{
    MKL_INT      i, j, l;
    MKL_Complex8 *addr, *addr1;
    MKL_INT      kll, kul, j_start, j_end, i_start, kl_rows, ku_rows;

    if (flag == 0)
        printf("\n      ARRAY %s  LD%s=" INT_FORMAT "  KL=" INT_FORMAT "  KU=" INT_FORMAT,
               name, name, lda, kl, ku);
    else
        printf("\n      ARRAY %s  LD%s=" INT_FORMAT, name, name, lda);

    if (*layout == CblasRowMajor) {
        for( i = 0; i < MIN( m, n ); i++ ) {
            printf("\n      ");
            addr = a + i*lda;
            kll = ( i - kl <= 0 ) ? i : kl;
            kul = ( i + ku >= n ) ? MAX(0,n-i-1) : ku;
            j_start = kl - kll;
            j_end = j_start + kll + kul;
            addr1 = addr + j_start;

```

```

        for( l=0; l < j_start; l++ )
            printf("                ");
        for( l=0; l < j_end-j_start+1; l++ )
            printf("(%.2f,%.2f) ", (addr1+l)->real, (addr1+l)->imag);
    }
    for( i = MIN( m, n ); i < MIN( m, MIN( m, n ) + kl); i++ ) {
        printf("\n            ");
        addr = a + i*lda;
        kl1 = n - i + kl;
        j_start = ( kl > n ) ? kl - n : n - kl;
        j_end = j_start + kl1 - 1;
        addr1 = addr + j_start;
        for( l=0; l < j_start; l++ )
            printf("                ");
        for( l=0; l < j_end-j_start+1; l++ )
            printf("(%.2f,%.2f) ", (addr1+l)->real, (addr1+l)->imag);
    }
} else if (*layout == CblasColMajor) {
    i_start = (ku > n) ? ku - n + 1 : 0;
    ku_rows = (ku > n) ? n - 1 : ku;
    j_start = ku_rows;
    for( i = 0; i < ku_rows; i++ ) {
        printf("\n            ");
        j = j_start*lda; addr1 = a + i + i_start;
        for( l=0; l < j_start; l++ )
            printf("                ");
        for( l=0; l < n-ku_rows; l++ )
            printf("(%.2f,%.2f) ",
                    (addr1+j+l*lda)->real, (addr1+j+l*lda)->imag );
        j_start--;
    }

    j_end = MIN(m,n);
    addr1 = a + ku;
    printf("\n            ");
    for( l=0; l < j_end; l++ )
        printf("(%.2f,%.2f) ",
                (addr1+l*lda)->real, (addr1+l*lda)->imag );

    kl_rows = ( kl <= m-1 ) ? kl : m - 1;
    for ( i = 1; i < kl_rows+1; i++ ) {
        printf("\n            ");
        kl1 = ( i+j_end <= m ) ? j_end : m - i;
        addr1 = a + ku + i;
        for( l=0; l < kl1; l++ )
            printf("(%.2f,%.2f) ",
                    (addr1+l*lda)->real, (addr1+l*lda)->imag );
    }
} /* if */
return;
} /* PrintBandArrayC */

void PrintBandArrayZ(CBLAS_LAYOUT*layout, int flag,
                    MKL_INT kl, MKL_INT ku, MKL_INT m, MKL_INT n,
                    MKL_Complex16 *a, MKL_INT lda, char *name)
{
    MKL_INT i, j, l;
    MKL_Complex16 *addr, *addr1;

```



```

MKL_INT      kl1, kul, j_start, j_end, i_start, kl_rows, ku_rows;

if (flag == 0)
    printf("\n      ARRAY %s    LD%s=" INT_FORMAT "    KL=" INT_FORMAT "    KU=" INT_FORMAT,
           name, name, lda, kl, ku);
else
    printf("\n      ARRAY %s    LD%s=" INT_FORMAT, name, name, lda);

if (*layout == CblasRowMajor) {
    for( i = 0; i < MIN( m, n ); i++ ) {
        printf("\n      ");
        addr = a + i*lda;
        kl1 = ( i - kl <= 0 ) ? i : kl;
        kul = ( i + ku >= n ) ? MAX(0,n-i-1) : ku;
        j_start = kl - kl1;
        j_end = j_start + kl1 + kul;
        addr1 = addr + j_start;
        for( l=0; l < j_start; l++ )
            printf("      ");
        for( l=0; l < j_end-j_start+1; l++ )
            printf("(%6.2f,%6.2f)  ", (addr1+l)->real, (addr1+l)->imag);
    }
    for( i = MIN( m, n ); i < MIN( m, MIN( m, n ) + kl ); i++ ) {
        printf("\n      ");
        addr = a + i*lda;
        kl1 = n - i + kl;
        j_start = ( kl > n ) ? kl - n : n - kl;
        j_end = j_start + kl1 - 1;
        addr1 = addr + j_start;
        for( l=0; l < j_start; l++ )
            printf("      ");
        for( l=0; l < j_end-j_start+1; l++ )
            printf("(%6.2f,%6.2f)  ", (addr1+l)->real, (addr1+l)->imag);
    }
} else if (*layout == CblasColMajor) {
    i_start = (ku > n) ? ku - n + 1 : 0;
    ku_rows = (ku > n) ? n - 1 : ku;
    j_start = ku_rows;
    for( i = 0; i < ku_rows; i++ ) {
        printf("\n      ");
        j = j_start*lda; addr1 = a + i + i_start;
        for( l=0; l < j_start; l++ )
            printf("      ");
        for( l=0; l < n-ku_rows; l++ )
            printf("(%6.2f,%6.2f)  ",
                   (addr1+j+l*lda)->real, (addr1+j+l*lda)->imag );
        j_start--;
    }

    j_end = MIN(m,n);
    addr1 = a + ku;
    printf("\n      ");
    for( l=0; l < j_end; l++ )
        printf("(%6.2f,%6.2f)  ",
               (addr1+l*lda)->real, (addr1+l*lda)->imag );

    kl_rows = ( kl <= m-1 ) ? kl : m - 1;
    for ( i = 1; i < kl_rows+1; i++ ) {

```

```

        printf("\n      ");
        k11 = ( i+j_end <= m ) ? j_end : m - i;
        addr1 = a + ku + i;
        for( l=0; l < k11; l++ )
            printf("(%.2f,%.2f)  ",
                (addr1+l*lda)->real, (addr1+l*lda)->imag );
    }
} /* if */
return;
} /* PrintBandArrayZ */

```

cblas_sgemmx.c

```

/*****
* Copyright 1999-2022 Intel Corporation.
*
* This software and the related documents are Intel copyrighted materials, and
* your use of them is governed by the express license under which they were
* provided to you (License). Unless the License provides otherwise, you may not
* use, modify, copy, publish, distribute, disclose or transmit this software or
* the related documents without Intel's prior written permission.
*
* This software and the related documents are provided as is, with no express
* or implied warranties, other than those that are expressly stated in the
* License.
*****/

/*
! Content:
!   C B L A S _ S G E M M Example Program Text ( C Interface )
!*****/

#include <stdio.h>
#include <stdlib.h>
#include "mkl.h"

#include "mkl_example.h"

int main(int argc, char *argv[])
{
    FILE *in_file;
    char *in_file_name;

    MKL_INT      m, n, k;
    MKL_INT      lda, ldb, ldc;
    MKL_INT      rmaxa, cmaxa, rmaxb, cmAXB, rmaxc, cmAXc;
    float        alpha, beta;
    float        *a, *b, *c;
    CBLAS_LAYOUT layout;
    CBLAS_TRANSPOSE transA, transB;
    MKL_INT      ma, na, mb, nb;

    printf("\n      C B L A S _ S G E M M  EXAMPLE PROGRAM\n");

    /*          Get input parameters          */

    if( argc == 1 ) {
        printf("\n You must specify in_file data file as 1-st parameter");
        return 1;
    }

```

```

    }
    in_file_name = argv[1];

/*      Get input data      */

    if( (in_file = fopen( in_file_name, "r" )) == NULL ) {
        printf("\n ERROR on OPEN '%s' with mode=\"%r\"\n", in_file_name);
        return 1;
    }
    if( GetIntegerParameters(in_file, &m, &n, &k) != 3 ) {
        printf("\n ERROR of m, n, k reading\n");
        fclose(in_file);
        return 1;
    }
    if( GetScalarsS(in_file, &alpha, &beta) != 2 ) {
        printf("\n ERROR of alpha, beta reading\n");
        fclose(in_file);
        return 1;
    }
    if( GetCblasCharParameters(in_file, &transA, &transB, &layout) != 3 ) {
        printf("\n ERROR of transA, transB, layout reading\n");
        fclose(in_file);
        return 1;
    }

    if( transA == CblasNoTrans ) {
        rmaxa = m + 1;
        cmaxa = k;
        ma    = m;
        na    = k;
    } else {
        rmaxa = k + 1;
        cmaxa = m;
        ma    = k;
        na    = m;
    }
    if( transB == CblasNoTrans ) {
        rmaxb = k + 1;
        cmaxb = n;
        mb    = k;
        nb    = n;
    } else {
        rmaxb = n + 1;
        cmaxb = k;
        mb    = n;
        nb    = k;
    }
    rmaxc = m + 1;
    cmaxc = n;
    a = (float *)mkl_calloc(rmaxa*cmaxa, sizeof( float ), 64);
    b = (float *)mkl_calloc(rmaxb*cmaxb, sizeof( float ), 64);
    c = (float *)mkl_calloc(rmaxc*cmaxc, sizeof( float ), 64);
    if( a == NULL || b == NULL || c == NULL ) {
        printf( "\n Can't allocate memory for arrays\n");
        mkl_free(a);
        mkl_free(b);
        mkl_free(c);
        fclose(in_file);
    }

```

```

        return 1;
    }
    if (layout == CblasRowMajor) {
        lda=cmaxa;
        ldb=cmaxb;
        ldc=cmaxc;
    } else {
        lda=rmaxa;
        ldb=rmaxb;
        ldc=rmaxc;
    }
    if( GetArrayS(in_file, &layout, GENERAL_MATRIX, &ma, &na, a, &lda) != 0 ) {
        printf("\n ERROR of array A reading\n");
        mkl_free(a);
        mkl_free(b);
        mkl_free(c);
        fclose(in_file);
        return 1;
    }
    if( GetArrayS(in_file, &layout, GENERAL_MATRIX, &mb, &nb, b, &ldb) != 0 ) {
        printf("\n ERROR of array B reading\n");
        mkl_free(a);
        mkl_free(b);
        mkl_free(c);
        fclose(in_file);
        return 1;
    }
    if( GetArrayS(in_file, &layout, GENERAL_MATRIX, &m, &n, c, &ldc) != 0 ) {
        printf("\n ERROR of array C reading\n");
        mkl_free(a);
        mkl_free(b);
        mkl_free(c);
        fclose(in_file);
        return 1;
    }
    fclose(in_file);

/*      Print input data                                     */

    printf("\n      INPUT DATA");
    printf("\n      M="INT_FORMAT" N="INT_FORMAT" K="INT_FORMAT, m, n, k);
    printf("\n      ALPHA=%5.1f BETA=%5.1f", alpha, beta);
    PrintParameters("TRANSA, TRANSB", transA, transB);
    PrintParameters("LAYOUT", layout);
    PrintArrayS(&layout, FULLPRINT, GENERAL_MATRIX, &ma, &na, a, &lda, "A");
    PrintArrayS(&layout, FULLPRINT, GENERAL_MATRIX, &mb, &nb, b, &ldb, "B");
    PrintArrayS(&layout, FULLPRINT, GENERAL_MATRIX, &m, &n, c, &ldc, "C");

/*      Call SGEMM subroutine ( C Interface )               */

    cblas_sgemm(layout, transA, transB, m, n, k, alpha,
                a, lda, b, ldb, beta, c, ldc);

/*      Print output data                                    */

    printf("\n\n      OUTPUT DATA");
    PrintArrayS(&layout, FULLPRINT, GENERAL_MATRIX, &m, &n, c, &ldc, "C");

```

```

    mkl_free(a);
    mkl_free(b);
    mkl_free(c);

    return 0;
}

```

cblas_sgemmx.d

```

: C B L A S _ S G E M M  EXAMPLE PROGRAM DATA
3 5 4                      :Values of m, n, k
0.5 1.2                    :Values of alpha, beta
111 111 101                :Values of transA, transB, layout
1.0  2.0  3.0  4.0
3.2  2.7  8.1  5.0
0.0  4.0  5.0  3.0      :Values of array A
1.0  2.0  3.0  4.0  5.0
1.0  2.0  3.0  4.0  5.0
1.0  2.0  3.0  4.0  5.0
1.0  2.0  3.0  4.0  5.0      :Values of array B
0.0  0.0  1.0  1.0  1.0
0.0  0.0  1.0  1.0  1.0
0.0  0.0  1.0  1.0  1.0      :Values of array C

```

CMakeLists.txt

```

=====
# Copyright 2023 Intel Corporation.
#
# This software and the related documents are Intel copyrighted materials, and
# your use of them is governed by the express license under which they were
# provided to you (License). Unless the License provides otherwise, you may not
# use, modify, copy, publish, distribute, disclose or transmit this software or
# the related documents without Intel's prior written permission.
#
# This software and the related documents are provided as is, with no express
# or implied warranties, other than those that are expressly stated in the
# License.
=====

cmake_minimum_required(VERSION 3.13)
enable_testing()

# Define language and file extensions
set(TEST_LANG C)
set(TEST_EXT c)
set(DATA_EXT d)

project(MKL_Example LANGUAGES ${TEST_LANG})

# Include MKLConfig.cmake
find_package(MKL CONFIG REQUIRED)

# Build extra files
add_library(blas_common common_func.c)
if(WIN32)
    set(TEST_COPT "-D_CRT_SECURE_NO_WARNINGS")
endif()

# Add some parts of MKL target: headers and compiler options

```

```
target_include_directories(blas_common PUBLIC
$<TARGET_PROPERTY:MKL::MKL,INTERFACE_INCLUDE_DIRECTORIES>)
target_compile_options(blas_common PUBLIC ${TEST_COPT})
$<TARGET_PROPERTY:MKL::MKL,INTERFACE_COMPILE_OPTIONS>)
set(TEST_LOPT blas_common)

# Build target
set(func "example")
set(func_name "cblas_sgemmx")
set(${func}_SRC ${func_name}.${TEST_EXT})

add_executable(${func} ${${func}_SRC})
set_property(TARGET ${func} PROPERTY C_STANDARD 99)
# Add MKL target: headers, compiler options and libraries to link
target_link_libraries(${func} PUBLIC ${TEST_LOPT} MKL::MKL)

# Register example as ctest
set(${func_name}_DATA ${PROJECT_SOURCE_DIR}/${func_name}.${DATA_EXT})
add_test(NAME ${func} COMMAND ${func} ${${func_name}_DATA})

# Add oneMKL environment variables if any defined
if(MKL_ENV)
    set_tests_properties(${func} PROPERTIES ENVIRONMENT "${MKL_ENV}")
endif()
```

ScaLAPACK Example

matrix_generator.h

```
/*
*****
* Copyright 2022 Intel Corporation.
*
* This software and the related documents are Intel copyrighted materials, and
* your use of them is governed by the express license under which they were
* provided to you (License). Unless the License provides otherwise, you may not
* use, modify, copy, publish, distribute, disclose or transmit this software or
* the related documents without Intel's prior written permission.
*
* This software and the related documents are provided as is, with no express
* or implied warranties, other than those that are expressly stated in the
* License.
*****
/*
! Content:
! Intel(R) oneAPI Math Kernel Library ScaLAPACK C example source file
!
! *****
! =====
! Matrix generator routines for ScaLAPACK p?getrf and p?getrs example program
! =====
! List of routines are given in the file:
!
! pdmatgen_random
! psmatgen_random
! pzmatgen_random
! psmatgen_random
!
! Definitions for element index numbers of ScaLAPACK array descriptors
```

```

*/
#include "mkl.h"
#include "mkl_blacs.h"
#include "mkl_pblas.h"
#include "mkl_scalapack.h"

#define DTYPE_ 1
#define CXTT_ 2
#define M_ 3
#define N_ 4
#define MB_ 5
#define NB_ 6
#define RSRC_ 7
#define CSRC_ 8
#define LLD_ 9
/* Additional definitions */
#ifndef min
#define min(a, b) ((a) < (b) ? (a) : (b))
#endif
#ifndef max
#define max(a, b) ((a) > (b) ? (a) : (b))
#endif
/* Local real and complex constants */
#define RZERO 0.0
#define RONE 1.0
#define RNEGONE -1.0
#define CZERO {0.0f, 0.0f}
#define CONE {1.0f, 0.0f}
#define CNEGONE {-1.0f, 0.0f}

const MKL_INT izero = 0, ione = 1, isix = 6, inegone = -1, dist=1;

void pdmatgen_random(MKL_INT type, MKL_INT* desca, double* a, MKL_INT* seed, MKL_INT nprow,
MKL_INT npcol)
/*
* Purpose
* =====
*
* pdmatgen_random: parallel real double precision matrix generator.
* Generate a sub-matrix of A.
*
* Arguments
* =====
*
* type      (global input)
*           Type of global matrix to be generated.
*           If type==0, local matrices are parts of the matrix
*           which possesses 1 as all elements along its leading (main) diagonal;
*           while all other elements in the matrix are 0.
*           A random matrix is generated if type != 0.
*
* desca     (global and local input) an array of (at least) length 9.
*           The array descriptor for the distributed matrix A.
*           See the notes below for the description of each element.
*
* a         (local output), pointer into the local
*           memory to an array containing the
*           local pieces of the distributed matrix.

```

```

*
* seed      (local input and output) integer array, dimension (4)
*            On entry, the seed of the random number generator.
*            On exit, the seed is updated.
*
* nprow     (global input)
*            The number of process rows in the grid.
*
* npcol     (global input)
*            The number of process columns in the grid.
*
* Notes:
* =====
*
* Each global data object is described by an associated description
* vector. This vector stores the information required to establish
* the mapping between an object element and its corresponding process
* and memory location.
*
*
* In the following comments, the character _ should be read as
* "of the distributed matrix". Let A be a generic term for any 2D
* block cyclicly distributed matrix. Its description vector is DESC_A:
*
* NOTATION      STORED IN      EXPLANATION
* -----
* DTYPE_A (global) DESC_A[ DTYPE_ ] The descriptor type.
* CTXT_A   (global) DESC_A[ CTXT_ ] The BLACS context handle, indicating
*                                   the NPROW x NPCOL BLACS process grid
*                                   A is distributed over. The context
*                                   itself is global, but the handle
*                                   (the integer value) may vary.
* M_A      (global) DESC_A[ M_ ]   The number of rows in the distribu-
*                                   ted matrix A, M_A >= 0.
* N_A      (global) DESC_A[ N_ ]   The number of columns in the distri-
*                                   buted matrix A, N_A >= 0.
* IMB_A    (global) DESC_A[ IMB_ ] The number of rows of the upper left
*                                   block of the matrix A, IMB_A > 0.
* INB_A    (global) DESC_A[ INB_ ] The number of columns of the upper
*                                   left block of the matrix A,
*                                   INB_A > 0.
* MB_A     (global) DESC_A[ MB_ ]  The blocking factor used to distri-
*                                   bute the last M_A-IMB_A rows of A,
*                                   MB_A > 0.
* NB_A     (global) DESC_A[ NB_ ]  The blocking factor used to distri-
*                                   bute the last N_A-INB_A columns of
*                                   A, NB_A > 0.
* RSRC_A   (global) DESC_A[ RSRC_ ] The process row over which the first
*                                   row of the matrix A is distributed,
*                                   NPROW > RSRC_A >= 0.
* CSRC_A   (global) DESC_A[ CSRC_ ] The process column over which the
*                                   first column of A is distributed.
*                                   NPCOL > CSRC_A >= 0.
* LLD_A    (local)  DESC_A[ LLD_ ] The leading dimension of the local
*                                   array storing the local blocks of
*                                   the distributed matrix A,
*                                   IF( Lc( 1, N_A ) > 0 )

```



```

*           LLD_A >= MAX( 1, Lr( 1, M_A ) )
*           ELSE
*           LLD_A >= 1.
*/
{
    MKL_INT myrow, mycol;
    MKL_INT iarow = 0, iacol = 0;
    MKL_INT ictxt = desca[CTXT-1];
    MKL_INT m      = desca[M-1];
    MKL_INT n      = desca[N-1];
    MKL_INT mb     = desca[MB-1];
    MKL_INT nb     = desca[NB-1];
    MKL_INT lld    = desca[LLD-1];
    MKL_INT i, j, ii, jj, il = 0, jl = 0;

    blacs_gridinfo(&ictxt, &nprow, &npcol, &myrow, &mycol);
    for (j = 0; j < n; j += nb) {
        for (i = 0; i < m; i += mb) {
            i++; j++;
            infog2l(&i, &j, desca, &nprow, &npcol, &myrow, &mycol, &il, &jl, &iarow, &iacol);
            i--; j--; il--; jl--;
            if (iarow==myrow && iacol==mycol) {
                for (jj = 0; jj < min(nb, n - j); jj++) {
                    for (ii = 0; ii < min(mb, m - i); ii++) {
                        if(type) {
                            a[il+ii+(jl+jj)*lld] = (double) dlarnd( &dist, seed );
                        } else {
                            if(i==j) a[il+ii+(jl+jj)*lld] = RONE;
                            else a[il+ii+(jl+jj)*lld] = RZERO;
                        }
                    }
                }
            }
        }
    }
}

void psmatgen_random(MKL_INT type, MKL_INT* desca, float* a, MKL_INT* seed, MKL_INT nprow,
MKL_INT npcil)
{
/*
* Purpose
* =====
*
* psmatgen_random: parallel real float precision matrix generator.
* Generate a sub-matrix of A.
*
* Arguments
* =====
*
* type      (global input)
*           Type of global matrix to be generated.
*           If type==0, local matrices are parts of the matrix
*           which possesses 1 as all elements along its leading (main) diagonal;
*           while all other elements in the matrix are 0.
*           A random matrix is generated if type != 0.
*
* desca     (global and local input) an array of (at least) length 9.

```

```

*      The array descriptor for the distributed matrix A.
*      See the notes below for the decription of each element.
*
*  a      (local output), pointer into the local
*          memory to a float array containing the
*          local pieces of the distributed matrix.
*
*  seed    (local input and output) integer array, dimension (4)
*          On entry, the seed of the random number generator.
*          On exit, the seed is updated.
*
*  nprow   (global input)
*          The number of process rows in the grid.
*
*  npcol   (global input)
*          The number of process columns in the grid.
*
*  Notes:
*  =====
*
*  Each global data object is described by an associated description
*  vector. This vector stores the information required to establish
*  the mapping between an object element and its corresponding process
*  and memory location.
*
*  In the following comments, the character _ should be read as
*  "of the distributed matrix". Let A be a generic term for any 2D
*  block cyclicly distributed matrix. Its description vector is DESC_A:
*
*  NOTATION      STORED IN      EXPLANATION
*  -----
*  DTYPE_A (global) DESC_A[ DTYPE_ ] The descriptor type.
*  CTXT_A   (global) DESC_A[ CTXT_ ] The BLACS context handle, indicating
*                                     the NPROW x NPCOL BLACS process grid
*                                     A is distributed over. The context
*                                     itself is global, but the handle
*                                     (the integer value) may vary.
*  M_A      (global) DESC_A[ M_ ] The number of rows in the distribu-
*                                   ted matrix A, M_A >= 0.
*  N_A      (global) DESC_A[ N_ ] The number of columns in the distri-
*                                   buted matrix A, N_A >= 0.
*  IMB_A    (global) DESC_A[ IMB_ ] The number of rows of the upper left
*                                   block of the matrix A, IMB_A > 0.
*  INB_A    (global) DESC_A[ INB_ ] The number of columns of the upper
*                                   left block of the matrix A,
*                                   INB_A > 0.
*  MB_A     (global) DESC_A[ MB_ ] The blocking factor used to distri-
*                                   bute the last M_A-IMB_A rows of A,
*                                   MB_A > 0.
*  NB_A     (global) DESC_A[ NB_ ] The blocking factor used to distri-
*                                   bute the last N_A-INB_A columns of
*                                   A, NB_A > 0.
*  RSRC_A   (global) DESC_A[ RSRC_ ] The process row over which the first
*                                   row of the matrix A is distributed,
*                                   NPROW > RSRC_A >= 0.
*  CSRC_A   (global) DESC_A[ CSRC_ ] The process column over which the

```

```

* first column of A is distributed.
* NPCOL > CSRC_A >= 0.
* LLD_A (local) DESCA[ LLD_ ] The leading dimension of the local
* array storing the local blocks of
* the distributed matrix A,
* IF( Lc( 1, N_A ) > 0 )
* LLD_A >= MAX( 1, Lr( 1, M_A ) )
* ELSE
* LLD_A >= 1.
*/

MKL_INT myrow, mycol;
MKL_INT iarow = 0, iacol = 0;
MKL_INT ictxt = desca[CTXT_-1];
MKL_INT m = desca[M_-1];
MKL_INT n = desca[N_-1];
MKL_INT mb = desca[MB_-1];
MKL_INT nb = desca[NB_-1];
MKL_INT lld = desca[LLD_-1];
MKL_INT i, j, ii, jj, il = 0, jl = 0;

blacs_gridinfo(&ictxt, &nprow, &npcol, &myrow, &mycol);
for (j = 0; j < n; j += nb) {
    for (i = 0; i < m; i += mb) {
        i++; j++;
        infog2l(&i, &j, desca, &nprow, &npcol, &myrow, &mycol, &il, &jl, &iarow, &iacol);
        i--; j--; il--; jl--;
        if ( iarow==myrow && iacol==mycol ) {
            for (jj = 0; jj < min(nb, n - j); jj++) {
                for (ii = 0; ii < min(mb, m - i); ii++) {
                    if( type ) {
                        a[il+ii+(jl+jj)*lld] = (float) dlarnd( &dist, seed );
                    } else {
                        if(i==j) a[il+ii+(jl+jj)*lld] = RONE;
                        else a[il+ii+(jl+jj)*lld] = RZERO;
                    }
                }
            }
        }
    }
}

void pzmatgen_random(MKL_INT type, MKL_INT* desca, MKL_Complex16* a, MKL_INT* seed, MKL_INT
nprow, MKL_INT npcol)
{
/*
* Purpose
* =====
*
* pzmatgen_random: parallel double complex precision matrix generator.
* Generate a sub-matrix of A.
*
* Arguments
* =====
*
* type (global input)
* Type of global matrix to be generated.

```

```

*      If type==0, local matrices are parts of the matrix
*      which possesses 1 as all elements along its leading (main) diagonal;
*      while all other elements in the matrix are 0.
*      Otherwise a random matrix is generated.

* type      (global input)
*      Type of global matrix to be generated.
*      If type==0, local matrices are parts of the identity matrix.
*      Otherwise a random matrix is generated.

* desca     (global and local input) an array of (at least) length 9.
*      The array descriptor for the distributed matrix A.
*      See the notes below for the decription of each element.

* a         (local output), pointer into the local
*      memory to a MKL_Complex16 array containing the
*      local pieces of the distributed matrix.

* seed      (local input and output) integer array, dimension (4)
*      On entry, the seed of the random number generator.
*      On exit, the seed is updated.

* nprow     (global input)
*      The number of process rows in the grid.

* npcol     (global input)
*      The number of process columns in the grid.

* Notes:
* =====
*
* Each global data object is described by an associated description
* vector. This vector stores the information required to establish
* the mapping between an object element and its corresponding process
* and memory location.
*
* In the following comments, the character _ should be read as
* "of the distributed matrix". Let A be a generic term for any 2D
* block cyclicly distributed matrix. Its description vector is DESC_A:
*
* NOTATION      STORED IN      EXPLANATION
* -----
* DTYPE_A (global) DESC_A[ DTYPE_ ] The descriptor type.
* CTXT_A   (global) DESC_A[ CTXT_ ] The BLACS context handle, indicating
*      the NPROW x NPCOL BLACS process grid
*      A is distributed over. The context
*      itself is global, but the handle
*      (the integer value) may vary.
* M_A       (global) DESC_A[ M_   ] The number of rows in the distribu-
*      ted matrix A, M_A >= 0.
* N_A       (global) DESC_A[ N_   ] The number of columns in the distri-
*      buted matrix A, N_A >= 0.
* IMB_A     (global) DESC_A[ IMB_ ] The number of rows of the upper left
*      block of the matrix A, IMB_A > 0.
* INB_A     (global) DESC_A[ INB_ ] The number of columns of the upper
*      left block of the matrix A,
*      INB_A > 0.
* MB_A      (global) DESC_A[ MB_  ] The blocking factor used to distri-

```

```
*
*       bute the last M_A-IMB_A rows of A,  
*       MB_A > 0.  
* NB_A      (global) DESCA[ NB_    ] The blocking factor used to distri-  
*       bute the last N_A-INB_A columns of  
*       A, NB_A > 0.  
* RSRC_A     (global) DESCA[ RSRC_  ] The process row over which the first  
*       row of the matrix A is distributed,  
*       NPROW > RSRC_A >= 0.  
* CSRC_A     (global) DESCA[ CSRC_  ] The process column over which the  
*       first column of A is distributed.  
*       NPCOL > CSRC_A >= 0.  
* LLD_A      (local)  DESCA[ LLD_   ] The leading dimension of the local  
*       array storing the local blocks of  
*       the distributed matrix A,  
*       IF( Lc( 1, N_A ) > 0 )  
*           LLD_A >= MAX( 1, Lr( 1, M_A ) )  
*       ELSE  
*           LLD_A >= 1.  
*/  
  
MKL_INT myrow, mycol;  
MKL_INT iarow = 0, iacol = 0;  
MKL_INT ictxt = desca[CTXT_-1];  
MKL_INT m      = desca[M_-1];  
MKL_INT n      = desca[N_-1];  
MKL_INT mb     = desca[MB_-1];  
MKL_INT nb     = desca[NB_-1];  
MKL_INT lld    = desca[LLD_-1];  
MKL_INT i, j, ii, jj, il = 0, jl = 0;  
  
blacs_gridinfo(&ictxt, &nproW, &npcol, &myrow, &mycol);  
for (j = 0; j < n; j += nb) {  
    for (i = 0; i < m; i += mb) {  
        i++; j++;  
        infog2l(&i, &j, desca, &nproW, &npcol, &myrow, &mycol, &il, &jl, &iarow, &iacol);  
        i--; j--; il--; jl--;  
        if (iarow==myrow && iacol==mycol) {  
            for (jj = 0; jj < min(nb, n - j); jj++) {  
                for (ii = 0; ii < min(mb, m - i); ii++) {  
                    if( type ) {  
                        a[i+ii+(jl+jj)*lld].real = (double) dlarnd( &dist, seed );  
                        a[i+ii+(jl+jj)*lld].imag = (double) dlarnd( &dist, seed );  
                    } else {  
                        if( i==j ) {  
                            a[i+ii+(jl+jj)*lld].real = RONE;  
                            a[i+ii+(jl+jj)*lld].imag = RZERO;  
                        } else {  
                            a[i+ii+(jl+jj)*lld].real = RZERO;  
                            a[i+ii+(jl+jj)*lld].imag = RZERO;  
                        }  
                    }  
                }  
            }  
        }  
    }  
}  
}
```

```

void pccmatgen_random(MKL_INT type, MKL_INT* desca, MKL_Complex8* a, MKL_INT* seed, MKL_INT
nprow, MKL_INT npc)
{
/*
* Purpose
* =====
*
* pccmatgen_random: parallel single complex precision matrix generator.
* Generate a sub-matrix of A.
*
* Arguments
* =====
*
* type      (global input)
*            Type of global matrix to be generated.
*            If type==0, local matrices are parts of the matrix
*            which possesses 1 as all elements along its leading (main) diagonal;
*            while all other elements in the matrix are 0.
*            Otherwise a random matrix is generated.
*
* desca     (global and local input) an array of (at least) length 9.
*            The array descriptor for the distributed matrix A.
*            See the notes below for the description of each element.
*
* a         (local output), pointer into the local
*            memory to a MKL_Complex8 array containing the
*            local pieces of the distributed matrix.
*
* seed      (local input and output) integer array, dimension (4)
*            On entry, the seed of the random number generator.
*            On exit, the seed is updated.
*
* nprow     (global input)
*            The number of process rows in the grid.
*
* npc       (global input)
*            The number of process columns in the grid.
*
* Notes:
* =====
*
* Each global data object is described by an associated description
* vector. This vector stores the information required to establish
* the mapping between an object element and its corresponding process
* and memory location.
*
* In the following comments, the character _ should be read as
* "of the distributed matrix". Let A be a generic term for any 2D
* block cyclicly distributed matrix. Its description vector is DESC_A:
*
* NOTATION      STORED IN      EXPLANATION
* -----
* DTYPE_A (global) DESC_A[ DTYPE_ ] The descriptor type.
* CTXT_A  (global) DESC_A[ CTXT_ ] The BLACS context handle, indicating
*                                the NPROW x NPCOL BLACS process grid
*                                A is distributed over. The context

```

```

*          itself is global, but the handle
*          (the integer value) may vary.
*  M_A      (global) DESCA[ M_    ] The number of rows in the distributed
*          matrix A, M_A >= 0.
*  N_A      (global) DESCA[ N_    ] The number of columns in the distributed
*          matrix A, N_A >= 0.
*  IMB_A    (global) DESCA[ IMB_  ] The number of rows of the upper left
*          block of the matrix A, IMB_A > 0.
*  INB_A    (global) DESCA[ INB_  ] The number of columns of the upper
*          left block of the matrix A,
*          INB_A > 0.
*  MB_A     (global) DESCA[ MB_   ] The blocking factor used to distribute
*          the last M_A-IMB_A rows of A,
*          MB_A > 0.
*  NB_A     (global) DESCA[ NB_   ] The blocking factor used to distribute
*          the last N_A-INB_A columns of
*          A, NB_A > 0.
*  RSRC_A   (global) DESCA[ RSRC_ ] The process row over which the first
*          row of the matrix A is distributed,
*          NPROW > RSRC_A >= 0.
*  CSRC_A   (global) DESCA[ CSRC_ ] The process column over which the
*          first column of A is distributed.
*          NPCOL > CSRC_A >= 0.
*  LLD_A    (local)  DESCA[ LLD_   ] The leading dimension of the local
*          array storing the local blocks of
*          the distributed matrix A,
*          IF( Lc( 1, N_A ) > 0 )
*             LLD_A >= MAX( 1, Lr( 1, M_A ) )
*          ELSE
*             LLD_A >= 1.
*/

MKL_INT myrow, mycol;
MKL_INT iarow = 0, iacol = 0;
MKL_INT ictxt = desca[CTXT_-1];
MKL_INT m      = desca[M_-1];
MKL_INT n      = desca[N_-1];
MKL_INT mb     = desca[MB_-1];
MKL_INT nb     = desca[NB_-1];
MKL_INT lld    = desca[LLD_-1];
MKL_INT i, j, ii, jj, il = 0, jl = 0;

blacs_gridinfo(&ictxt, &npro, &npcol, &myrow, &mycol);
for (j = 0; j < n; j += nb) {
    for (i = 0; i < m; i += mb) {
        i++; j++;
        infog2l(&i, &j, desca, &npro, &npcol, &myrow, &mycol, &il, &jl, &iarow, &iacol);
        i--; j--; il--; jl--;
        if (iarow==myrow && iacol==mycol) {
            for (jj = 0; jj < min(nb, n - j); jj++) {
                for (ii = 0; ii < min(mb, m - i); ii++) {
                    if( type ) {
                        a[il+ii+(jl+jj)*lld].real = (float) dlarnd( &dist, seed );
                        a[il+ii+(jl+jj)*lld].imag = (float) dlarnd( &dist, seed );
                    } else {
                        if( i==j ) {
                            a[il+ii+(jl+jj)*lld].real = RONE;
                            a[il+ii+(jl+jj)*lld].imag = RZERO;
                        } else {

```



```

! blacs_gridinit
! blacs_gridinfo
! blacs_gridexit
! blacs_exit
! descinit
! igamx2d
! igebr2d
! igebs2d
! sgebr2d
! sgebs2d
! numroc
! indxg2p
!
! The program must be driven by a short data file.
!
! 0          type, problem type, if type=0 the system  $A \cdot X = I$  is solved, otherwise  $A \cdot X = Y$ 
! 2000       n, dimension of matrix,  $0 < n$ 
! 1000       nrhs, number of right hand sides
! 16         nb, size of blocks, must be  $> 0$ 
! 2          p, number of rows in the process grid, must be  $> 0$ 
! 2          q, number of columns in the process grid, must be  $> 0$ ,  $p \cdot q$  = number of
processes
! 20.0       threshold for residual check
!
! *****/

#include <stdio.h>
#include <stdlib.h>
#include "matrix_generator.h"

MKL_INT test_solve(MKL_INT ictxt, MKL_INT type, MKL_INT n, MKL_INT nrhs, MKL_INT nb, MKL_INT
nprow, MKL_INT npc, MKL_INT* seed, float thresh)
{
    MKL_INT info = 0;
    MKL_INT err = 0;

    float *A = NULL;
    float *A_INI = NULL;
    float *X = NULL;
    float *Y = NULL;
    MKL_INT *IPIV = NULL;
/* Matrix descriptors */
    MKL_INT desca[9];
    MKL_INT descx[9];
    MKL_INT descy[9];
/* Local variables */

    MKL_INT myrow;
    MKL_INT mycol;
    MKL_INT iam, nprocs;
    float eps;
    float normA, normRhs, normSol, normRes, residual;

    float fzero = RZERO;
    float fone = RONE;
    float fnegone = RNEGONE;
/* Compute machine epsilon */
    eps = pslamch( &ictxt, "e" );

```

```

/* Get information about how many processes are used for program execution
and number of current process */
blacs_pinfo( &iam, &nprocs );
blacs_gridinfo(&ictxt, &nrow, &npcol, &myrow, &mycol);
/* Compute precise length of local pieces of distributed matrices */
MKL_INT loc_ra = numroc(&n, &nb, &myrow, &izero, &nrow);
MKL_INT loc_ca = numroc(&n, &nb, &mycol, &izero, &npcol);
MKL_INT loc_lda = max(1, loc_ra);
MKL_INT loc_sizeA =loc_ra*loc_ca;

MKL_INT loc_rx = numroc(&n, &nb, &myrow, &izero, &nrow);
MKL_INT loc_cx = numroc(&nrhs, &nb, &mycol, &izero, &npcol);
MKL_INT loc_ldx = max(1, loc_rx);
MKL_INT loc_ry = numroc(&n, &nb, &myrow, &izero, &nrow);
MKL_INT loc_cy = numroc(&nrhs, &nb, &mycol, &izero, &npcol);
MKL_INT loc_ldy = max(1, loc_ry);
/* Initialize descriptors for distributed arrays */

descinit(desca, &n, &n, &nb, &nb, &izero, &izero, &ictxt, &loc_lda, &info);
descinit(descx, &n, &nrhs, &nb, &nb, &izero, &izero, &ictxt, &loc_ldx, &info);
descinit(descy, &n, &nrhs, &nb, &nb, &izero, &izero, &ictxt, &loc_ldy, &info);

/* Allocate matrix, right-hand-side, vector solution x and the IPIV array
containing the pivoting information. The array A_INI is used to store
original matrix */
if( loc_sizeA ) {
    A = (float*)mkl_malloc( loc_sizeA * sizeof(float), 64 );
    A_INI=(float*)mkl_malloc( loc_sizeA * sizeof(float), 64 );
}
if( loc_rx * loc_cx ) X = (float*)mkl_malloc( loc_rx * loc_cx * sizeof(float), 64 );
if( loc_ry * loc_cy ) Y = (float*)mkl_malloc( loc_ry * loc_cy * sizeof(float), 64 );
IPIV = (MKL_INT*)mkl_malloc( (loc_ra + nb)*sizeof(MKL_INT), 64 );
if( A == NULL || X == NULL || Y == NULL || A_INI == NULL || IPIV == NULL ) info = 1;
igamx2d(&ictxt, "ALL", " ", &ione, &ione, &info, &ione, NULL, NULL, &inegone, &inegone,
&inegone);
if( info ) {
    err = 1;
    if ( iam == 0 ) printf( "\n Can't allocate memory for arrays\n" );
    goto early_exit;
}
/* Generate matrix and solution. Copy the initial matrix. */
seed[0]+= myrow + mycol;
psmatgen_random(1, desca, A, seed, nrow, npcol);
scopy(&loc_sizeA, A, &ione, A_INI, &ione);
psmatgen_random(type, descx, X, seed, nrow, npcol);
psmatgen_random(type, descy, Y, seed, nrow, npcol);
/* Compute the right hand side */
if( type ) {
    psgemm("N", "N", &n, &nrhs, &n, &fone, A, &ione, &ione, desca,
        X, &ione, &ione, descx,
        &fzero, Y, &ione, &ione, descy);
/* Copy the right hand side Y to X */
    psgemr2d(&n, &nrhs, Y, &ione, &ione, descy, X, &ione, &ione, descx, &ictxt);
}

/* Compute norm of the initial matrix and right hand side */
normA = pslange("M", &n, &n, A, &ione, &ione, desca, NULL);
normRhs = pslange("M", &n, &nrhs, Y, &ione, &ione, descy, NULL);

```

```

/* Compute the LU factorization of the initial matrix */
psgetrf(&n, &n, A, &ione, &ione, desca, IPIV, &info);
igamx2d(&ictxt, "ALL", " ", &ione, &ione, &info, &ione, NULL, NULL, &inegone, &inegone,
&inegone);
/* If psgetrf fails, deallocate all arrays and report an error */
if( info ) {
    err = 1;
    if ( iam == 0 ) printf( "\n psgetrf fails to compute LU decomposition \n" );
    goto early_exit;
}

psgetrs("N", &n, &nrhs, A, &ione, &ione, desca, IPIV, X, &ione, &ione, descx, &info);
igamx2d(&ictxt, "ALL", " ", &ione, &ione, &info, &ione, NULL, NULL, &inegone, &inegone,
&inegone);
/* If psgetrs fails, deallocate all arrays and report an error */
if( info ) {
    err = 1;
    if ( iam == 0 ) printf( "\n psgetrs fails to solve \n" );
    goto early_exit;
}

/* Normsol is the norm of the solution matrix: normsol = || X || */
normSol = pslange("M", &n, &nrhs, X, &ione, &ione, descx, NULL);

/* Compute the residual matrix || A*X - Y || */
psgemm("N", "N", &n, &nrhs, &n, &fone, A_INI, &ione, &ione, desca,
X, &ione, &ione, descx,
&fnegone, Y, &ione, &ione, descy);
/* Compute the residual as:
residual = ||A*X-Y||/(n*eps*(||A||*||X||+||Y||)) */

normRes = pslange("M", &n, &nrhs, Y, &ione, &ione, descy, NULL);
residual = normRes/( (float)n ) * eps * ( normA * normSol + normRhs );

/* Check if residual passed or failed the threshold */
if( iam == 0 ) {
    if ( ( thresh >= RZERO ) && !( residual <= thresh ) ){
        printf( "FAILED. Residual = %05.11f\n", residual );
        err = 1;
    } else {
        printf( "PASSED. Residual = %05.11f\n", residual );
    }
    printf( "=== END OF EXAMPLE =====\n" );
}

early_exit :
/* Destroy arrays */
if( A ) mkl_free(A);
if( A_INI ) mkl_free(A_INI);
if( X ) mkl_free(X);
if( Y ) mkl_free(Y);
if( IPIV ) mkl_free(IPIV);
return err;
}

int main(int argc, char *argv[])
{
/* ==== Declarations ===== */

```

```

/* File variables */
FILE      *fin;

/* Local scalars */
MKL_INT iam, nprocs, ictxt=-1, nprow, npcol;
MKL_INT n, nb, nrhs;
MKL_INT err;
MKL_INT type=1;
int      n_int, nb_int, nprow_int, npcol_int, nrhs_int, type_int;
float thresh;
MKL_INT seed[4] = { 105, 1410, 1860, 4085 };

/* Local arrays */
MKL_INT iw[ 6 ];

/* ==== Executable statements ===== */

/* Get information about how many processes are used for program execution
and number of current process */
blacs_pinfo( &iam, &nprocs );

/* Init temporary 1D process grid */
blacs_get( &inegone, &izero, &ictxt );
blacs_gridinit( &ictxt, "C", &nprocs, &ione );

/* Open input file */
if ( iam == 0 ) {
    fin = fopen( "psgetrf_example.in", "r" );
    if ( fin == NULL ) {
        printf( "Error while open input file." );
        return 1;
    }
}

/* Read data and send it to all processes */
if ( iam == 0 ) {

/*      Read parameters */
    fscanf( fin, "%d type, problem type, if type=0 the system A*X = I is solved, otherwise
A*X = Y", &type_int );
    fscanf( fin, "%d n, dimension of matrix, 0 < n", &n_int );
    fscanf( fin, "%d nrhs,number of right hand sides, 0 < nrhs", &nrhs_int );
    fscanf( fin, "%d nb, size of blocks, must be > 0", &nb_int );
    fscanf( fin, "%d p, number of rows in the process grid, must be > 0", &nrow_int );
    fscanf( fin, "%d q, number of columns in the process grid, must be > 0, p*q = number of
processes", &ncol_int );
    fscanf( fin, "%f threshold for residual check", &thresh );
    fclose( fin );
    type = (MKL_INT) type_int;
    n = (MKL_INT) n_int;
    nrhs = (MKL_INT) nrhs_int;
    nb = (MKL_INT) nb_int;
    nprow = (MKL_INT) nprow_int;
    npcol = (MKL_INT) npcol_int;
/*      Check if all parameters are correct */
    if( ( n<=0 ) || ( nrhs<=0 ) || ( nb<=0 ) || ( nprow<=0 ) ||
        ( npcol<=0 ) || ( nprow*npcol != nprocs ) ) {

```

```

        printf( "One or several input parameters has incorrect value. Limitations:\n" );
        printf( "n > 0, nrhs > 0, nb > 0, p > 0, q > 0 - integer\n" );
        printf( "p*q = number of processes\n" );
        printf( "threshold - float \n");
        return 1;
    }

/*    Pack data into array and send it to other processes */
    iw[ 0 ] = type;
    iw[ 1 ] = n;
    iw[ 2 ] = nrhs;
    iw[ 3 ] = nb;
    iw[ 4 ] = nprow;
    iw[ 5 ] = npcol;
    igebs2d( &ictxt, "All", " ", &isix, &ione, iw, &isix );
    sgebs2d( &ictxt, "All", " ", &ione, &ione, &thresh, &ione );
} else {

/*    Recieve and unpack data */
    igebr2d( &ictxt, "All", " ", &isix, &ione, iw, &isix, &izero, &izero );
    sgebr2d( &ictxt, "All", " ", &ione, &ione, &thresh, &ione, &izero, &izero );
    type = iw[ 0 ];
    n = iw[ 1 ];
    nrhs = iw[ 2 ];
    nb = iw[ 3 ];
    nprow = iw[ 4 ];
    npcol = iw[ 5 ];
}

/* Destroy temporary process grid */
blacs_gridexit( &ictxt );
/* Init workind 2D process grid */
blacs_get(&inegone, &izero, &ictxt);
blacs_gridinit(&ictxt, "R", &nprow, &npcol);
if ( iam == 0 ) {
/*    Print information of task */
    printf( "=== START OF EXAMPLE =====\n" );
    if( type ) {
        printf( "Solve matrix equation: A*X = Y\n" );
        printf( "with a general random n-by-n distributed matrix A \n" );
        printf( "Y is a random n-by-nrhs distributed matrix\n\n" );
    } else {
        printf( "Solve matrix equation: A*X = I\n" );
        printf( "with a general random n-by-n distributed matrix A \n" );
        printf( "I denotes a n-by-nrhs matrix which possesses 1 \n" );
        printf( "as all elements along its leading (main) diagonal \n" );
        printf( "while all other elements in the matrix are 0. \n" );
        printf( "If n==nrhs, I is the identity matrix and X is the matrix inverse of A.\n\n" );
    }
    printf( "n = %d, nrhs = %d, nb = %d; %dx%d - process grid\n\n", n_int, nrhs_int, nb_int,
nprow_int, npcol_int);
    printf( "Threshold for residual check = %05.11f\n", thresh );
}
err = test_solve(ictxt, type, n, nrhs, nb, nprow, npcol, seed, thresh);
/* Destroy process grid */
blacs_gridexit(&ictxt);

```

```

    blacs_exit(&izero);
    return err;
}

```

psgetrf_example.in

```

0      type, problem type, if type=0 the system A*X = I is solved, otherwise A*X = Y
2000   n, dimension of matrix, 0 < n
1000   nrhs, number of right hand sides
16     nb, size of blocks, must be > 0
2      p, number of rows in the process grid, must be > 0
2      q, number of columns in the process grid, must be > 0, p*q = number of processes
20.0   threshold for residual check

```

CMakeLists.txt

```

=====
# Copyright 2023 Intel Corporation.
#
# This software and the related documents are Intel copyrighted materials, and
# your use of them is governed by the express license under which they were
# provided to you (License). Unless the License provides otherwise, you may not
# use, modify, copy, publish, distribute, disclose or transmit this software or
# the related documents without Intel's prior written permission.
#
# This software and the related documents are provided as is, with no express
# or implied warranties, other than those that are expressly stated in the
# License.
=====

cmake_minimum_required(VERSION 3.13)
enable_testing()

# Enable SCALAPACK in oneMKL
# See MKLConfig.cmake top section for all cluster libraries
set(ENABLE_SCALAPACK ON)

# Define language and compiler
set(TEST_LANG C)
set(TEST_EXT c)

project(MKL_Example LANGUAGES ${TEST_LANG})
# Include oneMKL config file
find_package(MKL CONFIG REQUIRED)

# W/A for known problem in Intel(R) MPI 2021.1
if(MKL_MPI STREQUAL "intelmpi" AND DEFINED ENV{I_MPI_ROOT})
    if(UNIX AND NOT APPLE AND $ENV{I_MPI_ROOT} MATCHES "2021.1")
        set(MPI_C_ADDITIONAL_INCLUDE_DIRS $ENV{I_MPI_ROOT}/include)
    endif()
endif()

# Force mshpc to be the first in mpi find package
if(MKL_MPI STREQUAL "msmpi" OR MKL_MPI STREQUAL "mshpc")
    set(MPI_GUESS_LIBRARY_NAME MSMPI)
endif()

find_package(MPI REQUIRED)

```

```

# Override default compile/link lines for mpi script
if(USE_MPI_SCRIPT AND WIN32)
  set(CMAKE_C_COMPILE_OBJECT "<CMAKE_C_COMPILER> <DEFINES> <INCLUDES> <FLAGS> /Fo<OBJECT> -c
<SOURCE>")
  set(CMAKE_C_LINK_EXECUTABLE "<CMAKE_C_COMPILER> <OBJECTS> -o <TARGET> <LINK_LIBRARIES>")
endif()

# Try to identify MPI for correct BLACS if MKL_MPI is not defined
if(MPI_${TEST_LANG}_FOUND AND NOT MKL_MPI)
  if(UNIX AND NOT APPLE AND MPI_${TEST_LANG}_COMPILER MATCHES "openmpi")
    set(CURRENT_MPI "openmpi")
  elseif(WIN32 AND MPI_${TEST_LANG}_COMPILER MATCHES "msmpi" OR MPI_${TEST_LANG}_COMPILER
MATCHES "Microsoft MPI")
    set(CURRENT_MPI "msmpi")
  elseif(MPI_${TEST_LANG}_COMPILER MATCHES "mpich")
    set(CURRENT_MPI "mpich")
  elseif(NOT APPLE AND MPI_${TEST_LANG}_COMPILER MATCHES ".*intel.*mpi.*")
    set(CURRENT_MPI "intelmpi")
    if(WIN32 AND MPI_LOCALONLY)
      set(MPI_LOCAL_OPT -localonly)
    endif()
  endif()
  set(MKL_MPI "${CURRENT_MPI}")
endif()

# Define target
set(TEST_INCLUDE "${MPI_${TEST_LANG}_INCLUDE_DIRS}")

# Build example
set(func_name "psgetrf_example")
set(func "${func_name}")

file(GLOB_RECURSE ${func}_SRC ${PROJECT_SOURCE_DIR}/${func_name}.${TEST_EXT})
add_executable(${func} ${${func}_SRC})
target_include_directories(${func} PUBLIC ${TEST_INCLUDE})
if(WIN32)
  set(TEST_COPT "-D_CRT_SECURE_NO_WARNINGS")
  target_compile_options(${func} PUBLIC ${TEST_COPT})
endif()
target_link_libraries(${func} PUBLIC MKL::MKL MPI::MPI_C)

# Register example as ctest
configure_file(${PROJECT_SOURCE_DIR}/${func_name}.in ${CMAKE_CURRENT_BINARY_DIR} COPYONLY)
add_test(NAME ${func} COMMAND ${MPIEXEC_EXECUTABLE} ${MPIEXEC_NUMPROC_FLAG} 4 ${MPI_LOCAL_OPT} $
{PROJECT_BINARY_DIR}/${func})

# Add oneMKL environment variables
if(MKL_ENV)
  set_tests_properties(${func} PROPERTIES ENVIRONMENT "${MKL_ENV}")
endif()

```

Checking Your Installation

After installing the Intel® oneAPI Math Kernel Library (oneMKL), verify that the library is properly installed and configured:

1. Intel® oneAPI Math Kernel Library (oneMKL) installs in the *<parent directory>* directory.

2. Check that the subdirectory of `<parent directory>` referred to as `<mkl directory>` was created.
3. If you want to keep multiple versions of Intel® oneAPI Math Kernel Library (oneMKL) installed on your system, update your build scripts to point to the correct Intel® oneAPI Math Kernel Library (oneMKL) version.
4. Check that the following files appear in the `<mkl directory>/env` directory:
`*vars.sh`
 Use these files to assign Intel® oneAPI Math Kernel Library (oneMKL)-specific values to several environment variables, as explained in [Setting Environment Variables](#).
5. To understand how the Intel® oneAPI Math Kernel Library (oneMKL) directories are structured, see [Structure of the Intel® Math Kernel Library](#).
6. To make sure that Intel® oneAPI Math Kernel Library (oneMKL) runs on your system, launch an Intel® oneAPI Math Kernel Library (oneMKL) example, as explained in [Using Code Examples](#).

Setting Environment Variables

See Also

[Setting the Number of Threads Using an OpenMP* Environment Variable](#)

Scripts to Set Environment Variables

When the installation of Intel® oneAPI Math Kernel Library (oneMKL) for Linux* is complete, you can use the `setvars.sh` script in the installation directory to set environment variables MKLROOT, CPATH, LIBRARY_PATH, LD_LIBRARY_PATH, PATH, CMAKE_PREFIX_PATH, and PKG_CONFIG_PATH for oneMKL.

The scripts accept the oneMKL-specific parameters, explained in the following table:

Setting Specified	Required (Yes/No)	Possible Values	Comment
Use of Intel® oneAPI Math Kernel Library (oneMKL) Fortran modules precompiled with the Intel® Fortran compiler	No	mod	Supply this parameter only if you are using this compiler.
Use of the precompiled oneMKL Fortran modules for the LP64/ILP64 interface	No	lp64, default ilp64	This parameter has effects only when the parameter <code>mod</code> is also supplied.

For example:

- The command `setvars.sh` sets the environment for Intel® oneAPI Math Kernel Library (oneMKL) to use the Intel® 64 architecture.
- The command `setvars.sh mod ilp64` sets the environment for Intel® oneAPI Math Kernel Library (oneMKL) to use the Intel® 64 architecture and Fortran modules for the ILP64 interface.
- The command `setvars.sh mod` sets the environment for Intel® oneAPI Math Kernel Library (oneMKL) to use the Intel® 64 architecture and Fortran modules for the LP64 interface.

See Also

[High-level Directory Structure](#)

[Intel® oneAPI Math Kernel Library \(oneMKL\) Interface Libraries and Modules](#)

[Fortran 95 Interfaces to LAPACK and BLAS](#)

[Setting the Number of Threads Using an OpenMP* Environment Variable](#)

Modulefiles to Set Environment Variables

As an alternative, you can use modulefiles to set the environment on Linux*. When the installation of Intel® oneAPI Math Kernel Library (oneMKL) for Linux* is complete, you need to set some environment variables. First, setup modulefiles and directories in <oneapi_root>:

```
$/modulefiles-setup.sh
```

```
$module use modulefiles
```

Load modulefile:

Architecture	Command
For Intel® oneAPI Math Kernel Library (oneMKL) Intel® 64 architecture	<code>\$module load mkl/<version></code>

NOTE All existing variables will retain their original values and new values will be prepended to the following variables: `LD_LIBRARY_PATH`, `LIBRARY_PATH`, `CPATH`, `PKG_CONFIG_PATH`, `TBBROOT`. Since `MKLROOT` should contain only one path, it will be set to the new value and any old value of this variable will be lost.

Unloading the module resets the modified environment variables to their original values and new variables that were created will be undefined. `MKLROOT` will be undefined on module unload.

Architecture	Command
For Intel® oneAPI Math Kernel Library (oneMKL) Intel® 64 architecture	<code>\$module unload mkl/<version></code>

Automating the Process of Setting Environment Variables

To automate setting of environment variables for Intel® oneAPI Math Kernel Library (oneMKL), add `env/vars.sh` to your shell profile so that each time you log in, the script automatically runs and sets the paths to the appropriate Intel® oneAPI Math Kernel Library (oneMKL) directories. To do this with a local user account, edit the following files by adding the appropriate script to the path manipulation section right before exporting variables:

Shell	Files	Commands
bash	~/.bash_profile,	# setting up oneMKL environment for bash
	~/.bash_login	.
	or ~/.profile	<absolute_path_to_installed_oneMKL>/env/ vars.sh [mod [lp64 ilp64]]
sh	~/.profile	# setting up oneMKL environment for sh
		.
		<absolute_path_to_installed_oneMKL>/env/ vars.sh [mod [lp64 ilp64]]

If you have super user permissions, you can add the same commands to a general-system file in `/etc/profile`.

Caution

Before uninstalling Intel® oneAPI Math Kernel Library (oneMKL), remove the above commands from all profile files where the script execution was added. Otherwise you may experience problems logging in.

See Also

[Scripts to Set Environment Variables](#)

Using the CMake Config File

Intel® oneAPI Math Kernel Library (oneMKL) ships with a CMake config file for easy integration of oneMKL into user-applications that already use CMake. This is a Config-mode file that you can use in `find_package()` macros. For different package modes, refer to [find_package CMake documentation](#).

`MKLConfig.cmake` is in `<MKLROOT>/lib/cmake/mkl`. If CMake cannot find it by default, you can specify its path as `-DMKL_DIR=<MKLROOT>/lib/cmake/mkl` in the `cmake` configure command. The top of the file contains an up-to-date usage comment-block for reference, and you can unpack `<MKLROOT>/share/doc/mkl/examples/*.tgz` and then refer to `<MKLROOT>/share/doc/mkl/examples/<type>/CMakeLists.txt` for more details about how to consume `MKLConfig.cmake` in CMake projects.

Compiler Support

When building Intel® oneAPI Math Kernel Library (oneMKL) code examples for either C or Fortran, you can select a compiler: Intel® or GNU*.

Intel® oneAPI Math Kernel Library (oneMKL) provides a set of include files to simplify program development by specifying enumerated values and prototypes for the respective functions. Calling Intel® oneAPI Math Kernel Library (oneMKL) functions from your application without an appropriate include file may lead to incorrect behavior of the functions.

See Also

[Intel® oneAPI Math Kernel Library \(oneMKL\) Include Files](#)

Using Code Examples

The Intel® oneAPI Math Kernel Library (oneMKL) package includes code examples, located in the `share/doc/mkl/examples` subdirectory of the installation directory. Use the examples to determine:

- Whether Intel® oneAPI Math Kernel Library (oneMKL) is working on your system
- How to call the library
- How to link the library

The examples are grouped by programming languages and Intel® oneAPI Math Kernel Library (oneMKL) function domains. Refer to the `README.txt` file in the `share/doc/mkl/examples` subdirectory for instructions on building the examples. Source code for the examples is in the next-level `source` subdirectory.

See Also

[High-Level Directory Structure](#) A table listing the main components of the directory structure.

[oneAPI Samples Catalog](#) These samples were designed to help you develop, offload, and optimize multiarchitecture applications targeting CPUs, GPUs, and FPGAs.

These samples were designed to help you develop, offload, and optimize multiarchitecture applications targeting CPUs, GPUs, and FPGAs.

What You Need to Know Before You Begin Using the Intel® oneAPI Math Kernel Library

Mathematical problem	Identify all Intel® oneAPI Math Kernel Library (oneMKL) function domains that you require: • BLAS • Sparse BLAS • LAPACK • PBLAS
----------------------	---

- ScaLAPACK
- Sparse Solver routines
- Intel® Math Kernel Library Parallel Direct Sparse Solver for Clusters (Intel MKL Parallel Direct Sparse Solver for Clusters)
- Vector Mathematics functions (VM)
- Vector Statistics functions (VS)
- Fourier Transform functions (FFT)
- Cluster FFT
- Trigonometric Transform routines
- Poisson, Laplace, and Helmholtz Solver routines
- Optimization (Trust-Region) Solver routines
- Data Fitting Functions
- Extended Eigensolver Functions

Reason: The function domain you intend to use narrows the search in the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference* for specific routines you need. Additionally, if you are using the Intel® oneAPI Math Kernel Library (oneMKL) cluster software, your link line is function-domain specific (see [Working with the Intel® Math Kernel Library Cluster Edition Software](#)). Coding tips may also depend on the function domain (see [Other Tips and Techniques to Improve Performance](#)).

Programming language

Intel® oneAPI Math Kernel Library (oneMKL) provides support for both Fortran and C/C++ programming. Identify the language interfaces that your function domains support (see [Appendix A: Intel® oneAPI Math Kernel Library Language Interfaces Support](#)).

Reason: Intel® oneAPI Math Kernel Library (oneMKL) provides language-specific include files for each function domain to simplify program development (see [Language Interfaces Support_ by Function Domain](#)).

For a list of language-specific interface libraries and modules and an example how to generate them, see also [Using Language-Specific Interfaces with Intel® oneAPI Math Kernel Library](#).

Range of integer data

If your system is based on the Intel 64 architecture, identify whether your application performs calculations with large data arrays (of more than $2^{31}-1$ elements).

Reason: To operate on large data arrays, you need to select the ILP64 interface, where integers are 64-bit; otherwise, use the default, LP64, interface, where integers are 32-bit (see [Using the ILP64 Interface vs](#)).

Threading model

Identify whether and how your application is threaded:

- Threaded with the Intel compiler
- Threaded with a third-party compiler
- Not threaded

Reason: The compiler you use to thread your application determines which threading library you should link with your application. For applications threaded with a third-party compiler you may need to use Intel® oneAPI Math Kernel Library (oneMKL) in the sequential mode (for more information, see [Linking with Threading Libraries](#)).

Number of threads

If your application uses an OpenMP* threading run-time library, determine the number of threads you want Intel® oneAPI Math Kernel Library (oneMKL) to use.

Reason: By default, the OpenMP* run-time library sets the number of threads for Intel® oneAPI Math Kernel Library (oneMKL). If you need a different number, you have to set it yourself using one of the available mechanisms. For more information, see [Improving Performance with Threading](#).

Linking model

Decide which linking model is appropriate for linking your application with Intel® oneAPI Math Kernel Library (oneMKL) libraries:

- Static
- Dynamic

Reason: The link line syntax and libraries for static and dynamic linking are different. For the list of link libraries for static and dynamic models, linking examples, and other relevant topics, like how to save disk space by creating a custom dynamic library, see [Linking Your Application with the Intel® oneAPI Math Kernel Library](#).

MPI used

Decide what MPI you will use with the Intel® oneAPI Math Kernel Library (oneMKL) cluster software. You are strongly encouraged to use the latest available version of Intel® MPI.

Reason: To link your application with ScaLAPACK and/or Cluster FFT, the libraries corresponding to your particular MPI should be listed on the link line (see [Working with the Intel® Math Kernel Library Cluster Edition Software](#)).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Structure of the Intel® oneAPI Math Kernel Library

Architecture Support

Intel® oneAPI Math Kernel Library (oneMKL) for Linux* provides architecture-specific implementations for supported platforms. The following table lists the supported architectures and directories where each architecture-specific implementation is located.

Architecture	Location
Intel® 64 or compatible	<code><mkl directory>/lib</code>

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[High-level Directory Structure](#)

[Notational Conventions](#)

[Detailed Structure of the Intel® 64 Architecture Directory lib](#)

High-Level Directory Structure

Directory	Contents
<code><mkl directory></code>	Installation directory of the Intel® oneAPI Math Kernel Library (oneMKL)
Subdirectories of <code><mkl directory></code>	

Directory	Contents
env	Scripts and module files to set environmental variables in the user shell
share/mkl/ benchmarks/linpack	Shared-memory (SMP) version of the LINPACK benchmark
share/mkl/ benchmarks/ mp_linpack	Message-passing interface (MPI) version of the LINPACK benchmark
share/mkl/ benchmarks/hpcg	Intel® High Performance Conjugate Gradient Benchmark (Intel® HPCG) containing <code>hpcg_cpu/</code> and <code>hpcg_gpu/</code> folders with the optimized for CPU and optimized for GPU binaries and source code respectively
share/doc/mkl/ examples	Source and data files for Intel® oneAPI Math Kernel Library (oneMKL) examples. Provided in archives corresponding to Intel® oneAPI Math Kernel Library (oneMKL) components selected during installation
include	Include files for the library routines and examples
include/mkl/intel64/ ilp64	Fortran 95 .mod files for the Intel® 64 architecture, Intel® Fortran Compiler, and ILP64 interface
include/fftw	Header files for the FFTW2 and FFTW3 interfaces
include/oneapi	Header files for DPC++ interfaces.
share/mkl/ interfaces/blas95	Fortran 95 interfaces to BLAS and a makefile to build the library
share/mkl/ interfaces/ fftw2x_cdft	MPI FFTW 2.x interfaces to the Intel® oneAPI Math Kernel Library (oneMKL) Cluster FFT
share/mkl/ interfaces/ fftw3x_cdft	MPI FFTW 3.x interfaces to the Intel® oneAPI Math Kernel Library (oneMKL) Cluster FFT
share/mkl/ interfaces/fftw2xc	FFTW 2.x interfaces to the Intel® oneAPI Math Kernel Library (oneMKL) FFT (C interface)
share/mkl/ interfaces/fftw2xf	FFTW 2.x interfaces to the Intel® oneAPI Math Kernel Library (oneMKL) FFT (Fortran interface)
share/mkl/ interfaces/fftw3xc	FFTW 3.x interfaces to the Intel® oneAPI Math Kernel Library (oneMKL) FFT (C interface)
share/mkl/ interfaces/fftw3xf	FFTW 3.x interfaces to the Intel® oneAPI Math Kernel Library (oneMKL) FFT (Fortran interface)
share/mkl/ interfaces/lapack95	Fortran 95 interfaces to LAPACK and a makefile to build the library
share/mkl/ interfaces/mklmpi	Tool to create a custom MKLMPI wrapper library (BLACS) for use in MKL MPI-based applications like Cluster Sparse Solver and Scalapack.
lib	Static libraries and shared objects for the Intel® 64 architecture
share/mkl/tools	Tools and plug-ins

Directory	Contents
share/mkl/tools/ builder	Tools for creating custom dynamically linkable libraries
bin	mkl_link_tool (see Using the Command-Line Link Tool)

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Notational Conventions](#)

[Using Code Examples](#)

Layered Model Concept

Intel® oneAPI Math Kernel Library (oneMKL) is structured to support multiple compilers and interfaces, both serial and multi-threaded modes, different implementations of threading run-time libraries, and a wide range of processors. Conceptually Intel® oneAPI Math Kernel Library (oneMKL) can be divided into distinct parts to support different interfaces, threading models, and core computations:

1. Interface Layer
2. Threading Layer
3. Computational Layer

You can combine Intel® oneAPI Math Kernel Library (oneMKL) libraries to meet your needs by linking with one library in each part layer-by-layer.

To support threading with different compilers, you also need to use an appropriate threading run-time library (RTL). These libraries are provided by compilers and are not included in Intel® oneAPI Math Kernel Library (oneMKL).

The following table provides more details of each layer.

Layer	Description
Interface Layer	This layer matches compiled code of your application with the threading and/or computational parts of the library. This layer provides: • LP64 and ILP64 interfaces. • Compatibility with compilers that return function values differently.
Threading Layer	This layer: • Provides a way to link threaded Intel® oneAPI Math Kernel Library (oneMKL) with supported compilers. • Enables you to link with a threaded or sequential mode of the library. This layer is compiled for different environments (threaded or sequential) and compilers (from Intel, GNU*,).
Computational Layer	This layer accommodates multiple architectures through identification of architecture features and chooses the appropriate binary code at run time.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Product and Performance Information

Notice revision #20201201

See Also[Using the ILP64 Interface vs. LP64 Interface](#)[Linking Your Application with the Intel® oneAPI Math Kernel Library](#)[Linking with Threading Libraries](#)

Linking Your Application with the Intel® oneAPI Math Kernel Library

Linking Quick Start

Intel® oneAPI Math Kernel Library (oneMKL) provides several options for quick linking of your application, which depend on the way you link:

Using the Intel® Parallel Studio XE Composer Edition compiler	see Using the -mkl Compiler Option .
Explicit dynamic linking	see Using the Single Dynamic Library for how to simplify your link line.
Explicitly listing libraries on your link line	see Selecting Libraries to Link with for a summary of the libraries.
Using pkg-config tool to get compilation and link lines	see Using pkg-config metadata files for a summary on how to use Intel® oneAPI Math Kernel Library (oneMKL) pkg-config metadata files.
Using an interactive interface	see Using the Link-line Advisor to determine libraries and options to specify on your link or compilation line.
Using an internally provided tool	see Using the Command-line Link Tool to determine libraries, options, and environment variables or even compile and build your application.

Using the -qmkl Compiler Option

The Intel® oneAPI DPC++/C++ Compiler, the Intel® Fortran Compiler Classic, and the Intel® Fortran Compiler support the following `-qmkl` compiler option variants so that you can easily link against the Intel® oneAPI Math Kernel Library (oneMKL):

<code>-qmkl=parallel</code> (<code>-qmkl</code>)	Links against the oneMKL Intel threading layer and the necessary libraries. The <code>-qmkl=parallel</code> option can be simplified to <code>-qmkl</code> .
<code>-qmkl=parallel -qttbb</code> (<code>-qmkl -qttbb</code>)	Links against the oneMKL Intel TBB threading layer and the necessary libraries. The <code>-qttbb</code> option is supported only by the Intel® oneAPI DPC++/C++ Compiler.
<code>-qmkl=sequential</code>	Links against the oneMKL sequential threading layer and the necessary libraries.

For each `-qmk1` compiler option variant on Intel 64-architecture systems, the compiler links your application against the LP64 interface except when you are using the C++ compiler `icpx` with the `-fsycl` option, which links against the ILP64 interface. To link against the ILP64 interface in other cases, use the `-qmk1-ilp64` compiler option variants (see [Using the -qmk1-ilp64 Compiler Option](#)).

On Linux* systems, the `-qmk1` compiler option must be specified both in the compilation and link lines. In addition, dynamic linking against oneMKL is the default for the `-qmk1` compiler option. If you want to link against oneMKL statically, you must additionally specify `-static-intel`.

For the complete compilation and link lines, use the [oneMKL Link Line Advisor](#). For more information, see [Linking in Detail](#).

See Also

[Listing Libraries on a Link Line](#)

[Using the ILP64 Interface vs. LP64 Interface](#)

[Using the Link-line Advisor](#)

[Intel® Software Documentation Library](#) for Intel® compiler documentation
for Intel® compiler documentation

Using the -qmk1-ilp64 Compiler Option

The Intel® oneAPI DPC++/C++ Compiler, the Intel® Fortran Compiler Classic, and the Intel® Fortran Compiler support the following `-qmk1-ilp64` compiler option variants so that you can easily link against the Intel® oneAPI Math Kernel Library (oneMKL) ILP64 interface, which supports large data arrays (with more than $2^{31} - 1$ elements):

<code>-qmk1-ilp64=parallel</code> (<code>-qmk1-ilp64</code>)	Links against the oneMKL Intel threading layer and the necessary libraries. The <code>-qmk1-ilp64=parallel</code> option can be simplified to <code>-qmk1-ilp64</code> .
<code>-qmk1-ilp64=parallel -qtbb</code> (<code>-qmk1-ilp64 -qtbb</code>)	Links against the oneMKL ILP64 interface, the Intel TBB threading layer, and the necessary libraries. The <code>-qtbb</code> option is supported only by the Intel® oneAPI DPC++/C++ Compiler.
<code>-qmk1-ilp64=sequential</code>	Links against the oneMKL ILP64 interface, the oneMKL sequential threading layer, and the necessary libraries.

For each `-qmk1-ilp64` compiler option variant on Intel 64-architecture systems, the compiler links your application against the ILP64 interface. To link against the LP64 interface in other cases, use the `-qmk1` compiler option variants (see [Using the -qmk1 Compiler Option](#)).

If both `-qmk1-ilp64` and `-qmk1` are specified on the command line, the rightmost specified option takes precedence.

On Linux* systems, the `-qmk1-ilp64` compiler option must be specified both in the compilation and link lines. In addition, dynamic linking against oneMKL is the default for the `-qmk1-ilp64` compiler option. If you want to link against oneMKL statically, you must additionally specify `-static-intel`.

For the complete compilation and link lines, use the [oneMKL Link Line Advisor](#). For more information, see [Linking in Detail](#).

See Also

[Using the ILP64 Interface vs. LP64 Interface](#)

[Using the Link-line Advisor](#)

[Intel® Software Documentation Library](#) for Intel® compiler documentation
for Intel® compiler documentation

Using the Single Dynamic Library

You can simplify your link line through the use of the Intel® oneAPI Math Kernel Library (oneMKL) Single Dynamic Library (SDL).

To use SDL, place `libmkl_rt.so` on your link line. For example:

```
icx application.c -lmkl_rt
```

SDL enables you to select the interface and threading library for Intel® oneAPI Math Kernel Library (oneMKL) at run time. By default, linking with SDL provides:

- Intel LP64 interface on systems based on the Intel® 64 architecture
- Intel threading

To use other interfaces or change threading preferences, including use of the sequential version of Intel® oneAPI Math Kernel Library (oneMKL), you need to specify your choices using functions or environment variables as explained in section [Dynamically Selecting the Interface and Threading Layer](#).

NOTE Intel® oneAPI Math Kernel Library (oneMKL) SDL (`mkl_rt`) does not support DPC++ APIs. If your application requires support of Intel® oneAPI Math Kernel Library (oneMKL) DPC++ APIs, refer to [Intel® oneAPI Math Kernel Library Link-line Advisor](#) to configure your link command.

Selecting Libraries to Link with

To link with Intel® oneAPI Math Kernel Library (oneMKL):

- Choose one library from the Interface layer and one library from the Threading layer
- Add the only library from the Computational layer and run-time libraries (RTL)

The following table lists Intel® oneAPI Math Kernel Library (oneMKL) libraries to link with your application.

	Interface layer	Threading layer	Computational layer	RTL
Intel® 64 architecture, static linking	<code>libmkl_intel_lp64.a</code>	<code>libmkl_intel_thread.a</code>	<code>libmkl_core.a</code>	<code>libiomp5.so</code>
Intel® 64 architecture, dynamic linking	<code>libmkl_intel_lp64.so</code>	<code>libmkl_intel_thread.so</code>	<code>libmkl_core.so</code>	<code>libiomp5.so</code>
Intel® Many Integrated Core Architecture (Intel® MIC Architecture), static linking	<code>libmkl_intel_lp64.a</code>	<code>libmkl_intel_thread.a</code>	<code>libmkl_core.a</code>	<code>libiomp5.so</code>
Intel MIC Architecture, dynamic linking	<code>libmkl_intel_lp64.so</code>	<code>libmkl_intel_thread.so</code>	<code>libmkl_core.so</code>	<code>libiomp5.so</code>

The Single Dynamic Library (SDL) automatically links interface, threading, and computational libraries and thus simplifies linking. The following table lists Intel® oneAPI Math Kernel Library (oneMKL) libraries for dynamic linking using SDL. See [Dynamically Selecting the Interface and Threading Layer](#) for how to set the interface and threading layers at run time through function calls or environment settings.

	SDL	RTL
Intel® 64 architecture	<code>libmkl_rt.so</code>	<code>libiomp5.so^{††}</code>

^{††}Use the Link-line Advisor to check whether you need to explicitly link the `libiomp5.so` RTL.

For exceptions and alternatives to the libraries listed above, see [Linking in Detail](#).

See Also

[Layered Model Concept](#)

[Using the Link-line Advisor](#)

[Using the `-mkl` Compiler Option](#)

[Working with the Cluster Software](#)

Using the Link-line Advisor

Use the Intel® oneAPI Math Kernel Library (oneMKL) Link-line Advisor to determine the libraries and options to specify on your link or compilation line.

The latest version of the tool is available at [Link Line Advisor for Intel® oneAPI Math Kernel Library](#). The tool is also available in the documentation directory of the product.

The Advisor requests information about your system and on how you intend to use Intel® oneAPI Math Kernel Library (oneMKL) (link dynamically or statically, use threaded or sequential mode, and so on). The tool automatically generates the appropriate link line for your application.

See Also

[High-level Directory Structure](#)

Using the Command-Line Link Tool

Use the command-line Link Tool provided by Intel® oneAPI Math Kernel Library (oneMKL) to simplify building your application with Intel® oneAPI Math Kernel Library (oneMKL).

The tool not only provides the options, libraries, and environment variables to use, but also performs compilation and building of your application.

The tool `mkl_link_tool` is installed in the `<mkl_directory>/bin/<arch>` directory, and supports the modes described in the following table.

oneMKL Command-Line Link Tool Modes

Mode	Description	Usage	Example
Inquiry	The tool returns the compiler options, libraries, or environment variables necessary to build and execute the application.	Get Intel® oneAPI Math Kernel Library (oneMKL) libraries	<code>mkl_link_tool -libs [oneMKL Link Tool options]</code>
		Get compilation options	<code>mkl_link_tool -opts [oneMKL Link Tool options]</code>
		Get environment variables for the application executable	<code>mkl_link_tool -env [oneMKL Link Tool options]</code>
Compilation	The Intel® oneAPI Math Kernel Library (oneMKL) Link Tool builds the application.	—	<code>mkl_link_tool [options]</code> <code><compiler></code> <code>[options2] file1</code> <code>[file2 ...]</code> where: <code>options</code> represents any number of Link Tool options <code>compiler</code> represents the compiler name: <code>icx</code>, <code>ifx</code>, <code>cl</code>, <code>gcc</code> (<code>g+</code>), <code>gfortran</code>, <code>mpiic</code>,

Mode	Description	Usage	Example
			mpiifort, mpic (mpic ++), mpif77 (mpif90, mpif95), icpx -fsycl options2 represents any number of compiler options
Interactive	Allows you to go through all possible Intel® oneAPI Math Kernel Library (oneMKL) Link Tool supported options. The output provides libraries, options, or environment variables as in the inquiry mode, or a built application as in the compilation mode (depending on what you specify).	—	mkl_link_tool -interactive

Use the `-help` option for full help with the Intel® oneAPI Math Kernel Library (oneMKL) Link Tool and to show the defaults for the current system.

Linking Examples

See Also

[Using the Link-line Advisor](#)

[Examples for Linking with ScaLAPACK and Cluster FFT](#)

Linking on Intel(R) 64 Architecture Systems

The following examples illustrate linking that uses Intel(R) compilers.

Most examples use the `.f` Fortran source file. C/C++ users should instead specify a `.cpp` (C++) or `.c` (C) file and replace `ifx` with `icx`.

In these examples,

```
MKLPATH=$MKLROOT/lib/intel64_lin,
MKLINCLUDE=$MKLROOT/include.
```

NOTE

If you successfully completed the [Scripts to Set Environment Variables Setting Environment Variables](#) step of the Getting Started process, you can omit `-I$MKLINCLUDE` in all the examples and omit `-L$MKLPATH` in the examples for dynamic linking.

- Static linking of `myprog.f` and OpenMP* threaded Intel® oneAPI Math Kernel Library (oneMKL) supporting the LP64 interface:

```
ifx myprog.f -L$MKLPATH -I$MKLINCLUDE
-Wl,--start-group $MKLPATH/libmkl_intel_lp64.a $MKLPATH/libmkl_intel_thread.a
$MKLPATH/libmkl_core.a -Wl,--end-group -liomp5 -lpthread -lm
```

- Dynamic linking of `myprog.f` and OpenMP* threaded Intel® oneAPI Math Kernel Library (oneMKL) supporting the LP64 interface:

```
ifx myprog.f -L$MKLPATH -I$MKLINCLUDE
-lmkl_intel_lp64 -lmkl_intel_thread -lmkl_core
-liomp5 -lpthread -lm
```

- Static linking of `myprog.f` and sequential version of Intel® oneAPI Math Kernel Library (oneMKL) supporting the LP64 interface:

```
ifx myprog.f -L$MKLSPATH -I$MKLINCLUDE
-Wl,--start-group $MKLSPATH/libmkl_intel_lp64.a $MKLSPATH/libmkl_sequential.a
$MKLSPATH/libmkl_core.a -Wl,--end-group -lpthread -lm
```

- Dynamic linking of `myprog.f` and sequential version of Intel® oneAPI Math Kernel Library (oneMKL) supporting the LP64 interface:

```
ifx myprog.f -L$MKLSPATH -I$MKLINCLUDE
-lmkl_intel_lp64 -lmkl_sequential -lmkl_core -lpthread -lm
```

- Static linking of `myprog.f` and OpenMP* threaded Intel® oneAPI Math Kernel Library (oneMKL) supporting the ILP64 interface:

```
ifx myprog.f -L$MKLSPATH -I$MKLINCLUDE
-Wl,--start-group $MKLSPATH/libmkl_intel_ilp64.a $MKLSPATH/libmkl_intel_thread.a
$MKLSPATH/libmkl_core.a -Wl,--end-group -liomp5 -lpthread -lm
```

- Dynamic linking of `myprog.f` and OpenMP* threaded Intel® oneAPI Math Kernel Library (oneMKL) supporting the ILP64 interface:

```
ifx myprog.f -L$MKLSPATH -I$MKLINCLUDE
-lmkl_intel_ilp64 -lmkl_intel_thread -lmkl_core -liomp5 -lpthread -lm
```

- Dynamic linking of user code `myprog.f` and OpenMP* threaded or sequential Intel® oneAPI Math Kernel Library (oneMKL) (Call appropriate functions or set environment variables to choose threaded or sequential mode and to set the interface):

```
ifx myprog.f -lmkl_rt
```

- Static linking of `myprog.f`, Fortran BLAS and 95 LAPACK interfaces, and OpenMP* threaded Intel® oneAPI Math Kernel Library (oneMKL) supporting the LP64 interface:

```
ifx myprog.f -L$MKLSPATH -I$MKLINCLUDE -I$MKLINCLUDE/intel64/lp64
-lmkl_lapack95_lp64 -Wl,--start-group $MKLSPATH/libmkl_intel_lp64.a $MKLSPATH/
libmkl_intel_thread.a
$MKLSPATH/libmkl_core.a -Wl,--end-group -liomp5 -lpthread -lm
```

- Static linking of `myprog.c` and Intel® oneAPI Math Kernel Library (oneMKL) threaded with Intel® Threading Building Blocks (Intel® TBB), provided that the `TBBROOT` environment variable is defined:

```
icx myprog.c -I$MKLINCLUDE -Wl,--start-group $MKLSPATH/libmkl_intel_lp64.a $MKLSPATH/
libmkl_tbb_thread.a $MKLSPATH/libmkl_core.a -Wl,--end-group -L$TBBROOT/lib/intel64/
gcc.4.8 -ltbb -lstdc++ -lpthread -lm
```

- Dynamic linking of `myprog.c` and Intel® oneAPI Math Kernel Library (oneMKL) threaded with Intel® TBB, provided that the `LD_LIBRARY_PATH` environment variable contains the path to Intel® TBB library:

```
icx myprog.c -I$MKLINCLUDE -L$MKLSPATH -lmkl_intel_lp64 -lmkl_tbb_thread -lmkl_core -
lstdc++ -lpthread -lm
```

See Also

[Fortran 95 Interfaces to LAPACK and BLAS](#)

[Examples for linking a C application using cluster components](#)

[Examples for linking a Fortran application using cluster components](#)

[Using the Single Dynamic Library](#)

[Linking with System Libraries](#) for specifics of linking with a GNU compiler

Linking in Detail

This section recommends which libraries to link with depending on your Intel® oneAPI Math Kernel Library (oneMKL) usage scenario and provides details of the linking.

Listing Libraries on a Link Line

To link with Intel® oneAPI Math Kernel Library (oneMKL), specify paths and libraries on the link line as shown below.

NOTE

The syntax below is for dynamic linking. For static linking, replace each library name preceded with "-l" with the path to the library file. For example, replace `-lmkl_core` with `$MKL_PATH/libmkl_core.a`, where `$MKL_PATH` is the appropriate user-defined environment variable.

```
<files to link>
-L<MKL path>-I<MKL include>
[-I<MKL include>intel64|{ilp64|lp64}}]
[-lmkl_blas{95|95_ilp64|95_lp64}]
[-lmkl_lapack{95|95_ilp64|95_lp64}]
[<cluster components>]
-lmkl_{intel|intel_ilp64|intel_lp64|intel_sp2dp|gf|gf_ilp64|gf_lp64}
-lmkl_{intel_thread|tbb_thread|sequential}
-lmkl_core
[-liomp5] [-lpthread] [-lm] [-ldl] [-ltbb -lstdc++]
```

In the case of static linking, enclose the cluster components, interface, threading, and computational libraries in grouping symbols (for example, `-Wl,--start-group $MKL_PATH/libmkl_cdft_core.a $MKL_PATH/libmkl_blacs_intelmpi_ilp64.a $MKL_PATH/libmkl_intel_ilp64.a $MKL_PATH/libmkl_intel_thread.a $MKL_PATH/libmkl_core.a -Wl,--end-group`).

The order of listing libraries on the link line is essential, except for the libraries enclosed in the grouping symbols above.

See Also

[Using the Link Line Advisor](#)

[Linking Examples](#)

[Working with the Cluster Software](#)

Dynamically Selecting the Interface and Threading Layer

The Single Dynamic Library (SDL) enables you to dynamically select the interface and threading layer for Intel® oneAPI Math Kernel Library (oneMKL).

Setting the Interface Layer

To set the interface layer at run time, use the `mkl_set_interface_layer` function or the `MKL_INTERFACE_LAYER` environment variable.

Available interface layers depend on the architecture of your system.

The following table lists available interface layers for Intel® 64 architecture along with the values to be used to set each layer.

Specifying the Interface Layer for Intel® 64 Architecture

Interface Layer	Value of MKL_INTERFACE_LAYER	Value of the Parameter of mkl_set_interface_layer
Intel LP64, default	LP64	MKL_INTERFACE_LP64
Intel ILP64	ILP64	MKL_INTERFACE_ILP64
GNU* LP64	GNU, LP64	MKL_INTERFACE_LP64+MKL_INTERFACE_GNU
GNU ILP64	GNU, ILP64	MKL_INTERFACE_ILP64+MKL_INTERFACE_GNU

If the `mkl_set_interface_layer` function is called, the environment variable `MKL_INTERFACE_LAYER` is ignored.

Setting the Threading Layer

To set the threading layer at run time, use the `mkl_set_threading_layer` function or the `MKL_THREADING_LAYER` environment variable. The following table lists available threading layers along with the values to be used to set each layer.

Specifying the Threading Layer

Threading Layer	Value of MKL_THREADING_LAYER	Value of the Parameter of mkl_set_threading_layer
Intel threading, default	INTEL	MKL_THREADING_INTEL
Sequential mode of Intel® oneAPI Math Kernel Library (oneMKL)	SEQUENTIAL	MKL_THREADING_SEQUENTIAL
GNU threading	GNU	MKL_THREADING_GNU
Intel TBB threading	TBB	MKL_THREADING_TBB

If the `mkl_set_threading_layer` function is called, the environment variable `MKL_THREADING_LAYER` is ignored.

See Also

[Using the Single Dynamic Library Layered Model Concept](#)
[Directory Structure in Detail](#)

Linking with Interface Libraries

Using the ILP64 Interface vs. LP64 Interface

The Intel® oneAPI Math Kernel Library (oneMKL) ILP64 libraries use the 64-bit integer type (necessary for indexing large arrays, with more than $2^{31}-1$ elements), whereas the LP64 libraries index arrays with the 32-bit integer type.

The LP64 and ILP64 interfaces are implemented in the Interface layer. Link with the following interface libraries for the LP64 or ILP64 interface, respectively:

- `libmkl_intel_lp64.a` or `libmkl_intel_ilp64.a` for static linking

- `libmkl_intel_lp64.so` or `libmkl_intel_ilp64.so` for dynamic linking

The ILP64 interface provides for the following:

- Support large data arrays (with more than $2^{31}-1$ elements)
- Enable compiling your Fortran code with the `-i8` compiler option

The LP64 interface provides compatibility with the previous Intel® oneAPI Math Kernel Library (oneMKL) versions because "LP64" is just a new name for the only interface that the Intel® oneAPI Math Kernel Library (oneMKL) versions lower than 9.1 provided. Choose the ILP64 interface if your application uses Intel® oneAPI Math Kernel Library (oneMKL) for calculations with large data arrays or the library may be used so in the future.

On 64-bit platforms, selected domains provide API extensions with the `_64` suffix (for example, `SGEMM_64`) for supporting large data arrays in the LP64 library, which enables the mixing of data types in one application. The selected domains and APIs include the following:

- BLAS: Fortran-style APIs for C applications and CBLAS APIs with integer arguments
- LAPACK: Fortran-style APIs for C applications and LAPACKE APIs with integer arguments
- Vector Statistics: API for C applications only on 64-bit platforms for selected functions:
 - Random Number Generators: `vslNewStream_64`; `vslNewStreamEx_64`; all Distribution Generators (for example, `vsRngUniform_64`)
 - Summary Statistics: Task Constructors (`vsLSSNewTask`; `vsLdSSNewTask`)

Intel® oneAPI Math Kernel Library (oneMKL) provides the same include directory for the ILP64 and LP64 interfaces.

Compiling for LP64/ILP64

The table below shows how to compile for the ILP64 and LP64 interfaces:

Fortran	
Compiling for ILP64	<code>ifx -i8 -I<mkl directory>/include ...</code>
Compiling for LP64	<code>ifx -I<mkl directory>/include ...</code>
C or C++	
Compiling for ILP64	<code>icx -DMKL_ILP64 -I<mkl directory>/include ...</code>
Compiling for LP64	<code>icx -I<mkl directory>/include ...</code>

Caution

Linking of an application compiled with the `-i8` or `-DMKL_ILP64` option to the LP64 libraries may result in unpredictable consequences and erroneous output.

Coding for ILP64

You do not need to change existing code if you are not using the ILP64 interface.

To migrate to ILP64 or write new code for ILP64, use appropriate types for parameters of the Intel® oneAPI Math Kernel Library (oneMKL) functions and subroutines:

Integer Types	Fortran	C or C++
32-bit integers	<code>INTEGER*4</code> or	<code>int</code>

Integer Types	Fortran	C or C++
	INTEGER (KIND=4)	
Universal integers for ILP64/ LP64:	INTEGER without specifying KIND	MKL_INT
64-bit for ILP64 32-bit otherwise		
Universal integers for ILP64/ LP64:	INTEGER*8 or INTEGER (KIND=8)	MKL_INT64
64-bit integers		
FFT interface integers for ILP64/ LP64	INTEGER without specifying KIND	MKL_LONG

To determine the type of an integer parameter of a function, use appropriate include files. For functions that support only a Fortran interface, use the C/C++ include files *.h.

The above table explains which integer parameters of functions become 64-bit and which remain 32-bit for ILP64. The table applies to most Intel® oneAPI Math Kernel Library (oneMKL) functions except some Vector Mathematics and Vector Statistics functions, which require integer parameters to be 64-bit or 32-bit regardless of the interface:

- **Vector Mathematics:** The *mode* parameter of the functions is 64-bit.
- **Random Number Generators (RNG):**
All discrete RNG except `viRngUniformBits64` are 32-bit.
The `viRngUniformBits64` generator function and `vs1SkipAheadStream` service function are 64-bit.
- **Summary Statistics:** The *estimate* parameter of the `vs1sSSCompute/vsldSSCompute` function is 64-bit.

Refer to the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference* for more information.

To better understand ILP64 interface details, see also examples.

Limitations

All Intel® oneAPI Math Kernel Library (oneMKL) function domains support ILP64 programming but FFTW interfaces to Intel® oneAPI Math Kernel Library (oneMKL):

- FFTW 2.x wrappers do not support ILP64.
- FFTW 3.x wrappers support ILP64 by a dedicated set of functions `plan_guru64`.

See Also

[High-level Directory Structure](#)

[Intel® oneAPI Math Kernel Library \(oneMKL\) Include Files](#)

[Language Interfaces Support, by Function Domain](#)

[Layered Model Concept](#)

[Directory Structure in Detail](#)

Linking with Fortran 95 Interface Libraries

The `libmkl_blas95*.a` and `libmkl_lapack95*.a` libraries contain Fortran 95 interfaces for BLAS and LAPACK, respectively, which are compiler-dependent. In the Intel® oneAPI Math Kernel Library (oneMKL) package, they are prebuilt for the Intel® Fortran compiler. If you are using a different compiler, build these libraries before using the interface.

See Also

Fortran 95 Interfaces to LAPACK and BLAS

Compiler-dependent Functions and Fortran 90 Modules

Linking with Threading Libraries

Intel® oneAPI Math Kernel Library (oneMKL) threading layer defines how Intel® oneAPI Math Kernel Library (oneMKL) functions utilize multiple computing cores of the system that the application runs on. You must link your application with one appropriate Intel® oneAPI Math Kernel Library (oneMKL) library in this layer, as explained below. Depending on whether this is a threading or a sequential library, Intel® oneAPI Math Kernel Library (oneMKL) runs in a parallel or sequential mode, respectively.

In the *parallel mode*, Intel® oneAPI Math Kernel Library (oneMKL) utilizes multiple processor cores available on your system, uses the OpenMP* or Intel TBB threading technology, and requires a proper threading runtime library (RTL) to be linked with your application. Independently of use of Intel® oneAPI Math Kernel Library (oneMKL), the application may also require a threading RTL. You should link not more than one threading RTL to your application. Threading RTLs are provided by your compiler. Intel® oneAPI Math Kernel Library (oneMKL) provides several threading libraries, each dependent on the threading RTL of a certain compiler, and your choice of the Intel® oneAPI Math Kernel Library (oneMKL) threading library must be consistent with the threading RTL that you use in your application.

The OpenMP RTL of the Intel® compiler is the `libiomp5.so` library, located under `<parent_directory>/compiler/lib`. This RTL is compatible with the GNU* compilers (gcc and gfortran). You can find additional information about the Intel OpenMP RTL at <https://www.openmpRTL.org>.

The Intel TBB RTL of the Intel® compiler is the `libtbb.so` library, located under `<oneAPI_ROOT>/tbb/<version>/lib/<arch>/gcc4.8`. You can find additional information about the Intel TBB RTL at <https://www.threadingbuildingblocks.org>.

In the *sequential mode*, Intel® oneAPI Math Kernel Library (oneMKL) runs unthreaded code, does not require an threading RTL, and does not respond to environment variables and functions controlling the number of threads. Avoid using the library in the sequential mode unless you have a particular reason for that, such as the following:

- Your application needs a threading RTL that none of Intel® oneAPI Math Kernel Library (oneMKL) threading libraries is compatible with
- Your application is already threaded at a top level, and using parallel Intel® oneAPI Math Kernel Library (oneMKL) only degrades the application performance by interfering with that threading
- Your application is intended to be run on a single thread, like a message-passing interface (MPI) application

It is critical to link the application with the proper RTL. The table below explains what library in the Intel® oneAPI Math Kernel Library (oneMKL) threading layer and what threading RTL you should choose under different scenarios:

Application		Intel® oneAPI Math Kernel Library (oneMKL)		RTL Required
Uses OpenMP	Compiled with	Execution Mode	Threading Layer	
no	any compiler	parallel	Static linking: <code>libmkl_intel_thread.a</code> Dynamic linking: <code>libmkl_intel_thread.so</code>	<code>libiomp5.so</code>
no	any compiler	parallel	Static linking: <code>libmkl_tbb_thread.a</code> Dynamic linking:	<code>libtbb.so</code>

Application		Intel® oneAPI Math Kernel Library (oneMKL)		RTL Required
Uses OpenMP	Compiled with	Execution Mode	Threading Layer	
			libmkl_tbb_thread.so	
no	any compiler	sequential	Static linking: libmkl_sequential.a Dynamic linking: libmkl_sequential.so	none [†]
yes	Intel compiler	parallel	Static linking: libmkl_intel_thread.a Dynamic linking: libmkl_intel_thread.so	libiomp5.so
yes	GNU compiler	parallel	Static linking: libmkl_intel_thread.a Dynamic linking: libmkl_intel_thread.so	Recommended! libiomp5.so
yes	GNU compiler	parallel	Static linking: libmkl_gnu_thread.a Dynamic linking: libmkl_gnu_thread.so	GNU OpenMP RTL
yes	any other compiler	parallel	Not supported. Use Intel® oneAPI Math Kernel Library (oneMKL) in the sequential mode.	

[†] For the sequential mode, add the POSIX threads library (libpthread) to your link line because the libmkl_sequential.a and libmkl_sequential.so libraries depend on libpthread.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Layered Model Concept](#)
[Notational Conventions](#)

Linking with Computational Libraries

If you are not using the Intel® oneAPI Math Kernel Library (oneMKL) ScaLAPACK and Cluster Fast Fourier Transforms (FFT), you need to link your application with only one computational library, depending on the linking method:

Static Linking	Dynamic Linking
libmkl_core.a	libmkl_core.so

Computational Libraries for Applications that Use ScaLAPACK or Cluster FFT

ScaLAPACK and Cluster FFT require more computational libraries, which may depend on your architecture.

[†] Also add the library with BLACS routines corresponding to the MPI used.

The following table lists computational libraries for Intel® 64 or Intel® Many Integrated Core Architecture applications that use ScaLAPACK or Cluster FFT.

Computational Libraries for the Intel® 64 or Intel® Many Integrated Core Architecture

Function domain	Static Linking	Dynamic Linking
ScaLAPACK, LP64 interface [‡]	libmkl_scalapack_lp64.a libmkl_core.a	libmkl_scalapack_lp64.so libmkl_core.so
ScaLAPACK, ILP64 interface [‡]	libmkl_scalapack_ilp64.a libmkl_core.a	libmkl_scalapack_ilp64.so libmkl_core.so
Cluster Fourier Transform Functions [‡]	libmkl_cdft_core.a libmkl_core.a	libmkl_cdft_core.so libmkl_core.so

[‡] Also add the library with BLACS routines corresponding to the MPI used.

See Also

[Linking with ScaLAPACK and Cluster FFT](#)

[Using the Link-line Advisor](#)

[Using the ILP64 Interface vs. LP64 Interface](#)

Linking with Compiler Run-time Libraries

Dynamically link `libiomp5` or `libtbb` library even if you link other libraries statically.

Linking to the `libiomp5` statically can be problematic because the more complex your operating environment or application, the more likely redundant copies of the library are included. This may result in performance issues (oversubscription of threads) and even incorrect results.

To link `libiomp5` or `libtbb` dynamically, be sure the `LD_LIBRARY_PATH` environment variable is defined correctly.

See Also

[Scripts to Set Environment Variables](#)

[Layered Model Concept](#)

Linking with System Libraries

To use the Intel® oneAPI Math Kernel Library (oneMKL) FFT, Trigonometric Transform, or Poisson, Laplace, and HelmholtzSolver routines, link also the math support system library by adding `"-lm"` to the link line.

The `libiomp5` library relies on the native `pthread` library for multi-threading. Any time `libiomp5` is required, add `-lpthread` to your link line afterwards (the order of listing libraries is important).

The `libtbb` library relies on the compiler `libstdc++` library for C++ support. Any time `libtbb` is required, add `-lstdc++` to your link line afterwards (the order of listing libraries is important).

NOTE

To link with Intel® oneAPI Math Kernel Library (oneMKL) statically using a GNU compiler, link also the system library `libdl` by adding `-ldl` to your link line. The Intel compiler always passes `-ldl` to the linker.

See Also

[Linking Examples](#)

Building Custom Shared Objects

Custom shared objects reduce the collection of functions available in Intel® oneAPI Math Kernel Library (oneMKL) libraries to those required to solve your particular problems, which helps to save disk space and build your own dynamic libraries for distribution.

The Intel® oneAPI Math Kernel Library (oneMKL) custom shared object builder enables you to create a dynamic library (shared object) containing the selected functions and located in the `share/mkl/tools/builder` directory. The builder contains a makefile and a definition file with the list of functions.

NOTE

The objects in Intel® oneAPI Math Kernel Library (oneMKL) static libraries are position-independent code (PIC), which is not typical for static libraries. Therefore, the custom shared object builder can create a shared object from a subset of Intel® oneAPI Math Kernel Library (oneMKL) functions by picking the respective object files from the static libraries.

Using the Custom Shared Object Builder

To build a custom shared object, use the following command:

```
make target [<options>]
```

The following table lists possible values of `target` and explains what the command does for each value:

Value	Comment
<code>libintel64</code>	The builder uses static Intel® oneAPI Math Kernel Library (oneMKL) interface, threading, and core libraries to build a custom shared object for the Intel® 64 architecture.
<code>help</code>	The command prints Help on the custom shared object builder

The `<options>` placeholder stands for the list of parameters that define macros to be used by the makefile. The following table describes these parameters:

Parameter [Values]	Description
<code>interface = {lp64 ilp64}</code>	The <code>lp64</code> interface uses the 32-bit integer type, while the <code>ilp64</code> interface uses the 64-bit integer type. The default value is <code>lp64</code> .

Parameter [Values]	Description
<code>threading = {parallel sequential}</code> <code>parallel = {intel tbb gnu}</code>	Defines whether to use the Intel® oneAPI Math Kernel Library (oneMKL) in the threaded or sequential mode. The default value is <code>parallel</code>. Specifies whether to use Intel OpenMP, Intel® oneTBB, or GNU OpenMP (for the GNU compiler only). The default value is <code>intel</code>.
<code>cluster = {yes no}</code>	Specifies whether Intel® oneAPI Math Kernel Library (oneMKL) cluster components (BLACS, ScaLAPACK and/or CDFT) are needed to build the custom shared object. The default value is <code>no</code> .
<code>blacs_mpi = {intelmpi openmpi}</code> <code>blacs_name = <lib name></code>	Specifies the pre-compiled Intel® oneAPI Math Kernel Library (oneMKL) BLACS library to use. Ignored if <code>cluster=no</code>. The default value is <code>intelmpi</code>. Specifies the name (without extension) of a custom Intel® oneAPI Math Kernel Library (oneMKL) BLACS library to use. Ignored if <code>cluster=no</code>. 'blacs_mpi' is ignored if 'blacs_name' was explicitly specified. The default value is <code>mkl_blacs_<blacs_mpi>_<interface></code>.
<code>export = <file name></code>	Specifies the full name of the file that contains the list of entry-point functions to be included in the shared object. The default name is <code>user_example_list</code> (no extension).
<code>name = <so name></code>	Specifies the name of the library to be created. By default, the names of the created library is <code>mkl_custom.so</code> .
<code>xerbla = <error handler></code>	Specifies the name of the object file <code><user_xerbla>.o</code> that contains the user's error handler. The makefile adds this error handler to the library for use instead of the default Intel® oneAPI Math Kernel Library (oneMKL) error handler <code>xerbla</code> . If you omit this parameter, the native Intel® oneAPI Math Kernel Library (oneMKL) <code>xerbla</code> is used. See the description of the <code>xerbla</code> function in the Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference to develop your own error handler.
<code>MKLROOT = <mkl directory></code>	Specifies the location of Intel® oneAPI Math Kernel Library (oneMKL) libraries used to build the custom shared object. By default, the builder uses the Intel® oneAPI Math Kernel Library (oneMKL) installation directory.

All of the above parameters are optional.

In the simplest case, the command line is:

```
make libintel64
```

and the missing options have default values. This command creates the `mkl_custom.so` library. The command takes the list of functions from the `user_example_list` file and uses the native Intel® oneAPI Math Kernel Library (oneMKL) error handler `xerbla`.

Here is an example of a more complex case:

```
make libintel64 export=my_func_list name=mkl_small xerbla=my_xerbla.o
```

In this case, the command creates the `mkl_small.so` library. The command takes the list of functions from the `my_func_list` file and uses the error handler of the user `my_xerbla.o`.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

Using the Single Dynamic Library

Composing a List of Functions

To compose a list of functions for a minimal custom shared object needed for your application, you can use the following procedure:

1. Link your application with installed Intel® oneAPI Math Kernel Library (oneMKL) libraries to make sure the application builds.
2. Remove all Intel® oneAPI Math Kernel Library (oneMKL) libraries from the link line and start linking. Unresolved symbols indicate Intel® oneAPI Math Kernel Library (oneMKL) functions that your application uses.
3. Include these functions in the list.

Important

Each time your application starts using more Intel® oneAPI Math Kernel Library (oneMKL) functions, update the list to include the new functions.

See Also

Specifying Function Names

Specifying Function Names

In the file with the list of functions for your custom shared object, adjust function names to the required interface. For example, for Fortran functions append an underscore character "_" to the names as a suffix:

```

dgemm_
ddot_
dgetrf_

```

For more examples, see domain-specific lists of functions in the `<mkl_directory>/share/mkl/tools/builder` directory.

NOTE

The lists of functions are provided in the `<mkl_directory>/share/mkl/tools/builder` directory merely as examples. See [Composing a List of Functions](#) for how to compose lists of functions for your custom shared object.

Tip

Names of Fortran-style routines (BLAS, LAPACK, etc.) can be upper-case, lower-case, or lower-case with the trailing underscore. For example, these names are equivalent:

```

BLAS: dgemm, DGEMM, dgemm_.
LAPACK: dgetrf, DGETRF, dgetrf_.

```

Properly capitalize names of C support functions in the function list. To do this, follow the guidelines below:

1. In the `mkl_service.h` include file, look up a `#define` directive for your function (`mkl_service.h` is included in the `mkl.h` header file).
2. Take the function name from the replacement part of that directive.

For example, the `#define` directive for the `mkl_disable_fast_mm` function is `#define mkl_disable_fast_mm MKL_Disable_Fast_MM`.

Capitalize the name of this function in the list like this: `MKL_Disable_Fast_MM`.

For the names of the Fortran support functions, see the [tip](#).

NOTE

If selected functions have several processor-specific versions, the builder automatically includes them all in the custom library and the dispatcher manages them.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Distributing Your Custom Shared Object

To enable use of your custom shared object in a threaded mode, distribute `libiomp5.so` along with the custom shared object.

Managing Performance and Memory**Improving Performance with Threading**

Intel® oneAPI Math Kernel Library (oneMKL) is extensively parallelized. See [OpenMP* Threaded Functions and Problems](#) and [Functions Threaded with Intel® Threading Building Blocks](#) for lists of threaded functions and problems that can be threaded.

Intel® oneAPI Math Kernel Library (oneMKL) is *thread-safe*, which means that all Intel® oneAPI Math Kernel Library (oneMKL) functions (except the LAPACK deprecated routine `?lacon`) work correctly during simultaneous execution by multiple threads. In particular, any chunk of threaded Intel® oneAPI Math Kernel Library (oneMKL) code provides access for multiple threads to the same shared data, while permitting only one thread at any given time to access a shared piece of data. Therefore, you can call Intel® oneAPI Math Kernel Library (oneMKL) from multiple threads and not worry about the function instances interfering with each other.

If you are using OpenMP* threading technology, you can use the environment variable `OMP_NUM_THREADS` to specify the number of threads or the equivalent OpenMP run-time function calls. Intel® oneAPI Math Kernel Library (oneMKL) also offers variables that are independent of OpenMP, such as `asmkl_num_threads`, and equivalent Intel® oneAPI Math Kernel Library (oneMKL) functions for thread management. The Intel® oneAPI Math Kernel Library (oneMKL) variables are always inspected first, then the OpenMP variables are examined, and if neither is used, the OpenMP software chooses the default number of threads.

By default, Intel® oneAPI Math Kernel Library (oneMKL) uses the number of OpenMP threads equal to the number of physical cores on the system.

If you are using the Intel TBB threading technology, the OpenMP threading controls, such as the `OMP_NUM_THREADS` environment variable or `mkls_num_threads` function, have no effect. Use the Intel TBB application programming interface to control the number of threads.

To achieve higher performance, set the number of threads to the number of processors or physical cores, as summarized in [Techniques to Set the Number of Threads](#).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

Managing Multi-core Performance

OpenMP* Threaded Functions and Problems

The following Intel® oneAPI Math Kernel Library (oneMKL) function domains are threaded with the OpenMP* technology:

- Direct sparse solver.
- LAPACK.

For a list of threaded routines, see [LAPACK Routines](#).

- Level1 and Level2 BLAS.

For a list of threaded routines, see [BLAS Level1 and Level2 Routines](#).

- All Level 3 BLAS and all Sparse BLAS routines except Level 2 Sparse Triangular solvers.
- All Vector Mathematics functions (except service functions).
- FFT.

For a list of FFT transforms that can be threaded, see [Threaded FFT Problems](#).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

LAPACK Routines

In this section, ? stands for a precision prefix of *each* flavor of the respective routine and may have the value of s, d, c, or z.

The following LAPACK routines are threaded with OpenMP*:

- Linear equations, computational routines:
 - Factorization: ?getrf, ?getrfnpi, ?gbtrf, ?potrf, ?pptrf, ?sytrf, ?hetrf, ?sptrf, ?hptrf
 - Solving: ?dtttrs, ?gbtrs, ?gttrs, ?ppttrs, ?pbtrs, ?pttrs, ?sytrs, ?sptrs, ?hptrs, ?tptrs, ?tbtrs
- Orthogonal factorization, computational routines:
 - ?geqrf, ?ormqr, ?unmqr, ?ormql, ?unmlq, ?ormql, ?unmql, ?ormrq, ?unmrq
- Singular Value Decomposition, computational routines:
 - ?gebrd, ?bdsqr
- Symmetric Eigenvalue Problems, computational routines:
 - ?sytrd, ?hetrd, ?sptrd, ?hptrd, ?steqr, ?stedc.
- Generalized Nonsymmetric Eigenvalue Problems, computational routines:
 - chgeqz/zhgeqz.

A number of other LAPACK routines, which are based on threaded LAPACK or BLAS routines, make effective use of OpenMP* parallelism:

?gesv, ?posv, ?gels, ?gesvd, ?syev, ?heev, cgegs/zgegs, cgegv/zgegv, cgges/zgges, cggesx/zggesx, cggev/zggev, cggevx/zggev, and so on.

Threaded BLAS Level1 and Level2 Routines

In the following list, ? stands for a precision prefix of *each* flavor of the respective routine and may have the value of s, d, c, or z.

The following routines are threaded with OpenMP*:

- Level1 BLAS:
?axpy, ?copy, ?swap, ddot/sdot, cdotc, drot/srot
- Level2 BLAS:
?gemv, ?trsv, ?trmv, dsyr/ssyr, dsyr2/ssyr2, dsymv/ssymv

Threaded FFT Problems

The following characteristics of a specific problem determine whether your FFT computation may be threaded with OpenMP*:

- rank
- domain
- size/length
- precision (single or double)
- placement (in-place or out-of-place)
- strides
- number of transforms
- layout (for example, interleaved or split layout of complex data)

Most FFT problems are threaded. In particular, computation of multiple transforms in one call (number of transforms > 1) is threaded. Details of which transforms are threaded follow.

One-dimensional (1D) transforms

1D transforms are threaded in many cases.

1D complex-to-complex (c2c) transforms of size N using interleaved complex data layout are threaded under the following conditions depending on the architecture:

Architecture	Conditions
Intel® 64	N is a power of 2, $\log_2(N) > 9$, the transform is double-precision out-of-place, and input/output strides equal 1.
Any	N is composite, $\log_2(N) > 16$, and input/output strides equal 1.

1D complex-to-complex transforms using split-complex layout are not threaded.

Multidimensional transforms

All multidimensional transforms on large-volume data are threaded.

Functions Threaded with Intel® Threading Building Blocks

In this section, ? stands for a precision prefix or suffix of the routine name and may have the value of s, d, c, or z.

The following Intel® oneAPI Math Kernel Library (oneMKL) function domains are threaded with Intel® Threading Building Blocks (oneTBB):

- LAPACK.
For a list of threaded routines, see [LAPACK Routines](#).
 - Entire Level3 BLAS.
 - Level2 BLAS – ?GEMV.
 - Fast Poisson, Laplace, and Helmholtz Solver (Poisson Library).
 - All Vector Mathematics functions (except service functions).
 - Intel® oneAPI Math Kernel Library (oneMKL) PARDISO, a direct sparse solver based on Parallel Direct Sparse Solver (PARDISO*).
- For details, see [oneMKL PARDISO Steps](#).
- Sparse BLAS.

For a list of threaded routines, see [Sparse BLAS Routines](#).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

LAPACK Routines

The following LAPACK routines are threaded with oneTBB:

?geqrf, ?gelqf, ?getrf, ?potrf, ?unmqr*, ?ormqr*, ?unmrq*, ?ormrq*, ?unmlq*, ?ormlq*, ?unmql*, ?ormql*, ?sytrd, ?hetrd, ?syev, ?heev, and ?latrd.

A number of other LAPACK routines, which are based on threaded LAPACK or BLAS routines, make effective use of oneTBB threading:

?getrs, ?gesv, ?potrs, ?bdsqr, and ?gels.

oneMKL PARDISO Steps

Intel® oneAPI Math Kernel Library (oneMKL) PARDISO is threaded with oneTBB in the reordering and factorization steps. However, routines performing the solving step are still called sequentially when using oneTBB.

Sparse BLAS Routines

The Sparse BLAS inspector-executor application programming interface routines `mkl_sparse_?_mv` are threaded with oneTBB for the general compressed sparse row (CSR) and block sparse row (BSR) formats.

The following Sparse BLAS inspector-executor application programming routines are threaded with oneTBB:

- `mkl_sparse_?_mv` using the general compressed sparse row (CSR) and block sparse row (BSR) matrix formats.
- `mkl_sparse_?_mm` using the general CSR sparse matrix format and both row and column major storage formats for the dense matrix.

Avoiding Conflicts in the Execution Environment

Certain situations can cause conflicts in the execution environment that make the use of threads in Intel® oneAPI Math Kernel Library (oneMKL) problematic. This section briefly discusses why these problems exist and how to avoid them.

If your program is parallelized by means other than Intel® OpenMP* run-time library (RTL) and Intel® Threading Building Blocks (oneTBB) RTL, several calls to Intel® oneAPI Math Kernel Library (oneMKL) may operate in a multithreaded mode at the same time and result in slow performance due to overuse of machine resources.

The following table considers several cases where the conflicts may arise and provides recommendations depending on your threading model:

Threading model	Discussion
You parallelize the program using the technology other than Intel OpenMP and oneTBB (for example: <code>pthreads</code> on Linux*).	If more than one thread calls Intel® oneAPI Math Kernel Library (oneMKL), and the function being called is threaded, it may be important that you turn off Intel® oneAPI Math Kernel Library (oneMKL) threading. Set the number of threads to one by any of the available means (see Techniques to Set the Number of Threads).

Threading model	Discussion
You parallelize the program using OpenMP directives and/or pragmas and compile the program using a non-Intel compiler.	To avoid simultaneous activities of multiple threading RTLs, link the program against the Intel® oneAPI Math Kernel Library (oneMKL) threading library that matches the compiler you use (see Linking Examples on how to do this). If this is not possible, use Intel® oneAPI Math Kernel Library (oneMKL) in the sequential mode. To do this, you should link with the appropriate threading library: <code>libmkl_sequential.a</code> or <code>libmkl_sequential.so</code> (see Appendix C: Directory Structure in Detail).
You thread the program using oneTBB threading technology and compile the program using a non-Intel compiler.	To avoid simultaneous activities of multiple threading RTLs, link the program against the Intel® oneAPI Math Kernel Library (oneMKL) oneTBB threading library and oneTBB RTL if it matches the compiler you use. If this is not possible, use Intel® oneAPI Math Kernel Library (oneMKL) in the sequential mode. To do this, link with the appropriate threading library: <code>libmkl_sequential.a</code> or <code>libmkl_sequential.so</code> (see Appendix C: Directory Structure in Detail).
You run multiple programs calling Intel® oneAPI Math Kernel Library (oneMKL) on a multiprocessor system, for example, a program parallelized using a message-passing interface (MPI).	The threading RTLs from different programs you run may place a large number of threads on the same processor on the system and therefore overuse the machine resources. In this case, one of the solutions is to set the number of threads to one by any of the available means (see Techniques to Set the Number of Threads). The Intel® Distribution for LINPACK* Benchmark section discusses another solution for a Hybrid (OpenMP* + MPI) mode.

Using the `mkl_set_num_threads` and `mkl_domain_set_num_threads` functions to control parallelism of Intel® oneAPI Math Kernel Library (oneMKL) from parallel user threads may result in a race condition that impacts the performance of the application because these functions operate on internal control variables that are global, that is, apply to all threads. For example, if parallel user threads call these functions to set different numbers of threads for the same function domain, the number of threads actually set is unpredictable. To avoid this kind of data races, use the `mkl_set_num_threads_local` function (see the "Support Functions" section in the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference* for the function description).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Using Additional Threading Control](#)
[Linking with Compiler Support RTLs](#)

Techniques to Set the Number of Threads

Use the following techniques to specify the number of OpenMP threads to use in Intel® oneAPI Math Kernel Library (oneMKL):

- Set one of the OpenMP or Intel® oneAPI Math Kernel Library (oneMKL) environment variables:
 - `OMP_NUM_THREADS`
 - `MKL_NUM_THREADS`
 - `MKL_DOMAIN_NUM_THREADS`
- Call one of the OpenMP or Intel® oneAPI Math Kernel Library (oneMKL) functions:
 - `omp_set_num_threads()`
 - `mkl_set_num_threads()`

- `mkl_domain_set_num_threads()`
- `mkl_set_num_threads_local()`

NOTE

A call to the `mkl_set_num_threads` or `mkl_domain_set_num_threads` function changes the number of OpenMP threads available to all in-progress calls (in concurrent threads) and future calls to Intel® oneAPI Math Kernel Library (oneMKL) and may result in slow Intel® oneAPI Math Kernel Library (oneMKL) performance and/or race conditions reported by run-time tools, such as Intel® Inspector.

To avoid such situations, use the `mkl_set_num_threads_local` function (see the "Support Functions" section in the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference* for the function description).

When choosing the appropriate technique, take into account the following rules:

- The Intel® oneAPI Math Kernel Library (oneMKL) threading controls take precedence over the OpenMP controls because they are inspected first.
- A function call takes precedence over any environment settings. The exception, which is a consequence of the previous rule, is that a call to the OpenMP subroutine `omp_set_num_threads()` does not have precedence over the settings of Intel® oneAPI Math Kernel Library (oneMKL) environment variables such as `asmkl_num_threads`. See [Using Additional Threading Control](#) for more details.
- You cannot change run-time behavior in the course of the run using the environment variables because they are read only once at the first call to Intel® oneAPI Math Kernel Library (oneMKL).

If you use the Intel TBB threading technology, read the documentation for the `tbb::task_scheduler_init` class at <https://www.threadingbuildingblocks.org/documentation> to find out how to specify the number of threads.

Setting the Number of Threads Using an OpenMP* Environment Variable

You can set the number of threads using the environment variable `OMP_NUM_THREADS`. To change the number of OpenMP threads, use the appropriate command in the command shell in which the program is going to run, for example:

- For the bash shell, enter:
`export OMP_NUM_THREADS=<number of threads to use>`
- For the csh or tcsh shell, enter:
`setenv OMP_NUM_THREADS <number of threads to use>`

See Also

[Using Additional Threading Control](#)

Changing the Number of OpenMP* Threads at Run Time

You cannot change the number of OpenMP threads at run time using environment variables. However, you can call OpenMP routines to do this. Specifically, the following sample code shows how to change the number of threads during run time using the `omp_set_num_threads()` routine. For more options, see also [Techniques to Set the Number of Threads](#).

The example is provided for both C and Fortran languages. To run the example in C, use the `omp.h` header file from the Intel(R) compiler package. If you do not have the Intel compiler but wish to explore the functionality in the example, use Fortran API for `omp_set_num_threads()` rather than the C version. For example, `omp_set_num_threads_( &i_one );`

```
// ***** C language *****
#include "omp.h"
#include "mkl.h"
#include <stdio.h>
#define SIZE 1000
```

```

int main(int args, char *argv[]){
double *a, *b, *c;
a = (double*)malloc(sizeof(double)*SIZE*SIZE);
b = (double*)malloc(sizeof(double)*SIZE*SIZE);
c = (double*)malloc(sizeof(double)*SIZE*SIZE);
double alpha=1, beta=1;
int m=SIZE, n=SIZE, k=SIZE, lda=SIZE, ldb=SIZE, ldc=SIZE, i=0, j=0;
char transa='n', transb='n';
for( i=0; i<SIZE; i++)
{
    for( j=0; j<SIZE; j++)
    {
        a[i*SIZE+j]= (double)(i+j);
        b[i*SIZE+j]= (double)(i*j);
        c[i*SIZE+j]= (double)0;
    }
}
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
m, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
printf("row\ta\tc\n");
for ( i=0;i<10;i++)
{
printf("%d:\t%f\t%f\n", i, a[i*SIZE], c[i*SIZE]);
}
omp_set_num_threads(1);
for( i=0; i<SIZE; i++)
{
    for( j=0; j<SIZE; j++)
    {
        a[i*SIZE+j]= (double)(i+j);
        b[i*SIZE+j]= (double)(i*j);
        c[i*SIZE+j]= (double)0;
    }
}
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
m, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
printf("row\ta\tc\n");
for ( i=0;i<10;i++)
{
printf("%d:\t%f\t%f\n", i, a[i*SIZE], c[i*SIZE]);
}
omp_set_num_threads(2);
for( i=0; i<SIZE; i++)
{
    for( j=0; j<SIZE; j++)
    {
        a[i*SIZE+j]= (double)(i+j);
        b[i*SIZE+j]= (double)(i*j);
        c[i*SIZE+j]= (double)0;
    }
}
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
m, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
printf("row\ta\tc\n");
for ( i=0;i<10;i++)
{
printf("%d:\t%f\t%f\n", i, a[i*SIZE],
c[i*SIZE]);
}

```

```

}
free (a);
free (b);
free (c);
return 0;
}

```

```

// ***** Fortran language *****
PROGRAM DGEMM_DIFF_THREADS
INTEGER N, I, J
PARAMETER (N=100)
REAL*8 A(N,N), B(N,N), C(N,N)
REAL*8 ALPHA, BETA

ALPHA = 1.1
BETA = -1.2
DO I=1,N
    DO J=1,N
        A(I,J) = I+J
        B(I,J) = I*j
        C(I,J) = 0.0
    END DO
END DO
CALL DGEMM('N','N',N,N,N,ALPHA,A,N,B,N,BETA,C,N)
print *, 'Row A C'
DO i=1,10
write(*, '(I4,F20.8,F20.8)') I, A(1,I), C(1,I)
END DO
CALL OMP_SET_NUM_THREADS(1);
DO I=1,N
    DO J=1,N
        A(I,J) = I+J
        B(I,J) = I*j
        C(I,J) = 0.0
    END DO
END DO
CALL DGEMM('N','N',N,N,N,ALPHA,A,N,B,N,BETA,C,N)
print *, 'Row A C'
DO i=1,10
write(*, '(I4,F20.8,F20.8)') I, A(1,I), C(1,I)
END DO
CALL OMP_SET_NUM_THREADS(2);
DO I=1,N
    DO J=1,N
        A(I,J) = I+J
        B(I,J) = I*j
        C(I,J) = 0.0
    END DO
END DO
CALL DGEMM('N','N',N,N,N,ALPHA,A,N,B,N,BETA,C,N)
print *, 'Row A C'
DO i=1,10
write(*, '(I4,F20.8,F20.8)') I, A(1,I), C(1,I)
END DO

```

```
STOP
END
```

Using Additional Threading Control

oneMKL-specific Environment Variables for OpenMP Threading Control

Intel® oneAPI Math Kernel Library (oneMKL) provides environment variables and support functions to control Intel® oneAPI Math Kernel Library (oneMKL) threading independently of OpenMP. The Intel® oneAPI Math Kernel Library (oneMKL)-specific threading controls take precedence over their OpenMP equivalents. Use the Intel® oneAPI Math Kernel Library (oneMKL)-specific threading controls to distribute OpenMP threads between Intel® oneAPI Math Kernel Library (oneMKL) and the rest of your program.

NOTE

Some Intel® oneAPI Math Kernel Library (oneMKL) routines may use fewer OpenMP threads than suggested by the threading controls if either the underlying algorithms do not support the suggested number of OpenMP threads or the routines perform better with fewer OpenMP threads because of lower OpenMP overhead and/or better data locality. Set the `MKL_DYNAMIC` environment variable to `FALSE` or call `mkl_set_dynamic(0)` to use the suggested number of OpenMP threads whenever the algorithms permit and regardless of OpenMP overhead and data locality.

The table below lists the Intel® oneAPI Math Kernel Library (oneMKL) environment variables for threading control, their equivalent functions, and OMP counterparts:

Environment Variable	Support Function	Comment	Equivalent OpenMP* Environment Variable
MKL_NUM_THREADS	<code>mkl_set_num_threads</code> <code>mkl_set_num_threads_local</code>	Suggests the number of OpenMP threads to use.	OMP_NUM_THREADS
MKL_DOMAIN_NUM_THREADS	<code>mkl_domain_set_num_threads</code>	Suggests the number of OpenMP threads for a particular function domain.	
MKL_DYNAMIC	<code>mkl_set_dynamic</code>	Enables Intel® oneAPI Math Kernel Library (oneMKL) to dynamically change the number of OpenMP threads.	OMP_DYNAMIC

NOTE

Call `mkl_set_num_threads()` to force Intel® oneAPI Math Kernel Library (oneMKL) to use a given number of OpenMP threads and prevent it from reacting to the environment variables `MKL_NUM_THREADS`, `MKL_DOMAIN_NUM_THREADS`, and `OMP_NUM_THREADS`.

The example below shows how to force Intel® oneAPI Math Kernel Library (oneMKL) to use one thread:

```
// ***** C language *****
```

```
#include <mkl.h>
...
mkl_set_num_threads ( 1 );
```

```
// ***** Fortran language *****
...
call mkl_set_num_threads( 1 )
```

MKL_DYNAMIC

The `MKL_DYNAMIC` environment variable enables Intel® oneAPI Math Kernel Library (oneMKL) to dynamically change the number of threads.

The default value of `MKL_DYNAMIC` is `TRUE`, regardless of `OMP_DYNAMIC`, whose default value may be `FALSE`.

When `MKL_DYNAMIC` is `TRUE`, Intel® oneAPI Math Kernel Library (oneMKL) may use fewer OpenMP threads than the maximum number you specify.

For example, `MKL_DYNAMIC` set to `TRUE` enables optimal choice of the number of threads in the following cases:

- If the requested number of threads exceeds the number of physical cores (perhaps because of using the Intel® Hyper-Threading Technology), Intel® oneAPI Math Kernel Library (oneMKL) scales down the number of OpenMP threads to the number of physical cores.
- If you are able to detect the presence of a message-passing interface (MPI), but cannot determine whether it has been called in a thread-safe mode, Intel® oneAPI Math Kernel Library (oneMKL) runs one OpenMP thread.

When `MKL_DYNAMIC` is `FALSE`, Intel® oneAPI Math Kernel Library (oneMKL) uses the suggested number of OpenMP threads whenever the underlying algorithms permit. For example, if you attempt to do a size one matrix-matrix multiply across eight threads, the library may instead choose to use only one thread because it is impractical to use eight threads in this event.

If Intel® oneAPI Math Kernel Library (oneMKL) is called from an OpenMP parallel region in your program, Intel® oneAPI Math Kernel Library (oneMKL) uses only one thread by default. If you want Intel® oneAPI Math Kernel Library (oneMKL) to go parallel in such a call, link your program against an OpenMP threading RTL supported by Intel® oneAPI Math Kernel Library (oneMKL) and set the environment variables:

- `OMP_NESTED` to `TRUE`
- `OMP_DYNAMIC` and `MKL_DYNAMIC` to `FALSE`
- `MKL_NUM_THREADS` to some reasonable value

With these settings, Intel® oneAPI Math Kernel Library (oneMKL) uses `MKL_NUM_THREADS` threads when it is called from the OpenMP parallel region in your program.

In general, set `MKL_DYNAMIC` to `FALSE` only under circumstances that Intel® oneAPI Math Kernel Library (oneMKL) is unable to detect, for example, to use nested parallelism where the library is already called from a parallel section.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

MKL_DOMAIN_NUM_THREADS

The `MKL_DOMAIN_NUM_THREADS` environment variable suggests the number of OpenMP threads for a particular function domain.

MKL_DOMAIN_NUM_THREADS accepts a string value <MKL-env-string>, which must have the following format:

```
<MKL-env-string> ::= <MKL-domain-env-string> { <delimiter><MKL-domain-env-string> }
<delimiter> ::= [ <space-symbol>* ] ( <space-symbol> | <comma-symbol> | <semicolon-symbol> | <colon-symbol> ) [ <space-symbol>* ]
<MKL-domain-env-string> ::= <MKL-domain-env-name><uses><number-of-threads>
<MKL-domain-env-name> ::= MKL_DOMAIN_ALL | MKL_DOMAIN_BLAS | MKL_DOMAIN_FFT |
MKL_DOMAIN_VML | MKL_DOMAIN_PARDISO
<uses> ::= [ <space-symbol>* ] ( <space-symbol> | <equality-sign> | <comma-symbol> )
[ <space-symbol>* ]
<number-of-threads> ::= <positive-number>
<positive-number> ::= <decimal-positive-number> | <octal-number> | <hexadecimal-number>
```

In the syntax above, values of <MKL-domain-env-name> indicate function domains as follows:

MKL_DOMAIN_ALL	All function domains
MKL_DOMAIN_BLAS	BLAS Routines
MKL_DOMAIN_FFT	non-cluster Fourier Transform Functions
MKL_DOMAIN_LAPACK	LAPACK Routines
MKL_DOMAIN_VML	Vector Mathematics (VM)
MKL_DOMAIN_PARDISO	Intel® oneAPI Math Kernel Library (oneMKL) PARDISO, a direct sparse solver based on Parallel Direct Sparse Solver (PARDISO*)

For example, you could set the MKL_DOMAIN_NUM_THREADS environment variable to any of the following string variants, in this case, defining three specific domain variables internal to Intel® oneAPI Math Kernel Library (oneMKL):

```
MKL_DOMAIN_NUM_THREADS="MKL_DOMAIN_ALL=2, MKL_DOMAIN_BLAS=1, MKL_DOMAIN_FFT=4"
MKL_DOMAIN_NUM_THREADS="MKL_DOMAIN_ALL 2 : MKL_DOMAIN_BLAS 1 : MKL_DOMAIN_FFT 4"
MKL_DOMAIN_NUM_THREADS="MKL_DOMAIN_ALL=2 : MKL_DOMAIN_BLAS=1 : MKL_DOMAIN_FFT=4"
MKL_DOMAIN_NUM_THREADS="MKL_DOMAIN_ALL=2; MKL_DOMAIN_BLAS=1; MKL_DOMAIN_FFT=4"
MKL_DOMAIN_NUM_THREADS="MKL_DOMAIN_ALL=2 MKL_DOMAIN_BLAS 1, MKL_DOMAIN_FFT 4"
MKL_DOMAIN_NUM_THREADS="MKL_DOMAIN_ALL,2: MKL_DOMAIN_BLAS 1, MKL_DOMAIN_FFT,4"
```

NOTE Prepend the appropriate `set/export/setenv` command for your command shell and operating system. Refer to [Setting the Environment Variables for Threading Control](#) for more details.

The global variables MKL_DOMAIN_ALL, MKL_DOMAIN_BLAS, MKL_DOMAIN_FFT, MKL_DOMAIN_VML, and MKL_DOMAIN_PARDISO, as well as the interface for the Intel® oneAPI Math Kernel Library (oneMKL) threading control functions, can be found in the `themkl.h` header file.

NOTE You can retrieve the values of the specific domain variables that you have set in your code with a call to the `mkl_get_domain_max_threads(domain_name)` function per the [Fortran](#) and [C interface](#) with the desired domain variable name.

This table illustrates how values of MKL_DOMAIN_NUM_THREADS are interpreted.

Value of MKL_DOMAIN_NUM_THREADS	Interpretation
MKL_DOMAIN_ALL=4	All parts of Intel® oneAPI Math Kernel Library (oneMKL) should try four OpenMP threads. The actual number of threads may be still different because of the MKL_DYNAMIC setting or system resource issues. The setting is equivalent to MKL_NUM_THREADS = 4.
MKL_DOMAIN_ALL=1, MKL_DOMAIN_BLAS=4	All parts of Intel® oneAPI Math Kernel Library (oneMKL) should try one OpenMP thread, except for BLAS, which is suggested to try four threads.
MKL_DOMAIN_VML=2	VM should try two OpenMP threads. The setting affects no other part of Intel® oneAPI Math Kernel Library (oneMKL).

Be aware that the domain-specific settings take precedence over the overall ones. For example, the "MKL_DOMAIN_BLAS=4" value of MKL_DOMAIN_NUM_THREADS suggests trying four OpenMP threads for BLAS, regardless of later setting MKL_NUM_THREADS, and a function call "mkl_domain_set_num_threads (4, MKL_DOMAIN_BLAS);" suggests the same, regardless of later calls to mkl_set_num_threads(). However, a function call with input "MKL_DOMAIN_ALL", such as "mkl_domain_set_num_threads (4, MKL_DOMAIN_ALL);" is equivalent to "mkl_set_num_threads(4)", and thus it will be overwritten by later calls to mkl_set_num_threads. Similarly, the environment setting of MKL_DOMAIN_NUM_THREADS with "MKL_DOMAIN_ALL=4" will be overwritten with MKL_NUM_THREADS = 2.

Whereas the MKL_DOMAIN_NUM_THREADS environment variable enables you set several variables at once, for example, "MKL_DOMAIN_BLAS=4,MKL_DOMAIN_FFT=2", the corresponding function does not take string syntax. So, to do the same with the function calls, you may need to make several calls, which in this example are as follows:

```
mkl_domain_set_num_threads ( 4, MKL_DOMAIN_BLAS );
mkl_domain_set_num_threads ( 2, MKL_DOMAIN_FFT );
```

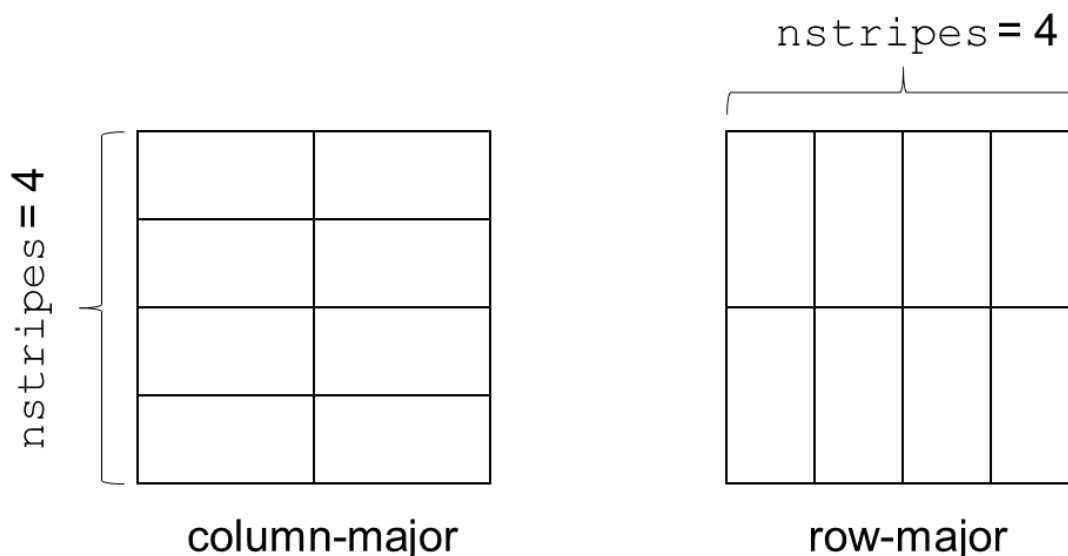
MKL_NUM_STRIPEs

The MKL_NUM_STRIPEs environment variable controls the Intel® oneAPI Math Kernel Library (oneMKL) threading algorithm for ?gemm functions. When MKL_NUM_STRIPEs is set to a positive integer value *nstripes*, Intel® oneAPI Math Kernel Library (oneMKL) tries to use a number of partitions equal to *nstripes* along the leading dimension of the output matrix.

The following table explains how the value *nstripes* of MKL_NUM_STRIPEs defines the partitioning algorithm used by Intel® oneAPI Math Kernel Library (oneMKL) for ?gemm output matrix; *max_threads_for_mkl* denotes the maximum number of OpenMP threads for Intel® oneAPI Math Kernel Library (oneMKL):

Value of MKL_NUM_STRIPEs	Partitioning Algorithm
$1 < nstripes < (max_threads_for_mkl / 2)$	2D partitioning with the number of partitions equal to <i>nstripes</i> : - Horizontal, for column-major ordering. - Vertical, for row-major ordering.
$nstripes = 1$	1D partitioning algorithm along the opposite direction of the leading dimension.
$nstripes \geq (max_threads_for_mkl / 2)$	1D partitioning algorithm along the leading dimension.
$nstripes < 0$	The default Intel® oneAPI Math Kernel Library (oneMKL) threading algorithm.

The following figure shows the partitioning of an output matrix for $nstrips = 4$ and a total number of 8 OpenMP threads for column-major and row-major orderings:



You can use support functions `mkl_set_num_stripes` and `mkl_get_num_stripes` to set and query the number of stripes, respectively.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Setting the Environment Variables for Threading Control

To set the environment variables used for threading control, in the command shell in which the program is going to run, enter the `export` or `setenv` commands, depending on the shell you use.

For a bash shell, use the `export` commands:

```
export <VARIABLE NAME>=<value>
```

For example:

```
export MKL_NUM_THREADS=4
export MKL_DOMAIN_NUM_THREADS="MKL_DOMAIN_ALL=1, MKL_DOMAIN_BLAS=4"
export MKL_DYNAMIC=FALSE
export MKL_NUM_STRIPES=4
```

For the csh or tcsh shell, use the `setenv` commands:

```
setenv <VARIABLE NAME><value>.
```

For example:

```
setenv MKL_NUM_THREADS 4
setenv MKL_DOMAIN_NUM_THREADS "MKL_DOMAIN_ALL=1, MKL_DOMAIN_BLAS=4"
setenv MKL_DYNAMIC FALSE
setenv MKL_NUM_STRIPES 4
```

Calling oneMKL Functions from Multi-threaded Applications

This section summarizes typical usage models and available options for calling Intel® oneAPI Math Kernel Library (oneMKL) functions from multi-threaded applications. These recommendations apply to any multi-threading environments: OpenMP*, Intel® Threading Building Blocks, POSIX* threads, and others.

Usage model: disable oneMKL internal threading for the whole application

When used: Intel® oneAPI Math Kernel Library (oneMKL) internal threading interferes with application's own threading or may slow down the application.

Example: the application is threaded at top level, or the application runs concurrently with other applications.

Options:

- Link statically or dynamically with the sequential library
- Link with the Single Dynamic Library `mkl_rt.so` and select the sequential library using an environment variable or a function call:
 - Set `MKL_THREADING_LAYER=sequential`
 - Call `mkl_set_threading_layer(MKL_THREADING_SEQUENTIAL)`[‡]

Usage model: partition system resources among application threads

When used: application threads are specialized for a particular computation.

Example: one thread solves equations on all cores but one, while another thread running on a single core updates a database.

Linking Options:

- Link statically or dynamically with a threading library
- Link with the Single Dynamic Library `mkl_rt.so` and select a threading library using an environment variable or a function call:
 - set `MKL_THREADING_LAYER=intel` or `MKL_THREADING_LAYER=tbb`
 - call `mkl_set_threading_layer(MKL_THREADING_INTEL)` or `mkl_set_threading_layer(MKL_THREADING_TBB)`

Other Options for OpenMP Threading:

- Set the `MKL_NUM_THREADS` environment variable to a desired number of OpenMP threads for Intel® oneAPI Math Kernel Library (oneMKL).
- Set the `MKL_DOMAIN_NUM_THREADS` environment variable to a desired number of OpenMP threads for Intel® oneAPI Math Kernel Library (oneMKL) for a particular function domain.

Use if the application threads work with different Intel® oneAPI Math Kernel Library (oneMKL) function domains.

- Call `mkl_set_num_threads()`

Use to globally set a desired number of OpenMP threads for Intel® oneAPI Math Kernel Library (oneMKL) at run time.

- Call `mkl_domain_set_num_threads()`.

Use if at some point application threads start working with different Intel® oneAPI Math Kernel Library (oneMKL) function domains.

- Call `mkl_set_num_threads_local()`.

Use to set the number of OpenMP threads for Intel® oneAPI Math Kernel Library (oneMKL) called from a particular thread.

NOTE

If your application uses OpenMP* threading, you may need to provide additional settings:

- Set the environment variable `OMP_NESTED=TRUE`, or alternatively call `omp_set_nested(1)`, to enable OpenMP nested parallelism.
- Set the environment variable `MKL_DYNAMIC=FALSE`, or alternatively call `mkl_set_dynamic(0)`, to prevent Intel® oneAPI Math Kernel Library (oneMKL) from dynamically reducing the number of OpenMP threads in nested parallel regions.

† For details of the mentioned functions, see the Support Functions section of the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference*, available in the [Intel® Software Documentation Library](#).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Linking with Threading Libraries](#)

[Dynamically Selecting the Interface and Threading Layer](#)

[oneMKL-specific Environment Variables for OpenMP Threading Control](#)

[MKL_DOMAIN_NUM_THREADS](#)

[Avoiding Conflicts in the Execution Environment](#)

[Intel® Software Documentation Library](#)

Using Intel® Hyper-Threading Technology

Intel® Hyper-Threading Technology (Intel® HT Technology) is especially effective when each thread performs different types of operations and when there are under-utilized resources on the processor. However, Intel® oneAPI Math Kernel Library (oneMKL) fits neither of these criteria because the threaded portions of the library execute at high efficiencies using most of the available resources and perform identical operations on each thread. You may obtain higher performance by disabling Intel HT Technology.

If you run with Intel HT Technology enabled, performance may be especially impacted if you run on fewer threads than physical cores. Moreover, if, for example, there are two threads to every physical core, the thread scheduler may assign two threads to some cores and ignore the other cores altogether. If you are using the OpenMP* library of the Intel Compiler, read the respective User Guide on how to best set the thread affinity interface to avoid this situation. For Intel® oneAPI Math Kernel Library (oneMKL), apply the following setting:

```
export KMP_AFFINITY=granularity=fine,compact,1,0
```

If you are using the Intel TBB threading technology, read the documentation on the `tbb::affinity_partitioner` class at <https://www.threadingbuildingblocks.org/documentation> to find out how to affinitize Intel TBB threads.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Managing Multi-core Performance

You can obtain best performance on systems with multi-core processors by requiring that threads do not migrate from core to core. To do this, bind threads to the CPU cores by setting an affinity mask to threads. Use one of the following options:

- OpenMP facilities (if available), for example, the `KMP_AFFINITY` environment variable using the Intel OpenMP library
- A system function, as explained below
- Intel TBB facilities (if available), for example, the `tbb::affinity_partitioner` class (for details, see <https://www.threadingbuildingblocks.org/documentation>)

Consider the following performance issue:

- The system has two sockets with two cores each, for a total of four cores (CPUs).
- The application sets the number of OpenMP threads to two and calls Intel® oneAPI Math Kernel Library (oneMKL) to perform a Fourier transform. This call takes considerably different amounts of time from run to run.

To resolve this issue, before calling Intel® oneAPI Math Kernel Library (oneMKL), set an affinity mask for each OpenMP thread using the `KMP_AFFINITY` environment variable or the `sched_setaffinity` system function. The following code example shows how to resolve the issue by setting an affinity mask by operating system means using the Intel compiler. The code calls the function `sched_setaffinity` to bind the threads to the cores on different sockets. Then the Intel® oneAPI Math Kernel Library (oneMKL) FFT function is called:

```
#define _GNU_SOURCE //for using the GNU CPU affinity
// (works with the appropriate kernel and glibc)
// Set affinity mask
#include <sched.h>
#include <stdio.h>
#include <unistd.h>
#include <omp.h>
int main(void) {
    int NCPUs = sysconf(_SC_NPROCESSORS_CONF);
    printf("Using thread affinity on %i NCPUs\n", NCPUs);
#pragma omp parallel default(shared)
    {
        cpu_set_t new_mask;
        cpu_set_t was_mask;
        int tid = omp_get_thread_num();

        CPU_ZERO(&new_mask);

        // 2 packages x 2 cores/pkg x 1 threads/core (4 total cores)
        CPU_SET(tid==0 ? 0 : 2, &new_mask);

        if (sched_getaffinity(0, sizeof(was_mask), &was_mask) == -1) {
            printf("Error: sched_getaffinity(%d, sizeof(was_mask), &was_mask)\n", tid);
        }
        if (sched_setaffinity(0, sizeof(new_mask), &new_mask) == -1) {
            printf("Error: sched_setaffinity(%d, sizeof(new_mask), &new_mask)\n", tid);
        }
        printf("tid=%d new_mask=%08X was_mask=%08X\n", tid,
            *(unsigned int*)&new_mask, *(unsigned int*)&was_mask);
    }
    // Call Intel MKL FFT function
    return 0;
}
```

Compile the application with the Intel compiler using the following command:

```
icx test_application.c -openmp
```

where `test_application.c` is the filename for the application.

Build the application. Run it in two threads, for example, by using the environment variable to set the number of threads:

```
env OMP_NUM_THREADS=2 ./a.out
```

See the *Linux Programmer's Manual* (in man pages format) for particulars of the `sched_setaffinity` function used in the above example.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Managing Performance with Heterogeneous Cores

A hybrid architecture offers heterogeneous CPU cores. For example, the 12th Gen Intel® Core™ processor (Alder Lake) contains two types of cores: Performance-cores (P-cores) and Efficient-cores (E-cores).

Achieving the best performance on a hybrid architecture is harder because load balancing with heterogeneous cores is more complicated. Therefore, for hybrid architectures like Alder Lake, we recommend running threads on the P-cores only. This approach might not yield the best performance, but it is simple and predictable.

To specify P-cores with OpenMP, users can use the environment variable `KMP_HW_SUBSET`. For a detailed description of this environment variable, refer to the [Intel® C++ Compiler Classic Developer Guide and Reference](#). In the case of an Alder Lake processor with eight P-cores, either of the following two commands can be used for restricting threads to run only on the P-cores:

```
export KMP_HW_SUBSET=8c:intel_core
```

—or—

```
export KMP_HW_SUBSET=8c:effl
```

Note that for higher performance, Intel® Hyper-Threading Technology on P-cores must be disabled. You can achieve this either by changing the BIOS setting or by using `KMP_HW_SUBSET` to specify P-cores and one-thread-per-core with the following command:

```
export KMP_HW_SUBSET=8c:intel_core,1t
```

—or—

```
export KMP_HW_SUBSET=8c:effl,1t
```

If the user decides to adopt the more difficult approach of running on both P-cores and E-cores to maximize performance, there are a few aspects to take into consideration:

- Static versus dynamic load balancing
- Problem size
- Number of P-cores and E-cores
- OpenMP versus oneTBB

If there are similar or equal numbers of P-cores and E-cores and if both core types are used, using static load balancing for splitting the work items is likely to result in lower performance because E-cores will take longer to complete the work items assigned to them. For large GEMMs and {S,D}GETRF routines, oneMKL has implemented dynamic load balancing with OpenMP and will automatically select the best load balancing scheme. For most cases with small or regular problem sizes, static load balancing on P-cores is likely to give better performance. If the problem size is very large, the overhead of dynamic scheduling is small compared to overall computation time and dynamic load balancing will make more efficient use of P-cores and E-cores.

If the number of P-cores is much smaller than the number of E-cores, running on all cores may outperform limiting computations to only P-cores. Additional performance measurements would be needed to determine the best strategy.

As an alternative to OpenMP, users can also try [oneTBB](#), which might give better results for a given set of supported operations.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Improving Performance for Small Size Problems

The overhead of calling an Intel® oneAPI Math Kernel Library (oneMKL) function for small problem sizes can be significant when the function has a large number of parameters or internally checks parameter errors. To reduce the performance overhead for these small size problems, the Intel® oneAPI Math Kernel Library (oneMKL) *direct call* feature works in conjunction with the compiler to preprocess the calling parameters to supported Intel® oneAPI Math Kernel Library (oneMKL) functions and directly call or inline special optimized small-matrix kernels that bypass error checking. For a list of functions supporting direct call, see [Limitations of the Direct Call](#).

To activate the feature, do the following:

- Compile your C or Fortran code with the preprocessor macro depending on whether a threaded or sequential mode of Intel® oneAPI Math Kernel Library (oneMKL) is required by supplying the compiler option as explained below:

Intel® oneAPI Math Kernel Library (oneMKL) Mode	Macro	Compiler Option
Threaded	MKL_DIRECT_CALL	-DMKL_DIRECT_CALL
Sequential	MKL_DIRECT_CALL_SEQ	-DMKL_DIRECT_CALL_SEQ

- For Fortran applications:
 - Enable preprocessor by using the `-fpp` option for Intel® Fortran Compiler.
 - Include the Intel® oneAPI Math Kernel Library (oneMKL) Fortran include file `filemkl_direct_call.fi`.

Intel® oneAPI Math Kernel Library (oneMKL) skips error checking and intermediate function calls if the problem size is small enough (for example: a call to a function that supports direct call, such as `asdgemm`, with matrix ranks smaller than 50).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Using MKL_DIRECT_CALL in C Applications

The following examples of code and link lines show how to activate direct calls to Intel® oneAPI Math Kernel Library (oneMKL) kernels in C applications:

- Include the `mkl.h` header file:

```
#include "mkl.h"
int main(void) {

// Call Intel MKL DGEMM
```



```
return 0;
}
```

- For multi-threaded Intel® oneAPI Math Kernel Library (oneMKL), compile with `MKL_DIRECT_CALL` preprocessor macro:

```
icx -DMKL_DIRECT_CALL -std=c99 your_application.c -Wl,--start-group $(MKLROOT)/lib/intel64/
libmkl_intel_lp64.a
$(MKLROOT)/lib/intel64/libmkl_core.a $(MKLROOT)/lib/intel64/libmkl_intel_thread.a -Wl,--end-
group -lpthread -lm -openmp -I$(MKLROOT)/include
```

- To use Intel® oneAPI Math Kernel Library (oneMKL) in the sequential mode, compile with `MKL_DIRECT_CALL_SEQ` preprocessor macro:

```
icx -DMKL_DIRECT_CALL_SEQ -std=c99 your_application.c -Wl,--start-group $(MKLROOT)/lib/intel64/
libmkl_intel_lp64.a
$(MKLROOT)/lib/intel64/libmkl_core.a $(MKLROOT)/lib/intel64/libmkl_sequential.a -Wl,--end-group -
lpthread -lm -I$(MKLROOT)/include
```

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Using MKL_DIRECT_CALL in Fortran Applications

The following examples of code and link lines show how to activate direct calls to Intel® oneAPI Math Kernel Library (oneMKL) kernels in Fortran applications:

- Include `mkl_direct_call.fi`, to be preprocessed by the Fortran compiler preprocessor

```
#    include "mkl_direct_call.fi"
program    DGEMM_MAIN
....
*        Call Intel MKL DGEMM
....
    call sub1()
    stop 1
end

*        A subroutine that calls DGEMM
subroutine sub1
*        Call Intel MKL DGEMM

end
```

- For multi-threaded Intel® oneAPI Math Kernel Library (oneMKL), compile with `-fpp` option for Intel® Fortran compiler and with `MKL_DIRECT_CALL` preprocessor macro:

```
ifx -DMKL_DIRECT_CALL -fpp your_application.f -Wl,--start-group $(MKLROOT)/lib/intel64/
libmkl_intel_lp64.a
$(MKLROOT)/lib/intel64/libmkl_core.a $(MKLROOT)/lib/intel64/libmkl_intel_thread.a -Wl,--end-
group -lpthread -lm -openmp -I$(MKLROOT)/include
```

- To use Intel® oneAPI Math Kernel Library (oneMKL) in the sequential mode, compile with `-fpp` option for Intel® Fortran compiler and with `MKL_DIRECT_CALL_SEQ` preprocessor macro:

```
ifx -DMKL_DIRECT_CALL_SEQ -fpp your_application.f -Wl,--start-group $(MKLROOT)/lib/intel64/
libmkl_intel_lp64.a
$(MKLROOT)/lib/intel64/libmkl_core.a $(MKLROOT)/lib/intel64/libmkl_sequential.a -Wl,--end-group -
lpthread -lm -I$(MKLROOT)/include
```

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Limitations of the Direct Call

Directly calling the Intel® oneAPI Math Kernel Library (oneMKL) kernels has the following limitations:

- If the `MKL_DIRECT_CALL` or `MKL_DIRECT_CALL_SEQ` macro is used, Intel® oneAPI Math Kernel Library (oneMKL) may skip error checking.

Important

With a limited error checking, you are responsible for checking the correctness of function parameters to avoid unsafe and incorrect code execution.

- The feature is only available for the following functions:
 - BLAS: `?gemm`, `?gemm3m`, `?syrk`, `?trsm`, `?axpy`, and `?dot`
 - LAPACK: `?getrf`, `?getrs`, `?getri`, `?potrf`, and `?geqrf`. (available for C applications only)
- Intel® oneAPI Math Kernel Library (oneMKL) Verbose mode, Conditional Numerical Reproducibility, and BLAS95 interfaces are not supported.
- GNU* Fortran compilers are not supported.
- For C applications, you must enable mixing declarations and user code by providing the `-std=c99` option for Intel® compilers.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Other Tips and Techniques to Improve Performance**See Also**

[Managing Performance of the Cluster Fourier Transform Functions](#)

Coding Techniques

To improve performance, properly align arrays in your code.

Data Alignment and Leading Dimensions

To improve performance of your application that calls Intel® oneAPI Math Kernel Library (oneMKL), align your arrays on 64-byte boundaries and pad the leading dimensions, keeping in mind the following recommendations:

Recommendation 1 For best performance, the leading dimensions of matrices must be a multiple of 64 bytes (full cache line size). For single-precision real data, this means that the leading dimension must be a multiple of `(64 / sizeof(float))`, which is equal to 16 elements. Similarly, for double-precision real data or single-precision complex data, the leading dimension must be a multiple of 8 elements, while for double-precision complex data, the leading dimension should be a multiple of 4 elements.

Recommendation 2 For best performance, leading dimensions must not be a multiple of a large power of 2 (for example, 4096 bytes). Increasing the leading dimension slightly (for example, from 4096 bytes to 4096+64 bytes) can improve performance in some cases.

LAPACK Packed Routines

The routines with the names that contain the letters HP, OP, PP, SP, TP, UP in the matrix type and storage position (the second and third letters respectively) operate on the matrices in the packed format (see LAPACK "Routine Naming Conventions" sections in the Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference). Their functionality is strictly equivalent to the functionality of the unpacked routines with the names containing the letters HE, OR, PO, SY, TR, UN in the same positions, but the performance is significantly lower.

If the memory restriction is not too tight, use an unpacked routine for better performance. In this case, you need to allocate $N^2/2$ more memory than the memory required by a respective packed routine, where N is the problem size (the number of equations).

For example, to speed up solving a symmetric eigenproblem with an expert driver, use the unpacked routine:

```
call dsyevx(jobz, range, uplo, n, a, lda, vl, vu, il, iu, abstol, m, w, z, ldz, work, lwork,
iwork, ifail, info)
```

where a is the dimension lda -by- n , which is at least N^2 elements, instead of the packed routine:

```
call dspevx(jobz, range, uplo, n, ap, vl, vu, il, iu, abstol, m, w, z, ldz, work, iwork, ifail,
info)
```

where ap is the dimension $N*(N+1)/2$.

Product and Performance Information
Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex .
Notice revision #20201201

See Also

[Managing Performance of the Cluster Fourier Transform Functions](#)

Improving oneMKL Performance on Specific Processors

Product and Performance Information
Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex .
Notice revision #20201201

Dual-Core Intel® Xeon® Processor 5100 Series

To get the best performance with Intel® oneAPI Math Kernel Library (oneMKL) on Dual-Core Intel® Xeon® processor 5100 series systems, enable the Hardware DPL (streaming data) Prefetcher functionality of this processor. To configure this functionality, use the appropriate BIOS settings, as described in your BIOS documentation.

Operating on Denormals

The IEEE 754-2008 standard, "An IEEE Standard for Binary Floating-Point Arithmetic", defines *denormal* (or *subnormal*) numbers as non-zero numbers smaller than the smallest possible normalized numbers for a specific floating-point format. Floating-point operations on denormals are slower than on normalized operands because denormal operands and results are usually handled through a software assist mechanism rather than directly in hardware. This software processing causes Intel® oneAPI Math Kernel Library (oneMKL) functions that consume denormals to run slower than with normalized floating-point numbers.

You can mitigate this performance issue by setting the appropriate bit fields in the MXCSR floating-point control register to flush denormals to zero (FTZ) or to replace any denormals loaded from memory with zero (DAZ). Check your compiler documentation to determine whether it has options to control FTZ and DAZ. Note that these compiler options may slightly affect accuracy.

Using Memory Functions

Avoiding Memory Leaks in oneMKL

When running, Intel® oneAPI Math Kernel Library (oneMKL) may allocate and deallocate internal buffers to facilitate better performance. Memory leaks can occur if the Intel® oneAPI Math Kernel Library is unloaded before freeing the internal buffers.

You can free the internal buffers by calling the `mkl_free_buffers()` function or, for more granular control, the `mkl_thread_free_buffers()` function.

Alternatively, setting the `MKL_DISABLE_FAST_MM` environment variable to 1 or calling the `mkl_disable_fast_mm()` function disables the internal memory manager. Be aware that this change may negatively impact the performance of some oneMKL functions, especially for small problem sizes.

See Also

[Intel® Software Documentation Library](#)

Redefining Memory Functions

In C/C++ programs, you can replace Intel® oneAPI Math Kernel Library (oneMKL) memory functions that the library uses by default with your own functions. To do this, use the *memory renaming* feature.

Memory Renaming

In addition to the `memkind` library, Intel® oneAPI Math Kernel Library (oneMKL) memory management by default uses standard C run-time memory functions to allocate or free memory. These functions can be replaced using memory renaming.

Intel® oneAPI Math Kernel Library (oneMKL) accesses the memory functions by pointers `i_malloc`, `i_free`, `i_calloc`, and `i_realloc`, which are visible at the application level. You can programmatically redefine values of these pointers to the addresses of your application's memory management functions.

Redirecting the pointers is the only correct way to use your own set of memory management functions. If you call your own memory functions without redirecting the pointers, the memory will get managed by two independent memory management packages, which may cause unexpected memory issues.

How to Redefine Memory Functions

To redefine memory functions, use the following procedure:

1. Include the `i_malloc.h` header file in your code.
This header file contains all declarations required for replacing the memory allocation functions. The header file also describes how memory allocation can be replaced in those Intel libraries that support this feature.

2. Redefine values of pointers `i_malloc`, `i_free`, `i_calloc`, and `i_realloc` prior to the first call to Intel® oneAPI Math Kernel Library (oneMKL) functions, as shown in the following example:

```
#include "i_malloc.h"
. . .
i_malloc = my_malloc;
i_calloc = my_calloc;
i_realloc = my_realloc;
i_free   = my_free;
. . .
// Now you may call Intel MKL functions
```

Language-Specific Usage Options

The Intel® oneAPI Math Kernel Library (oneMKL) provides broad support for Fortran and C/C++ programming. However, not all functions support both Fortran and C interfaces. For example, some LAPACK functions have no C interface. You can call such functions from C using mixed-language programming.

If you want to use LAPACK or BLAS functions that support Fortran 77 in the Fortran 95 environment, additional effort may be initially required to build compiler-specific interface libraries and modules from the source code provided with Intel® oneAPI Math Kernel Library (oneMKL).

See Also

[Language Interfaces Support, by Function Domain](#)

Using Language-Specific Interfaces with Intel® oneAPI Math Kernel Library

This section discusses mixed-language programming and the use of language-specific interfaces with Intel® oneAPI Math Kernel Library (oneMKL).

See also "FFTW Interface to Intel® oneAPI Math Kernel Library" in the Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference (available in the [Intel® Software Documentation Library](#)) for details of the FFTW interfaces to Intel® oneAPI Math Kernel Library (oneMKL).

Interface Libraries and Modules

You can create the following interface libraries and modules using the respective makefiles located in the interfaces directory.

File name	Contains
Libraries, in Intel® oneAPI Math Kernel Library (oneMKL) architecture-specific directories	
<code>libmkl_blas95_lp64.a</code> ¹	Fortran 95 wrappers for BLAS (BLAS95) supporting LP64/ILP64 interface.
<code>libmkl_blas95_ilp64.a</code> ¹	
<code>libmkl_lapack95_lp64.a</code> ¹	Fortran 95 wrappers for LAPACK (LAPACK95) supporting LP64/ILP64 interface.
<code>libmkl_lapack95_ilp64.a</code> ¹	
<code>libfftw2xc_single_intel.a</code>	Single/double-precision interfaces for FFTW version 2.x (C interface for Intel compilers) to call Intel® oneAPI Math Kernel Library (oneMKL) FFT.
<code>libfftw2xc_double_intel.a</code>	

File name	Contains
libfftw2xc_single_gnu.a	Single/double-precision interfaces for FFTW version 2.x (C interface for GNU compilers) to call Intel® oneAPI Math Kernel Library (oneMKL) FFT.
libfftw2xc_double_gnu.a	
libfftw2xf_single_intel.a	Single/double-precision interfaces for FFTW version 2.x (Fortran interface for Intel compilers) to call Intel® oneAPI Math Kernel Library (oneMKL) FFT.
libfftw2xf_double_intel.a	
libfftw2xf_single_gnu.a	Single/double-precision interfaces for FFTW version 2.x (Fortran interface for GNU compiler) to call Intel® oneAPI Math Kernel Library (oneMKL) FFT.
libfftw2xf_double_gnu.a	
libfftw3xc_intel.a ²	Interfaces for FFTW version 3.x (C interface for Intel compiler) to call Intel® oneAPI Math Kernel Library (oneMKL) FFT.
libfftw3xc_gnu.a	Interfaces for FFTW version 3.x (C interface for GNU compilers) to call Intel® oneAPI Math Kernel Library (oneMKL) FFT.
libfftw3xf_intel.a ²	Interfaces for FFTW version 3.x (Fortran interface for Intel compilers) to call Intel® oneAPI Math Kernel Library (oneMKL) FFT.
libfftw3xf_gnu.a	Interfaces for FFTW version 3.x (Fortran interface for GNU compilers) to call Intel® oneAPI Math Kernel Library (oneMKL) FFT.
libfftw2x_cdft_SINGLE_lp64.a	Single/double-precision interfaces for MPI FFTW version 2.x (C interface) to call Intel® oneAPI Math Kernel Library (oneMKL) cluster FFT supporting the LP64/ILP64 interface.
libfftw2x_cdft_SINGLE_ilp64.a	
libfftw2x_cdft_DOUBLE_lp64.a	
libfftw2x_cdft_DOUBLE_ilp64.a	
libfftw3x_cdft_lp64.a	Interfaces for MPI FFTW version 3.x (C interface) to call Intel® oneAPI Math Kernel Library (oneMKL) cluster FFT supporting the LP64/ILP64 interface.
libfftw3x_cdft_ilp64.a	
Modules, in architecture- and interface-specific subdirectories of the Intel® oneAPI Math Kernel Library (oneMKL) include directory	
blas95.mod ¹	Fortran 95 interface module for BLAS (BLAS95).
lapack95.mod ¹	Fortran 95 interface module for LAPACK (LAPACK95).
f95_precision.mod ¹	Fortran 95 definition of precision parameters for BLAS95 and LAPACK95.
mk1_service.mod ¹	Fortran 95 interface module for Intel® oneAPI Math Kernel Library (oneMKL) support functions.

¹ Prebuilt for the Intel® Fortran compiler

² FFTW3 interfaces are integrated with Intel® oneAPI Math Kernel Library (oneMKL). Look in `<mk1_directory>/share/mkl/interfaces/fftw3x*/makefile` for options defining how to build and where to place the standalone library with the wrappers.

See Also

Fortran 95 Interfaces to LAPACK and BLAS

Fortran 95 Interfaces to LAPACK and BLAS

Fortran 95 interfaces are compiler-dependent. Intel® oneAPI Math Kernel Library (oneMKL) provides the interface libraries and modules precompiled with the Intel® Fortran compiler. Additionally, the Fortran 95 interfaces and wrappers are delivered as sources. (For more information, see [Compiler-dependent Functions and Fortran 90 Modules](#)). If you are using a different compiler, build the appropriate library and modules with your compiler and link the library as a user's library:

1. Go to the respective directory `<mkl_directory>/share/mkl/interfaces/blas95` or `<mkl_directory>/share/mkl/interfaces/lapack95`
2. Enter:

```
make libintel64 [interface=lp64|ilp64] INSTALL_DIR=<user_dir>
```

Important

The parameter `INSTALL_DIR` is required.

As a result, the required library is built and installed in the `<user_dir>/lib` directory, and the `.mod` files are built and installed in the `<user_dir>/include/mkl/intel64/{lp64|ilp64}` directory.

By default, the `ifx` compiler is assumed. You may change the compiler to `gfortran` with an additional parameter of `FC=gfortran`. For example:

```
make libintel64 INSTALL_DIR=<user_dir> FC=gfortran
```

To delete the library from the building directory, enter the following:

```
make clean INSTALL_DIR=<user_dir>
```

Caution

Even if you have administrative rights, avoid setting `INSTALL_DIR=../../..` or `INSTALL_DIR=<mkl_directory>` in a build or clean command above because these settings replace or delete the Intel® oneAPI Math Kernel Library (oneMKL) prebuilt Fortran 95 library and modules.

Compiler-dependent Functions and Fortran 90 Modules

Compiler-dependent functions occur whenever the compiler inserts into the object code function calls that are resolved in its run-time library (RTL). Linking of such code without the appropriate RTL will result in undefined symbols. Intel® oneAPI Math Kernel Library (oneMKL) has been designed to minimize RTL dependencies.

In cases where RTL dependencies might arise, the functions are delivered as source code and you need to compile the code with whatever compiler you are using for your application.

In particular, Fortran 90 modules result in the compiler-specific code generation requiring RTL support. Therefore, Intel® oneAPI Math Kernel Library (oneMKL) delivers these modules compiled with the Intel compiler, along with source code, to be used with different compilers.

Mixed-Language Programming with the Intel Math Kernel Library

[Appendix A Intel® oneAPI Math Kernel Library Language Interfaces Support](#) lists the programming languages supported for each Intel® oneAPI Math Kernel Library (oneMKL) function domain. However, you can call Intel® oneAPI Math Kernel Library (oneMKL) routines from different language environments.

See also these Knowledge Base articles:

- <https://community.intel.com/t5/Intel-oneAPI-Math-Kernel-Library/Using-Boost-Align-and-Boost-Numeric-uBlas-with-Intel-MKL/m-p/1008732> for how to perform BLAS matrix-matrix multiplication in C++ using Intel® oneAPI Math Kernel Library (oneMKL) substitution of Boost* uBLAS functions.
- <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-mkl-and-third-party-applications-how-to-use-them-together.html> for a list of articles describing how to use Intel® oneAPI Math Kernel Library (oneMKL) with third-party libraries and applications.

Calling LAPACK, BLAS, and CBLAS Routines from C/C++ Language Environments

Not all Intel® oneAPI Math Kernel Library (oneMKL) function domains support both C and Fortran environments. To use Intel® oneAPI Math Kernel Library (oneMKL) Fortran-style functions in C/C++ environments, you should observe certain conventions, which are discussed for LAPACK and BLAS in the subsections below.

Caution

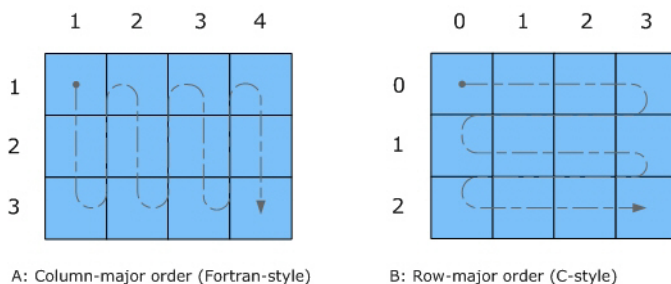
Avoid calling BLAS 95/LAPACK 95 from C/C++. Such calls require skills in manipulating the descriptor of a deferred-shape array, which is the Fortran 90 type. Moreover, BLAS95/LAPACK95 routines contain links to a Fortran RTL.

LAPACK and BLAS

Because LAPACK and BLAS routines are Fortran-style, when calling them from C-language programs, follow the Fortran-style calling conventions:

- Pass variables by *address*, not by *value*.
Function calls in [Example "Calling a Complex BLAS Level 1 Function from C++"](#) and [Example "Using CBLAS Interface Instead of Calling BLAS Directly from C"](#) illustrate this.
- Store your data in Fortran style, that is, column-major rather than row-major order.

With row-major order, adopted in C, the last array index changes most quickly and the first one changes most slowly when traversing the memory segment where the array is stored. With Fortran-style column-major order, the last index changes most slowly whereas the first index changes most quickly (as illustrated by the figure below for a two-dimensional array).



For example, if a two-dimensional matrix A of size $m \times n$ is stored densely in a one-dimensional array B, you can access a matrix element like this:

$A[i][j] = B[i*n+j]$ in C ($i=0, \dots, m-1, j=0, \dots, n-1$)

$A(i,j) = B((j-1)*m+i)$ in Fortran ($i=1, \dots, m, j=1, \dots, n$).

When calling LAPACK or BLAS routines from C, be aware that because the Fortran language is case-insensitive, the routine names can be upper-case, lower-case, or lower-case with the trailing underscore. For example, the following names are equivalent:

- LAPACK: `dgetrf`, `dgetrf_`, and `DGETRF`
- BLAS: `dgemm`, `dgemm_`, and `DGEMM`

See [Example "Calling a Complex BLAS Level 1 Function from C++"](#) on how to call BLAS routines from C.

See also the Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference for a description of the C interface to LAPACK functions.

CBLAS

Instead of calling BLAS routines from a C-language program, you can use the CBLAS interface.

CBLAS is a C-style interface to the BLAS routines. You can call CBLAS routines using regular C-style calls. Use the `mk1.h` header file with the CBLAS interface. `mk1.h` includes the `mk1_cblas.h` header file, which specifies enumerated values and prototypes of all the functions. It also determines whether the program is being compiled with a C++ compiler, and if it is, the included file will be correct for use with C++ compilation.

[Example "Using CBLAS Interface Instead of Calling BLAS Directly from C"](#) illustrates the use of the CBLAS interface.

C Interface to LAPACK

Instead of calling LAPACK routines from a C-language program, you can use the C interface to LAPACK provided by Intel® oneAPI Math Kernel Library (oneMKL).

The C interface to LAPACK is a C-style interface to the LAPACK routines. This interface supports matrices in row-major and column-major order, which you can define in the first function argument `matrix_order`. Use the `mk1.h` header file with the C interface to LAPACK. `mk1.h` includes the `mk1_lapacke.h` header file, which specifies constants and prototypes of all the functions. It also determines whether the program is being compiled with a C++ compiler, and if it is, the included file will be correct for use with C++ compilation. You can find examples of the C interface to LAPACK in the `examples/lapacke` subdirectory in the Intel® oneAPI Math Kernel Library (oneMKL) installation directory.

Using Complex Types in C/C++

C/C++ does not directly implement the Fortran types `COMPLEX(4)` and `COMPLEX(8)`. However, you can write equivalent structures. The type `COMPLEX(4)` consists of two 4-byte floating-point numbers. The first of them is the real-number component, and the second one is the imaginary-number component. The type `COMPLEX(8)` is similar to `COMPLEX(4)` except that it contains two 8-byte floating-point numbers.

Intel® oneAPI Math Kernel Library (oneMKL) provides complex types `MKL_Complex8` and `MKL_Complex16`, which are structures equivalent to the Fortran complex types `COMPLEX(4)` and `COMPLEX(8)`, respectively. The `MKL_Complex8` and `MKL_Complex16` types are defined in the `mk1_types.h` header file. You can use these types to define complex data. You can also redefine the types with your own types before including the `mk1_types.h` header file. The only requirement is that the types must be compatible with the Fortran complex layout, that is, the complex type must be a pair of real numbers for the values of real and imaginary parts.

For example, you can use the following definitions in your C++ code:

```
#define MKL_Complex8 std::complex<float>
```

and

```
#define MKL_Complex16 std::complex<double>
```

See [Example "Calling a Complex BLAS Level 1 Function from C++"](#) for details.

You can also define these types in the command line:

```
-DMKL_Complex8="std::complex<float>"
-DMKL_Complex16="std::complex<double>"
```

See Also

[Intel® Software Documentation Library](#) for the Intel® Fortran Compiler documentation
for the Intel® Fortran Compiler documentation

Calling BLAS Functions That Return the Complex Values in C/C++ Code

Complex values that functions return are handled differently in C and Fortran. Because BLAS is Fortran-style, you need to be careful when handling a call from C to a BLAS function that returns complex values. However, in addition to normal function calls, Fortran enables calling functions as though they were subroutines, which provides a mechanism for returning the complex value correctly when the function is called from a C program. When a Fortran function is called as a subroutine, the return value is the first parameter in the calling sequence. You can use this feature to call a BLAS function from C.

The following example shows how a call to a Fortran function as a subroutine converts to a call from C and the hidden parameter result gets exposed:

Normal Fortran function call: `result = cdotc( n, x, 1, y, 1 )`

A call to the function as a subroutine: `call cdotc( result, n, x, 1, y, 1)`

A call to the function from C: `cdotc( &result, &n, x, &one, y, &one )`

NOTE

Intel® oneAPI Math Kernel Library (oneMKL) has upper-case, lower-case, and lower-case with the trailing underscore entry points in the Fortran-style (case-insensitive) BLAS. So, all these names are equivalent and acceptable: `cdotc`, `cdotc_`, and `CDOTC`.

The above example shows one of the ways to call several level 1 BLAS functions that return complex values from your C and C++ applications. An easier way is to use the CBLAS interface. For instance, you can call the same function using the CBLAS interface as follows:

`cblas_cdotc( n, x, 1, y, 1, &result )`

NOTE

The complex value comes last on the argument list in this case.

The following examples show use of the Fortran-style BLAS interface from C and C++, as well as the CBLAS (C language) interface:

- [Example "Calling a Complex BLAS Level 1 Function from C"](#)
- [Example "Calling a Complex BLAS Level 1 Function from C++"](#)
- [Example "Using CBLAS Interface Instead of Calling BLAS Directly from C"](#)

Example "Calling a Complex BLAS Level 1 Function from C"

The example below illustrates a call from a C program to the complex BLAS Level 1 function `zdotc()`. This function computes the dot product of two double-precision complex vectors.

In this example, the complex dot product is returned in the structure `c`.

```
#include "mkl.h"
#define N 5
int main()
{
    int n = N, inca = 1, incb = 1, i;
    MKL_Complex16 a[N], b[N], c;
    for( i = 0; i < n; i++ )
    {
        a[i].real = (double)i; a[i].imag = (double)i * 2.0;
        b[i].real = (double)(n - i); b[i].imag = (double)i * 2.0;
    }
    zdotc( &c, &n, a, &inca, b, &incb );
```

```
printf( "The complex dot product is: ( %.2f, %.2f)\n", c.real, c.imag );
return 0;
}
```

In this example, the complex dot product for large data size is returned in the structure `c`.

```
#include "mkl.h"
#define N 5
int main()
{
    MKL_INT64 n = N, inca = 1, incb = 1, i;
    MKL_Complex16 a[N], b[N], c;
    for( i = 0; i < n; i++ )
    {
        a[i].real = (double)i; a[i].imag = (double)i * 2.0;
        b[i].real = (double)(n - i); b[i].imag = (double)i * 2.0;
    }
    zdotc_64( &c, &n, a, &inca, b, &incb );
    printf( "The complex dot product is: ( %.2f, %.2f)\n", c.real, c.imag );
    return 0;
}
```

Example "Calling a Complex BLAS Level 1 Function from C++"

Below is the C++ implementation:

```
#include <complex>
#include <iostream>
#define MKL_Complex16 std::complex<double>
#include "mkl.h"

#define N 5

int main()
{
    int n, inca = 1, incb = 1, i;
    std::complex<double> a[N], b[N], c;
    n = N;

    for( i = 0; i < n; i++ )
    {
        a[i] = std::complex<double>(i,i*2.0);
        b[i] = std::complex<double>(n-i,i*2.0);
    }
    zdotc(&c, &n, a, &inca, b, &incb );
    std::cout << "The complex dot product is: " << c << std::endl;
    return 0;
}
```

Example "Using CBLAS Interface Instead of Calling BLAS Directly from C"

This example uses CBLAS:

```
#include <stdio.h>
#include "mkl.h"
typedef struct{ double re; double im; } complex16;
```

```
#define N 5
int main()
{
    int n, inca = 1, incb = 1, i;
    complex16 a[N], b[N], c;
    n = N;
    for( i = 0; i < n; i++ )
    {
        a[i].re = (double)i; a[i].im = (double)i * 2.0;
        b[i].re = (double)(n - i); b[i].im = (double)i * 2.0;
    }
    cblas_zdotc_sub(n, a, inca, b, incb, &c );
    printf( "The complex dot product is: ( %6.2f, %6.2f)\n", c.re, c.im );
    return 0;
}
```

Obtaining Numerically Reproducible Results

Intel® oneAPI Math Kernel Library (oneMKL) offers functions and environment variables that help you obtain Conditional Numerical Reproducibility (CNR) of floating-point results when calling the library functions from your application. These new controls enable Intel® oneAPI Math Kernel Library (oneMKL) to run in a special mode, when functions return bitwise reproducible floating-point results from run to run under the following conditions:

- Calls to Intel® oneAPI Math Kernel Library (oneMKL) occur in a single executable
- The number of computational threads used by the library does not change in the run

For a limited set of routines, you can eliminate the second condition by using Intel® oneAPI Math Kernel Library (oneMKL) in [strict CNR mode](#).

It is well known that for general single and double precision IEEE floating-point numbers, the associative property does not always hold, meaning $(a+b)+c$ may not equal $a+(b+c)$. Let's consider a specific example. In infinite precision arithmetic $2^{-63} + 1 + -1 = 2^{-63}$. If this same computation is done on a computer using double precision floating-point numbers, a rounding error is introduced, and the order of operations becomes important:

$$(2^{-63} + 1) + (-1) \simeq 1 + (-1) = 0$$

versus

$$2^{-63} + (1 + (-1)) \simeq 2^{-63} + 0 = 2^{-63}$$

This inconsistency in results due to order of operations is precisely what the new functionality addresses.

The application related factors that affect the order of floating-point operations within a single executable program include selection of a code path based on run-time processor dispatching, alignment of data arrays, variation in number of threads, threaded algorithms and internal floating-point control settings. You can control most of these factors by controlling the number of threads and floating-point settings and by taking steps to align memory when it is allocated. However, run-time dispatching and certain threaded algorithms do not allow users to make changes that can ensure the same order of operations from run to run.

Intel® oneAPI Math Kernel Library (oneMKL) does run-time processor dispatching in order to identify the appropriate internal code paths to traverse for the Intel® oneAPI Math Kernel Library (oneMKL) functions called by the application. The code paths chosen may differ across a wide range of Intel processors and Intel architecture compatible processors and may provide differing levels of performance. For example, an Intel® oneAPI Math Kernel Library (oneMKL) function running on an Intel® Pentium® 4 processor may run one code path, while on the latest Intel® Xeon® processor it will run another code path. This happens because each unique code path has been optimized to match the features available on the underlying processor. One key way that the new features of a processor are exposed to the programmer is through the instruction set architecture (ISA). Because of this, code branches in Intel® oneAPI Math Kernel Library (oneMKL) are designated by the latest ISA they use for optimizations: from the Intel® Streaming SIMD Extensions 2 (Intel®

SSE2) to the Intel® Advanced Vector Extensions2 (Intel® AVX2). The feature-based approach introduces a challenge: if any of the internal floating-point operations are done in a different order or are re-associated, the computed results may differ.

Dispatching optimized code paths based on the capabilities of the processor on which the code is running is central to the optimization approach used by Intel® oneAPI Math Kernel Library (oneMKL). So it is natural that consistent results require some performance trade-offs. If limited to a particular code path, performance of Intel® oneAPI Math Kernel Library (oneMKL) can in some circumstances degrade by more than a half. To understand this, note that matrix-multiply performance nearly doubled with the introduction of new processors supporting Intel AVX2 instructions. Even if the code branch is not restricted, performance can degrade by 10-20% because the new functionality restricts algorithms to maintain the order of operations.

Numerically Reproducible Results for GPU Computations

oneMKL 2024.1 introduces CNR support for certain GPU computations: level-3 BLAS routines (for example, `gemm` and `trsm`) and batched versions of these routines. When any CNR code branch is enabled, GPU CNR support is enabled automatically, ensuring run-to-run bitwise reproducible results for these routines on Intel® GPUs. The specific code branch chosen (AVX2, AVX-512, and so forth) is ignored for GPU execution.

When GPU CNR mode is enabled, oneMKL guarantees that running the same computation multiple times on the same GPU will result in deterministic, bitwise-identical results. Two GPUs are considered to be the same if they have the same product name (for example, Intel® Arc™ A770). Note that results may differ across devices (between CPU and GPU, or between different kinds of GPUs).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at [www.Intel.com/PerformanceIndex](https://www.intel.com/PerformanceIndex).

Notice revision #20201201

Getting Started with Conditional Numerical Reproducibility

Intel® oneAPI Math Kernel Library (oneMKL) offers functions and environment variables to help you get reproducible results. You can configure Intel® oneAPI Math Kernel Library (oneMKL) using functions or environment variables, but the functions provide more flexibility.

The following specific examples introduce you to the conditional numerical reproducibility.

While these examples recommend aligning input and output data, you can supply unaligned data to Intel® oneAPI Math Kernel Library (oneMKL) functions running in the CNR mode, but refer to [Reproducibility Conditions](#) for details related to data alignment.

Intel CPUs Supporting Intel AVX2

To ensure Intel® oneAPI Math Kernel Library (oneMKL) calls return the same results on every Intel CPU supporting Intel AVX2 instructions:

1. Make sure that your application uses a fixed number of threads
2. (Recommended) Properly align input and output arrays in Intel® oneAPI Math Kernel Library (oneMKL) function calls
3. Do either of the following:
 - Call


```
mkl_cbwr_set(MKL_CBWR_AVX2)
```
 - Set the environment variable:


```
export MKL_CBWR = AVX2
```

NOTE

On non-Intel CPUs and on Intel CPUs that do not support Intel AVX2, this environment setting may cause results to differ because the `AUTO` branch is used instead, while the above function call returns an error and does not enable the CNR mode.

Intel CPUs Supporting Intel SSE2

To ensure Intel® oneAPI Math Kernel Library (oneMKL) calls return the same results on every Intel CPU supporting Intel SSE2 instructions:

1. Make sure that your application uses a fixed number of threads
2. (Recommended) Properly align input and output arrays in Intel® oneAPI Math Kernel Library (oneMKL) function calls
3. Do either of the following:

- Call

```
mkl_cbwr_set(MKL_CBWR_SSE2)
```

- Set the environment variable:

```
export MKL_CBWR = SSE2
```

NOTE

On non-Intel CPUs, this environment setting may cause results to differ because the `AUTO` branch is used instead, while the above function call returns an error and does not enable the CNR mode.

Intel or Intel-Compatible CPUs Supporting Intel SSE2

On non-Intel CPUs, only the `MKL_CBWR_AUTO` and `MKL_CBWR_COMPATIBLE` options are supported for function calls and only `AUTO` and `COMPATIBLE` options for environment settings.

To ensure Intel® oneAPI Math Kernel Library (oneMKL) calls return the same results on all Intel or Intel compatible CPUs supporting Intel SSE2 instructions:

1. Make sure that your application uses a fixed number of threads
2. (Recommended) Properly align input and output arrays in Intel® oneAPI Math Kernel Library (oneMKL) function calls
3. Do either of the following:

- Call

```
mkl_cbwr_set(MKL_CBWR_COMPATIBLE)
```

- Set the environment variable:

```
export MKL_CBWR = COMPATIBLE
```

NOTE

The special `MKL_CBWR_COMPATIBLE/COMPATIBLE` option is provided because Intel and Intel compatible CPUs have a few instructions, such as approximation instructions `rcpps/rsqrtps`, that may return different results. This option ensures that Intel® oneAPI Math Kernel Library (oneMKL) does not use these instructions and forces a single Intel SSE2 only code path to be executed.

Intel GPUs

To ensure that selected oneMKL calls return the same results on every Intel GPU, do either of the following:

- Call `mkl_cbwr_set(MKL_CBWR_AUTO)`.
- Set the following environment variable: `export MKL_CBWR = AUTO`.

Next Steps

See [Specifying the Code Branches](#)

for details of specifying the branch using environment variables.

See the following sections in the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference*:

Support Functions for Conditional Numerical Reproducibility

for how to configure the CNR mode of Intel® oneAPI Math Kernel Library (oneMKL) using functions.

Intel® oneAPI Math Kernel Library (oneMKL) PARDISO - Parallel Direct Sparse Solver Interface

for how to configure the CNR mode for PARDISO.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Code Examples](#)

Specifying Code Branches

Intel® oneAPI Math Kernel Library (oneMKL) provides a conditional numerical reproducibility (CNR) functionality that enables you to obtain reproducible results from oneMKL routines. When enabling CNR, you choose a specific code branch of Intel® oneAPI Math Kernel Library (oneMKL) that corresponds to the instruction set architecture (ISA) that you target. You can specify the code branch and other CNR options using the `MKL_CBWR` environment variable.

- `MKL_CBWR="<branch>[,STRICT]"` or
- `MKL_CBWR="BRANCH=<branch>[,STRICT]"`

Use the `STRICT` flag to enable strict CNR mode. For more information, see [Reproducibility Conditions](#).

The `<branch>` placeholder specifies the CNR branch with one of the following values:

Value	Description
AUTO	CNR mode uses the standard ISA-based dispatching model while ensuring fixed cache sizes, deterministic reductions, and static scheduling
COMPATIBLE	Intel® Streaming SIMD Extensions 2 (Intel® SSE2) without rcpps/rsqrtps instructions
SSE2	Intel SSE2
SSE3	DEPRECATED. Intel® Streaming SIMD Extensions 3 (Intel® SSE3). This setting is kept for backward compatibility and is equivalent to <code>SSE2</code> .
SSSE3	DEPRECATED. Supplemental Streaming SIMD Extensions 3 (SSSE3)
SSE4_2	Intel® Streaming SIMD Extensions 4.2 (Intel® SSE4.2)

Value	Description
AVX	DEPRECATED. Intel® Advanced Vector Extensions (Intel® AVX)
AVX2	Intel® Advanced Vector Extensions 2 (Intel® AVX2)
AVX512	Intel AVX-512 on Intel® Xeon® processors
AVX512_E1	Intel® Advanced Vector Extensions 512 (Intel® AVX-512) with support for Vector Neural Network Instructions
AVX512_MIC	DEPRECATED. Intel® Advanced Vector Extensions 512 (Intel® AVX-512) on Intel® Xeon Phi™ processors. This setting is kept for backward compatibility and is equivalent to AVX2.
AVX512_MIC_E1	DEPRECATED. Intel® Advanced Vector Extensions 512 (Intel® AVX-512) with support for Vector Neural Network Instructions on Intel® Xeon Phi™ processors. This setting is kept for backward compatibility and is equivalent to AVX2.

When specifying the CNR branch, be aware of the following:

- Reproducible results are provided under [Reproducibility Conditions](#).
- Settings other than `AUTO` or `COMPATIBLE` are available only for Intel processors.
- To get the CNR branch optimized for the processor where your program is currently running, choose the value of `AUTO` or call the `mkl_cbwr_get_auto_branch` function.
- Strict CNR mode is supported only for AVX2, AVX512, AVX512_E1, AVX512_MIC, and AVX512_MIC_E1 branches. You can also use strict CNR mode with the `AUTO` branch when running on Intel processors that support one of these instruction set architectures (ISAs).

Setting the `MKL_CBWR` environment variable or a call to an equivalent `mkl_cbwr_set` function fixes the code branch and sets the reproducibility mode.

NOTE

- If the value of the branch is incorrect or your processor or operating system does not support the specified ISA, CNR ignores this value and uses the `AUTO` branch without providing any warning messages.
- Calls to functions that define the behavior of CNR must precede any of the math library functions that they control.
- Settings specified by the functions take precedence over the settings specified by the environment variable.

See the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference* for how to specify the branches using functions.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Getting Started with Conditional Numerical Reproducibility](#)

Reproducibility Conditions

To get reproducible results from run to run, ensure that the number of threads is fixed and constant. Specifically:

- If you are running your program with OpenMP* parallelization on different processors, explicitly specify the number of threads.
- To ensure that your application has deterministic behavior with OpenMP* parallelization and does not adjust the number of threads dynamically at run time, set `MKL_DYNAMIC` and `OMP_DYNAMIC` to `FALSE`. This is especially needed if you are running your program on different systems.
- If you are running your program with the Intel® Threading Building Blocks parallelization, numerical reproducibility is not guaranteed.

Strict CNR Mode

In strict CNR mode, oneAPI Math Kernel Library provides bitwise reproducible results for a limited set of functions and code branches even when the number of threads changes. These routines and branches support strict CNR mode (64-bit libraries only):

- `?gemm`, `?symm`, `?hemm`, `?trsm`, and their CBLAS equivalents (`cblas_?gemm`, `cblas_?symm`, `cblas_?hemm`, and `cblas_?trsm`).
- Intel® Advanced Vector Extensions 2 (Intel® AVX2) or Intel® Advanced Vector Extensions 512 (Intel® AVX-512).

When using other routines or CNR branches, oneAPI Math Kernel Library operates in standard (non-strict) CNR mode, subject to the restrictions described above. Enabling strict CNR mode can reduce performance.

Reproducibility Conditions for Intel® GPUs

When CNR mode is enabled (any code branch other than OFF), oneMKL provides bitwise-reproducible results for a limited set of functions on Intel® GPUs:

- BLAS level-3 routines (`gemm`, `symm`, `hemm`, `syrk`, `herk`, `syr2k`, `her2k`, `trmm`, `trsm`), all precisions
- BLAS level-3 extensions (batched versions of the above, and `gemmt`), all precisions

Both the OpenMP* offload APIs and the DPC++ APIs support GPU CNR mode.

Reproducibility is guaranteed to run when the code is running on the same GPU, or between two GPUs with identical product names (for example, Intel® Arc™ A770).

NOTE

- As usual, you should align your data, even in CNR mode, to obtain the best possible performance. While CNR mode also fully supports unaligned input and output data, the use of it might reduce the performance of some oneAPI Math Kernel Library functions on earlier Intel processors. Refer to coding techniques that improve performance for more details.
- Conditional Numerical Reproducibility does not ensure that bitwise-identical NaN values are generated when the input data contains NaN values.
- If dynamic memory allocation fails on one run but succeeds on another run, you may fail to get reproducible results between these two runs.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[MKL_DYNAMIC](#)

[Coding Techniques](#)

Setting the Environment Variable for Conditional Numerical Reproducibility

The following examples illustrate the use of the `MKL_CBWR` environment variable. The first command in each list sets Intel® oneAPI Math Kernel Library (oneMKL) to run in the CNR mode based on the default dispatching for your platform. The other two commands in each list are equivalent and set the CNR branch to Intel AVX2.

For the bash shell:

- `export MKL_CBWR="AUTO"`
- `export MKL_CBWR="AVX2"`
- `export MKL_CBWR="BRANCH=AVX2"`

For the C shell (csh or tcsh):

- `setenv MKL_CBWR "AUTO"`
- `setenv MKL_CBWR "AVX2"`
- `setenv MKL_CBWR "BRANCH=AVX2"`

See Also

[Specifying Code Branches](#)

Code Examples

The following simple programs show how to obtain reproducible results from run to run of Intel® oneAPI Math Kernel Library (oneMKL) functions. See the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference* for more examples.

C Example of CNR

```
#include <mkl.h>
int main(void) {
    int my_cbwr_branch;
    /* Align all input/output data on 64-byte boundaries */
    /* for best performance of Intel® oneAPI Math Kernel Library (oneMKL) */
    void *darray;
    int darray_size=1000;
    /* Set alignment value in bytes */
    int alignment=64;
    /* Allocate aligned array */
    darray = mkl_malloc (sizeof(double)*darray_size, alignment);
    /* Find the available MKL_CBWR_BRANCH automatically */
    my_cbwr_branch = mkl_cbwr_get_auto_branch();
    /* User code without oneMKL calls */
    /* Piece of the code where CNR of oneMKL is needed */
    /* The performance of oneMKL functions might be reduced for CNR mode */
    /* If the "IF" statement below is commented out, Intel® oneAPI Math Kernel Library (oneMKL) will
    run in a regular mode, */
    /* and data alignment will allow you to get best performance */
    if (mkl_cbwr_set(my_cbwr_branch)) {
        printf("Error in setting MKL_CBWR_BRANCH! Aborting...\n");
        return;
    }
    /* CNR calls to oneMKL + any other code */
    /* Free the allocated aligned array */
    mkl_free(darray);
}
```

Fortran Example of CNR

```

PROGRAM MAIN
    INCLUDE 'mkl.fi'
    INTEGER*4 MY_CBWR_BRANCH
! Align all input/output data on 64-byte boundaries
! for best performance of Intel® oneAPI Math Kernel Library (oneMKL)
! Declare oneMKL memory allocation routine
    DOUBLE PRECISION DARRAY
    POINTER (P_DARRAY,DARRAY(1))
    INTEGER DARRAY_SIZE
    PARAMETER (DARRAY_SIZE=1000)
! Set alignment value in bytes
    INTEGER ALIGNMENT
    PARAMETER (ALIGNMENT=64)
! Allocate aligned array
    INTEGER*8 ALLOC_SIZE
    ALLOC_SIZE = 8*DARRAY_SIZE
    P_DARRAY = MKL_MALLOC (ALLOC_SIZE, ALIGNMENT);
! Find the available MKL_CBWR_BRANCH automatically
    MY_CBWR_BRANCH = MKL_CBWR_GET_AUTO_BRANCH()
! User code without oneMKL calls
! Piece of the code where CNR of oneMKL is needed
! The performance of oneMKL functions may be reduced for CNR mode
! If the "IF" statement below is commented out,
! Intel® oneAPI Math Kernel Library (oneMKL) will run in a
! regular mode, and data alignment will enable you to get the best performance
    IF (MKL_CBWR_SET (MY_CBWR_BRANCH) .NE. MKL_CBWR_SUCCESS) THEN
        PRINT *, 'Error in setting MKL_CBWR_BRANCH! Aborting...'
        STOP 0
    ENDIF
! CNR calls to oneMKL + any other code
! Free the allocated aligned array
    CALL MKL_FREE(P_DARRAY)
END

```

Use of CNR with Unaligned Data in C

```

#include <mkl.h>
int main(void) {
    int my_cbwr_branch;
    /* If it is not possible to align all input/output data on 64-byte boundaries */
    /* to achieve performance, use unaligned IO data with possible performance */
    /* penalty */
    /* Using unaligned IO data */
    double *darray;
    int darray_size=1000;
    /* Allocate array, malloc aligns data on 8/16-byte boundary only */
    darray = (double *)malloc (sizeof(double)*darray_size);
    /* Find the available MKL_CBWR_BRANCH automatically */
    my_cbwr_branch = mkl_cbwr_get_auto_branch();
    /* User code without oneMKL calls */
    /* Piece of the code where CNR of oneMKL is needed */
    /* The performance of oneMKL functions might be reduced for CNR mode */
    /* If the "IF" statement below is commented out, oneMKL will run in a regular mode, */
    /* and you will NOT get best performance without data alignment */
    if (mkl_cbwr_set(my_cbwr_branch)) {
        printf("Error in setting MKL_CBWR_BRANCH! Aborting...\n");
        return;
    }
}

```

```

}
/* CNR calls to oneMKL + any other code */
/* Free the allocated array */
free(darray);

```

Use of CNR with Unaligned Data in Fortran

```

PROGRAM MAIN
  INCLUDE 'mkl.fi'
  INTEGER*4 MY_CBWR_BRANCH
! If it is not possible to align all input/output data on 64-byte boundaries
! to achieve performance, use unaligned IO data with possible performance
! penalty
  DOUBLE PRECISION, DIMENSION(:), ALLOCATABLE :: DARRAY
  INTEGER DARRAY_SIZE, STATUS
  PARAMETER (DARRAY_SIZE=1000)
! Allocate array with undefined alignment
  ALLOCATE(DARRAY(DARRAY_SIZE));
! Find the available MKL_CBWR_BRANCH automatically
  MY_CBWR_BRANCH = MKL_CBWR_GET_AUTO_BRANCH()
! User code without oneMKL calls
! Piece of the code where CNR of oneMKL is needed
! The performance of oneMKL functions might be reduced for CNR mode
! If the "IF" statement below is commented out, oneMKL will run in a regular mode,
! and you will NOT get best performance without data alignment
  IF (MKL_CBWR_SET(MY_CBWR_BRANCH) .NE. MKL_CBWR_SUCCESS) THEN
    PRINT *, 'Error in setting MKL_CBWR_BRANCH! Aborting...'
    RETURN
  ENDIF
! CNR calls to oneMKL + any other code
! Free the allocated array
  DEALLOCATE(DARRAY)
END

```

Coding Tips

This section provides coding tips for managing data alignment and version-specific compilation.

See Also

[Mixed-language Programming with the Intel® oneAPI Math Kernel Library](#) Tips on language-specific programming

[Managing Performance and Memory](#) Coding tips related to performance improvement and use of memory functions

[Obtaining Numerically Reproducible Results](#) Tips for obtaining numerically reproducible results of computations

Example of Data Alignment

Needs for best performance with Intel® oneAPI Math Kernel Library (oneMKL) or for reproducible results from run to run of Intel® oneAPI Math Kernel Library (oneMKL) functions require alignment of data arrays. The following example shows how to align an array on 64-byte boundaries. To do this, use `mkl_malloc()` in place of system provided memory allocators, as shown in the code example below.

Aligning Addresses on 64-byte Boundaries

```
// ***** C language *****
...
#include <stdlib.h>
#include <mkl.h>
...
void *darray;
int workspace;
// Set value of alignment
int alignment=64;
...
// Allocate aligned workspace
darray = mkl_malloc( sizeof(double)*workspace, alignment );
...
// call the program using oneMKL
mkl_app( darray );
...
// Free workspace
mkl_free( darray );
```

```
! ***** Fortran language *****
...
! Set value of alignment
integer    alignment
parameter (alignment=64)
...
! Declare oneMKL routines
integer*8 mkl_malloc
external mkl_malloc, mkl_free, mkl_app
...
double precision darray
pointer (p_wrk,darray(1))
integer workspace
...
! Allocate aligned workspace
p_wrk = mkl_malloc( %val(8*workspace), %val(alignment) )
...
! call the program using oneMKL
call mkl_app( darray )
...
! Free workspace
call mkl_free(p_wrk)
```

Using Predefined Preprocessor Symbols for Intel® oneMKL Version-Dependent Compilation

Preprocessor symbols (macros) substitute values in a program before it is compiled. The substitution is performed in the preprocessing phase.

The following preprocessor symbols are available:

Predefined Preprocessor Symbol	Description
<code>__INTEL_MKL__</code>	Intel® oneAPI Math Kernel Library (oneMKL) major version

Predefined Preprocessor Symbol	Description
<code>__INTEL_MKL_MINOR__</code>	The value of this macro is 0 for backward compatibility.
<code>__INTEL_MKL_UPDATE__</code>	Intel® oneAPI Math Kernel Library (oneMKL) update version
<code>__INTEL_MKL_PATCH__</code>	Intel® oneAPI Math Kernel Library (oneMKL) patch version
<code>INTEL_MKL_VERSION</code>	Intel® oneAPI Math Kernel Library (oneMKL) full version in the following format: $INTEL_MKL_VERSION = (_INTEL_MKL_ _ * 100 + _INTEL_MKL_UPDATE_) * 100 + _INTEL_MKL_PATCH_$

These symbols enable conditional compilation of code that uses new features introduced in a particular version of the library.

To perform conditional compilation:

1. Depending on your compiler, include in your code the file where the macros are defined:

C/C++ compiler:	<code>mkl_version.h</code> , or <code>mkl.h</code> , which includes <code>mkl_version.h</code>
Intel®Fortran compiler:	<code>mkl.fi</code>
Any Fortran compiler with enabled preprocessing:	<code>mkl_version.h</code> Read the documentation for your compiler for the option that enables preprocessing.

2. [Optionally] Use the following preprocessor directives to check whether the macro is defined:

- `#ifdef`, `#endif` for C/C++
- `!DEC$IF DEFINED`, `!DEC$ENDIF` for Fortran

3. Use preprocessor directives for conditional inclusion of code:

- `#if`, `#endif` for C/C++
- `!DEC$IF`, `!DEC$ENDIF` for Fortran

Example

This example shows how to compile a code segment conditionally for a specific version of Intel® oneAPI Math Kernel Library (oneMKL). In this case, the version is 2025.0:

Intel®Fortran Compiler:

```
include "mkl.fi"
!DEC$IF DEFINED INTEL_MKL_VERSION
!DEC$IF INTEL_MKL_VERSION .EQ. 20250000
*   Code to be conditionally compiled
!DEC$ENDIF
!DEC$ENDIF
```

C/C++ Compiler. Fortran Compiler with Enabled Preprocessing:

```
#include "mkl.h"
#ifdef INTEL_MKL_VERSION
#if INTEL_MKL_VERSION == 20250000
...   Code to be conditionally compiled
#endif
#endif
```

Querying oneMATH Specification Version Compliance

Each Intel® oneAPI Math Kernel Library (oneMKL) domain defines a preprocessor macro to represent the version of the specification that the implementation is compliant with, in accordance with the [oneMATH specification](#).

The macros for each domain are listed as follows:

```
ONEMATH_BLAS_SPEC_VERSION
ONEMATH_LAPACK_SPEC_VERSION
ONEMATH_SPBLAS_SPEC_VERSION
ONEMATH_DFT_SPEC_VERSION
ONEMATH_RNG_SPEC_VERSION
ONEMATH_STATS_SPEC_VERSION
ONEMATH_VM_SPEC_VERSION
```

To support modern C++ usage, we also add `constexpr` of type `SpecVersion` for each domain in `oneapi/mkl/<domain>/spec.hpp`:

```
oneapi::mkl::blas::spec_version
oneapi::mkl::lapack::spec_version
oneapi::mkl::sparse::spec_version
oneapi::mkl::dft::spec_version
oneapi::mkl::rng::spec_version
oneapi::mkl::stats::spec_version
oneapi::mkl::vm::spec_version
```

To simplify version comparison, we add version enums in `include/oneapi/mkl/spec.hpp`:

```
enum class SpecVersion{
    not_compliant = 1,
    version_1_1 = 101,
    version_1_2 = 102,
    version_1_3 = 103,
    version_1_4 = 104,
    not_released
};
```

C Macro Usage Example

```
#include <oneapi/mkl/spec.hpp> // or mkl.hpp

#define MY_THRESHOLD_VERSION 102
#ifdef ONEMATH_BLAS_SPEC_VERSION
    #if ONEMATH_BLAS_SPEC_VERSION > MY_THRESHOLD_VERSION
        #define my_call call_optimization_A
    #elif ONEMATH_BLAS_SPEC_VERSION > 001
        #define my_call call_optimization_B
    #else
        #define my_call call_reference
    #endif
#else
    #define my_call call_reference
#endif

int main(){
    my_call();
    return 0;
}
```

C++ Usage Example

```
#include <oneapi/mkl/spec.hpp> // or mkl.hpp

using namespace oneapi::mkl;

int main(){
    SpecVersion my_threshold_version = SpecVersion::version_1_2;

    if (oneapi::mkl::blas::spec_version > my_threshold_version)
        call_optimization_A(); // App's optimization for new spec
    else if (oneapi::mkl::blas::spec_version > SpecVersion::not_compliant)
        call_optimization_B(); // App's optimization for old spec
    else
        call_reference();      // App's reference implementation

    return 0;
}
```

Compliance Policy

1. oneMKL domains can be compliant with any version of the oneMATH specification.
2. oneMKL can increment the compliance version with new releases of oneMKL.
3. A new release of the oneMATH specification does not require updating the oneMKL compliance version.

Values

1. 1 if the domain does not comply with any released version of the oneMATH specification.
2. <xxyy> if the domain is released with support for one of the available oneMATH specification versions, where *xx* is the major version and *yy* is the minor version of the oneMath Specification it is compliant with. For example, the integer value 103 represents oneMATH Specification version 1.3.
3. Versions from 1 through 100 are reserved for alpha/beta support of the specification in oneMKL and should be used only for special cases.

Managing Output

Using oneMKL Verbose Mode

When building applications that call Intel® oneAPI Math Kernel Library (oneMKL) functions, it may be useful to determine:

- which computational functions are called
- what parameters are passed to them
- how much time is spent to execute the functions
- (for GPU applications) which GPU device the kernel is executed on

You can get an application to print this information to a standard output device by enabling Intel® oneAPI Math Kernel Library (oneMKL) Verbose. Functions that can print this information are referred to as *verbose-enabled functions*.

When Verbose mode is active in an Intel® oneAPI Math Kernel Library (oneMKL) domain, every call of a verbose-enabled function finishes with printing a human-readable line describing the call. However, if your application gets terminated for some reason during the function call, no information for that function will be printed. The first call to a verbose-enabled function also prints a version information line.

For GPU applications, additional information (one or more GPU information lines) will also be printed by the first call to a verbose-enabled function, following the version information line which will be printed for the host CPU. If there is more than one GPU detected, each GPU device will be printed in a separate line.

We have different implementations for verbose with CPU applications and verbose with GPU applications. The Intel® MKL Verbose mode has 2 modes when used with CPU applications: disabled (default) and enabled. The Intel® MKL Verbose mode has three modes when used with GPU applications: disabled (default), enabled without timing, and enabled with synchronous timing.

To change the verbose mode, either set the environment variable `MKL_VERBOSE`:

	CPU application	GPU application
Set <code>MKL_VERBOSE</code> to 0	to disable Verbose	to disable Verbose
Set <code>MKL_VERBOSE</code> to 1	to enable Verbose	to enable Verbose without timing
Set <code>MKL_VERBOSE</code> to 2	to enable Verbose	to enable Verbose with synchronous timing

or call the support function `mkl_verbose(int mode)`:

	CPU application	GPU application
Call <code>mkl_verbose(0)</code>	to disable Verbose	to disable Verbose
Call <code>mkl_verbose(1)</code>	to enable Verbose	to enable Verbose without timing
Call <code>mkl_verbose(2)</code>	to enable Verbose	to enable Verbose with synchronous timing

Verbose with CPU Applications

Verbose output will be consisted of version information line and call description lines for CPU.

For CPU applications, you can enable oneMKL Verbose mode in these domains:

- BLAS (and BLAS-like extensions)
- LAPACK
- ScaLAPACK (selected functionality)
- FFT
- RNG

Verbose with GPU Applications

The verbose feature is enabled for GPU applications that uses DPC++ API or C/Fortran API with OpenMP offload. When used with GPU applications, verbose allows the measurement of execution time to be enabled or disabled with verbose mode. Timing is taken synchronously, so if verbose is enabled with timing, kernel executions will become synchronous (previous kernel will block later kernels)

Verbose output will be consisted of version information line, GPU information lines, and call description lines for GPU.

NOTE Timing for GPU applications is reported for overall execution. For selected functionality, device execution time can also be reported if the input queue was created with profiling information (see the [oneAPI GPU Optimization Guide](#)).

For GPU applications, you can enable oneMKL Verbose mode in these domains:

- BLAS (and BLAS-like extensions)
- LAPACK
- FFT
- RNG (DPC++ API only)

For Both CPU and GPU Verbose

Both enabling and disabling of the Verbose mode using the function call takes precedence over the environment setting. For a full description of the `mkl_verbose` function, see either the Intel® oneAPI Math Kernel Library Developer Reference for C or the Intel® oneAPI Math Kernel Library Developer Reference for Fortran. Both references are available in the Intel® Software Documentation Library.

Intel® oneAPI Math Kernel Library (oneMKL) Verbose mode is not a thread-local but a global state. In other words, if an application changes the mode from multiple threads, the result is undefined.

WARNING

The performance of an application may degrade with the Verbose mode enabled, especially when the number of calls to verbose-enabled functions is large, because every call to a verbose-enabled function requires an output operation.

See Also

[Intel Software Documentation Library](#)

Version Information Line

In the Intel® oneAPI Math Kernel Library (oneMKL) Verbose mode, the first call to a verbose-enabled function prints a version information line. The line begins with the `MKL_VERBOSE` character string and uses spaces as delimiters. The format of the rest of the line may change in a future release.

The following table lists information contained in a version information line and provides available links for more information:

Information	Description	Related Links
Intel® oneAPI Math Kernel Library (oneMKL) version.	This information is separated by a comma from the rest of the line.	
Operating system.	Possible values: • <code>Lnx</code> for Linux* OS • <code>Win</code> for Windows* OS • <code>OSX</code> for macOS*	
The host CPU frequency.		
Intel® oneAPI Math Kernel Library (oneMKL) interface layer used by the application.	Possible values: • <code>lp64</code> or <code>ilp64</code> on systems based on the Intel® 64 architecture.	Using the ILP64 Interface vs. LP64 Interface
Intel® oneAPI Math Kernel Library (oneMKL) threading layer used by the application.	Possible values: <code>intel_thread</code>, <code>gnu_thread</code>, <code>tbb_thread</code>, or <code>sequential</code>.	Linking with Threading Libraries

The following is an example of a version information line:

```
MKL_VERBOSE Intel(R) MKL 11.2 Beta build 20131126 for Intel(R) 64 architecture Intel(R)
Advanced Vector Extensions (Intel(R) AVX) Enabled Processor, Lnx 3.10GHz lp64
intel_thread
```

GPU Information Line

In Intel® oneAPI Math Kernel Library (oneMKL) Verbose mode for GPU applications, the first call to a verbose-enabled function prints out the GPU information line or lines for all detected GPU devices, each in a separate line. The line begins with the `MKL_VERBOSE Detected` character string and uses spaces as delimiters. The format of the rest of the line may change in a future release.

Only Intel® GPU is supported.

The following table lists information contained in a GPU information line.

See Also [Call Description Line for GPU](#)

Information	Description
GPU index	The index of the GPU device will be printed after the character string "GPU" (e.g. GPU0, GPU1, GPU2, etc). This GPU index will be used as a nickname of the device in call description lines to refer to the device.
Intel® GPU architecture	The value can be one of the following: • Intel(R) Gen9 • Intel(R) Xe_LP • Intel(R) Xe_HP • Intel(R) Xe_HPG • Intel(R) Xe_HPC • Unknown GPU
Runtime backend	The value printed is prefixed with Backend:
Vector Engine number	The value printed is prefixed with VE:
Stack number	The value printed is prefixed with Stack:
Maximum workgroup size	The value printed is prefixed with maxWGsize:

The following is an example of a GPU information line:

```
MKL_VERBOSE Detected GPU0 Intel(R)_Gen9 Backend:OpenCL VE:72 Stack:1 maxWGsize:256
```

Call Description Line

Call Description Line for CPU

In Intel® oneAPI Math Kernel Library (oneMKL) Verbose mode, each verbose-enabled function called from your application prints a call description line. The line begins with the `MKL_VERBOSE` character string and uses spaces as delimiters. The format of the rest of the line is subject to change in a future release.

The following table lists information contained in a call description line for Verbose with CPU applications and provides available links for more information:

Information	Description	Related Links
The name of the function.	Although the name printed may differ from the name used in the source code of the application (for example, the <code>cblas_</code> prefix of CBLAS functions is not printed), you can easily recognize the function by the printed name.	

Information	Description	Related Links
Values of the arguments.	- The values are listed in the order of the formal argument list. The list directly follows the function name, it is parenthesized and comma-separated. - Arrays are printed as addresses (to see the alignment of the data). - Integer scalar parameters passed by reference are printed by value. Zero values are printed for <code>NULL</code> references. - Character values are printed without quotes. - For all parameters passed by reference, the values printed are the values <i>returned by the function</i>. For example, the printed value of the <code>info</code> parameter of a LAPACK function is its value after the function execution. - For verbose-enabled functions in the ScaLAPACK domain, in addition to the standard input parameters, information about blocking factors, MPI rank, and process grid is also printed.	
Time taken by the function.	- The time is printed in convenient units (seconds, milliseconds, and so on), which are explicitly indicated. - The time may fluctuate from run to run. - The time printed may occasionally be larger than the time actually taken by the function call, especially for small problem sizes and multi-socket machines. To reduce this effect, bind threads that call Intel® oneAPI Math Kernel Library (oneMKL) to CPU cores by setting an affinity mask.	Managing Multi-core Performance for options to set an affinity mask.
Value of the <code>MKL_CBWR</code> environment variable.	The value printed is prefixed with <code>CNR</code> :	Getting Started with Conditional Numerical Reproducibility
Value of the <code>MKL_DYNAMIC</code> environment variable.	The value printed is prefixed with <code>Dyn</code> :	MKL_DYNAMIC
Status of the Intel® oneAPI Math Kernel Library (oneMKL) memory manager.	The value printed is prefixed with <code>FastMM</code> :	Avoiding Memory Leaks in oneMKL for a description of the Intel® oneAPI Math Kernel Library (oneMKL) memory manager
OpenMP* thread number of the calling thread.	The value printed is prefixed with <code>TID</code> :	
Values of Intel® oneAPI Math Kernel Library (oneMKL)	The first value printed is prefixed with <code>NThr</code> :	oneMKL-specific Environment Variables for Threading Control

Information	Description	Related Links
environment variables defining the general and domain-specific numbers of threads, separated by a comma.		

The following is an example of a call description line (with OpenMP threading):

```
MKL_VERBOSE
```

```
DGEMM(n,n,1000,1000,240,0x7ffff708bb30,0x7ff2aea4c000,1000,0x7ff28e92b000,240,0x7ffff708bb38,0x7ff28e08d000,1000) 1.66ms CNR:OFF Dyn:1 FastMM:1 TID:0 NThr:16
```

The following is an example of a call description line (with TBB threading):

```
MKL_VERBOSE
```

```
DGEMM(n,n,1000,1000,240,0x7ffff708bb30,0x7ff2aea4c000,1000,0x7ff28e92b000,240,0x7ffff708bb38,0x7ff28e08d000,1000) 1.66ms CNR:OFF Dyn:1 FastMM:1
```

NOTE For more information about selected threading, refer to [Version Information Line](#).

The following information is not printed because of limitations of Intel® oneAPI Math Kernel Library (oneMKL) Verbose mode:

- Input values of parameters passed by reference if the values were changed by the function.
For example, if a LAPACK function is called with a workspace query, that is, the value of the *lwork* parameter equals -1 on input, the call description line prints the result of the query and not -1.
- Return values of functions.
For example, the value returned by the function *ilaenv* is not printed.
- Floating-point scalars passed by reference.

Call Description Line for GPU

In Intel® oneAPI Math Kernel Library (oneMKL) Verbose mode, each verbose-enabled function called from your application prints a call description line. The line begins with the `MKL_VERBOSE` character string and uses spaces as delimiters. The format of the rest of the line may change in a future release.

The following table lists information contained in a call description line for verbose with GPU applications.

See Also [GPU Information Line](#)

Information	Description
The name of the function	Although the name printed may differ from the name used in the source code of the application, you can easily recognize the function by the printed name.
The values of the arguments	• The values are listed in the order of the formal argument list. The list directly follows the function name, and it is parenthesized and comma-separated.

Information	Description
Time taken by the function	- Arrays are printed as addresses (to show the alignment of the data). - Integer scalar parameters passed by reference are printed by value. Zero values are printed for NULL references. - Character values are printed without quotation marks. - For all parameters passed by reference, the values printed are the values returned by the function.
Device index	- If verbose is enabled with timing for GPU applications, kernel executions will become synchronous (previous kernel will block later kernels) and the measured time may include potential data transfers and/or data copies in host and devices. - If Verbose is enabled without timing for GPU applications, time will be printed out as 0. - The time is printed in convenient units (seconds, milliseconds, and so on), which are explicitly indicated. - The time may fluctuate from run to run. - The time printed may occasionally be larger than the time actually taken by the function call, especially for small problem sizes. The index of the GPU device on which the kernel is being executed will be printed after the character string "GPU" (e.g. GPU0, GPU1, GPU2, etc). Use the index and refer to the GPU information lines for more information about the specific device. If the kernel is executed on the host CPU, this field will be empty.

The following is an example of a call description line:

```
MKL_VERBOSE FFT(dcfi64) 224.30us GPU0
```

Some Limitations

For GPU applications, the call description lines may be printed out-of-order (the order of the call description lines printed in the verbose output may not be the order in which the kernels are submitted in the functions) for the following two cases:

- Verbose is enabled without timing and the kernel executions stay asynchronous.
- The kernel is not executed on one of the GPU devices, but on the host CPU (the device index will not be printed in this case).

Working with the Intel® Math Kernel Library Cluster Edition Software

Intel® oneAPI Math Kernel Library (oneMKL) includes distributed memory function domains for use on clusters:

- ScaLAPACK
- Cluster Fourier Transform Functions (Cluster FFT)
- Intel® Math Kernel Library Parallel Direct Sparse Solver for Clusters (Intel MKL Parallel Direct Sparse Solver for Clusters)

ScaLAPACK, Cluster FFT, and Cluster Sparse Solver are only provided for the Intel® 64 and Intel® Many Integrated Core architectures.

See Also

[Intel® oneAPI Math Kernel Library Structure](#)

[Managing Performance of the Cluster Fourier Transform Functions](#)

[Intel® Distribution for LINPACK* Benchmark](#)

Linking with Intel® Math Kernel Library Cluster Edition Software

The Intel® oneAPI Math Kernel Library (oneMKL) ScaLAPACK, Cluster FFT, and Cluster Sparse Solver support MPI implementations identified in the *Intel® oneAPI Math Kernel Library (oneMKL) Release Notes*.

To link a program that calls ScaLAPACK, Cluster FFT, or Cluster Sparse Solver, you need to know how to link a message-passing interface (MPI) application first.

Use mpi scripts to do this. For example, mpicc or mpif77 are C or FORTRAN 77 scripts, respectively, that use the correct MPI header files. The location of these scripts and the MPI library depends on your MPI implementation. For example, for the default installation of MPICH3, /opt/mpich/bin/mpicc and /opt/mpich/bin/mpif90 are the compiler scripts and /opt/mpich/lib/libmpi.a is the MPI library.

Check the documentation that comes with your MPI implementation for implementation-specific details of linking.

To link with ScaLAPACK, Cluster FFT, and/or Cluster Sparse Solver, use the following general form:

```
<MPI linker script> <files to link> \
-L <MKL path> [-Wl,--start-group] [<MKL cluster library>] \
<BLACS> <MKL core libraries> [-Wl,--end-group]
```

where the placeholders stand for paths and libraries as explained in the following table:

<MKL cluster library>	One of libraries for ScaLAPACK or Cluster FFT and appropriate architecture and programming interface (LP64 or ILP64). Available libraries are listed in Appendix C: Directory Structure in Detail . For example, for the LP64 interface, it is -lmkl_scalapack_lp64 or -lmkl_cdft_core. Cluster Sparse Solver does not require an additional computation library.
<BLACS>	The BLACS library corresponding to your architecture, programming interface (LP64 or ILP64), and MPI used. Available BLACS libraries are listed in Appendix C: Directory Structure in Detail . Specifically, choose one of -lmkl_blacs_intelmpi_lp64 or -lmkl_blacs_intelmpi_ilp64.
<MKL core libraries>	Processor optimized kernels, threading library, and system library for threading support, linked as described in Listing Libraries on a Link Line .

<MPI linker script>

A linker script that corresponds to the MPI version.

For example, if you are using Intel® MPI Library, want to statically link with ScaLAPACK using the LP64 interface, and have only one MPI process per core (and thus do not use threading), specify the following linker options:

```
-L$MKL_PATH -I$MKL_INCLUDE -Wl,--start-group $MKL_PATH/libmkl_scalapack_lp64.a $MKL_PATH/libmkl_blacs_intelmpi_lp64.a $MKL_PATH/libmkl_intel_lp64.a $MKL_PATH/libmkl_sequential.a $MKL_PATH/libmkl_core.a -static_mpi -Wl,--end-group -lpthread -lm
```

NOTE

Grouping symbols `-Wl,--start-group` and `-Wl,--end-group` are required for static linking.

Tip

Use the [Using the Link-line Advisor](#) to quickly choose the appropriate set of <MKL cluster Library>, <BLACS>, and <MKL core libraries>.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Linking Your Application with the Intel® oneAPI Math Kernel Library \(oneMKL\)](#)

[Examples of Linking for Clusters](#)

Setting the Number of OpenMP* Threads

The OpenMP* run-time library responds to the environment variable `OMP_NUM_THREADS`. Intel® oneAPI Math Kernel Library (oneMKL) also has other mechanisms to set the number of OpenMP threads, such as `themKL_NUM_THREADS` or `MKL_DOMAIN_NUM_THREADS` environment variables (see [Using Additional Threading Control](#)).

Make sure that the relevant environment variables have the same and correct values on all the nodes. Intel® oneAPI Math Kernel Library (oneMKL) does not set the default number of OpenMP threads to one, but depends on the OpenMP libraries used with the compiler to set the default number. For the threading layer based on the Intel compiler (`libmkl_intel_thread.a`), this value is the number of CPUs according to the OS.

Caution

Avoid over-prescribing the number of OpenMP threads, which may occur, for instance, when the number of MPI ranks per node and the number of OpenMP threads per node are both greater than one. The number of MPI ranks per node multiplied by the number of OpenMP threads per node should not exceed the number of hardware threads per node.

If you are using your login environment to set an environment variable, such as `OMP_NUM_THREADS`, remember that changing the value on the head node and then doing your run, as you do on a shared-memory (SMP) system, does not change the variable on all the nodes because `mpirun` starts a fresh default shell on all the nodes. To change the number of OpenMP threads on all the nodes, in `.bashrc`, add a line at the top, as follows:

```
OMP_NUM_THREADS=1; export OMP_NUM_THREADS
```


You can run multiple CPUs per node using MPICH. To do this, build MPICH to enable multiple CPUs per node. Be aware that certain MPICH applications may fail to work perfectly in a threaded environment (see the Known Limitations section in the *Release Notes*). If you encounter problems with MPICH and setting of the number of OpenMP threads is greater than one, first try setting the number of threads to one and see whether the problem persists.

Important

For Cluster Sparse Solver, set the number of OpenMP threads to a number greater than one because the implementation of the solver only supports a multithreaded algorithm.

See Also

[Techniques to Set the Number of Threads](#)

Using Shared Libraries

All needed shared libraries must be visible on all nodes at run time. To achieve this, set the `LD_LIBRARY_PATH` environment variable accordingly.

If Intel® oneAPI Math Kernel Library (oneMKL) is installed only on one node, link statically when building your Intel® oneAPI Math Kernel Library (oneMKL) applications rather than use shared libraries.

The Intel® compilers or GNU compilers can be used to compile a program that uses Intel® oneAPI Math Kernel Library (oneMKL). However, make sure that the MPI implementation and compiler match up correctly.

Setting Environment Variables on a Cluster

By default, when you call the MPI launch command `mpiexec`, the entire launching node environment is passed to the MPI processes. However, if there are undefined variables or variables that are different from what is stored in your environment, you can use `-env` or `-genv` options with `mpiexec`. Each of these options take two arguments- the name and the value of the environment variable to be passed.

```
-genv NAME1 VALUE1 -genv NAME2 VALUE2
```

```
-env NAME VALUE -genv
```

See these Intel® MPI Library examples on how to set the value of `OMP_NUM_THREADS` explicitly:

```
mpiexec -genv OMP_NUM_THREADS 2 ....
```

```
mpiexec -n 1 -host first -env OMP_NUM_THREADS 2 test.exe : -n 2 -host second -env  
OMP_NUM_THREADS 3 test.exe ....
```

See these Intel MPI Library examples on how to set the value of `MKL_BLACS_MPI` explicitly:

```
mpiexec -genv MKL_BLACS_MPI INTEL MPI ....
```

```
mpiexec -n 1 -host first -env MKL_BLACS_MPI INTEL MPI test.exe : -n 1 -host second -env  
MKL_BLACS_MPI INTEL MPI test.exe.
```

Interaction with the Message-Passing Interface

To improve performance of cluster applications, it is critical for Intel® oneAPI Math Kernel Library (oneMKL) to use the optimal number of threads, as well as the correct thread affinity. Usually, the optimal number is the number of available cores per node divided by the number of MPI processes per node. You can set the number of threads using one of the available methods, described in [Techniques to Set the Number of Threads](#).

If the number of threads is not set, Intel® oneAPI Math Kernel Library (oneMKL) checks whether it runs under MPI provided by the Intel® MPI Library. If this is true, the following environment variables define Intel® oneAPI Math Kernel Library (oneMKL) threading behavior:

- `I_MPI_THREAD_LEVEL`
- `MKL_MPI_PPN`
- `I_MPI_NUMBER_OF_MPI_PROCESSES_PER_NODE`
- `I_MPI_PIN_MAPPING`
- `OMPI_COMM_WORLD_LOCAL_SIZE`
- `MPI_LOCALNRANKS`

The threading behavior depends on the value of `I_MPI_THREAD_LEVEL` as follows:

- 0 or undefined.

Intel® oneAPI Math Kernel Library (oneMKL) considers that thread support level of Intel MPI Library is `ISMPI_THREAD_SINGLE` and defaults to sequential execution.

- 1, 2, or 3.

This value determines Intel® oneAPI Math Kernel Library (oneMKL) conclusion of the thread support level:

- 1 - `MPI_THREAD_FUNNELED`
- 2 - `MPI_THREAD_SERIALIZED`
- 3 - `MPI_THREAD_MULTIPLE`

In all these cases, Intel® oneAPI Math Kernel Library (oneMKL) determines the number of MPI processes per node using the other environment variables listed and defaults to the number of threads equal to the number of available cores per node divided by the number of MPI processes per node.

Important

Instead of relying on the discussed implicit settings, explicitly set the number of threads for Intel® oneAPI Math Kernel Library (oneMKL).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Managing Multi-core Performance](#)

[Intel® Software Documentation Library](#) for more information on Intel MPI Library
for more information on Intel MPI Library

Using a Custom Message-Passing Interface

While different message-passing interface (MPI) libraries are compatible at the application programming interface (API) level, they are often incompatible at the application binary interface (ABI) level. Therefore, Intel® oneAPI Math Kernel Library (oneMKL) provides a set of prebuilt BLACS libraries that support certain MPI libraries, but this, however, does not enable use of Intel® oneAPI Math Kernel Library (oneMKL) with other MPI libraries. To fill this gap, Intel® oneAPI Math Kernel Library (oneMKL) also includes the MKL MPI wrapper, which provides an MPI-independent ABI to Intel® oneAPI Math Kernel Library (oneMKL). The adaptor is provided as source code. To use Intel® oneAPI Math Kernel Library (oneMKL) with an MPI library that is not supported by default, you can use the adapter to build custom static or dynamic BLACS libraries and use them similarly to the prebuilt libraries.

Building a Custom BLACS Library

The MKL MPI wrapper is located in the `<mk1_directory>/interfaces/mklmpi` directory.

To build a custom BLACS library, from the above directory run the `make` command.

For example: the command

```
make libintel64
```

builds a static custom BLACS library `libmkl_blacs_custom_lp64.a` using the MPI compiler from the current shell environment. Look into the `<mkl_directory>/interfaces/mklmpi/makefile` for targets and variables that define how to build the custom library. In particular, you can specify the compiler through the `MPICC` variable.

For more control over the building process, refer to the documentation available through the command

```
make help
```

Using a Custom BLACS Library

Use custom BLACS libraries exactly the same way as you use the prebuilt BLACS libraries, but pass the custom library to the linker. For example, instead of passing the `libmkl_blacs_intelmpi_lp64.a` library, pass `libmkl_blacs_custom_lp64.a`.

See Also

[Linking with Intel® Math Kernel Library Cluster Edition Software](#)

Examples of Linking for Clusters

This section provides examples of linking with ScaLAPACK, Cluster FFT, and Cluster Sparse Solver.

Note that a binary linked with the Intel® oneAPI Math Kernel Library (oneMKL) cluster function domains runs the same way as any other MPI application (refer to the documentation that comes with your MPI implementation). For instance, the script `mpirun` is used in the case of MPICH2 or higher and OpenMPI, and the number of MPI processes is set by `-np`. In the case of MPICH2 or higher and Intel® MPI Library, start the daemon before running your application; the execution is driven by the script `mpiexec`.

For further linking examples, see the support website for Intel products at <https://www.intel.com/content/www/us/en/developer/get-help/overview.html>.

See Also

[Directory Structure in Detail](#)

Examples for Linking a C Application

These examples illustrate linking of an application under the following conditions:

- Main module is in C.
- You are using the Intel® oneAPI DPC++/C++ CompilerIntel® C++ Compiler.
- You are using MPICH2.
- Intel® oneAPI Math Kernel Library (oneMKL) functions use LP64 interfaces.
- The `PATH` environment variable contains a directory with the MPI linker scripts.
- `$MKLPATH` is a user-defined variable containing `<mkl_directory>/lib`.

To link dynamically with ScaLAPACK for a cluster of systems based on the Intel® 64 architecture, use the following link line:

```
mpicc <user files to link> \
-L$MKLPATH \
-lmkl_scalapack_lp64 \
-lmkl_blacs_intelmpi_lp64 \
-lmkl_intel_lp64 \
-lmkl_intel_thread -lmkl_core \
-liomp5 -lpthread
```

To link statically with Cluster FFT for a cluster of systems based on the Intel® 64 architecture, use the following link line:

```
mpicc <user files to link> \
-Wl,--start-group \
$MKL_PATH/libmkl_cdft_core.a \
$MKL_PATH/libmkl_blacs_intelmpi_lp64.a \
$MKL_PATH/libmkl_intel_lp64.a \
$MKL_PATH/libmkl_intel_thread.a \
$MKL_PATH/libmkl_core.a \
-Wl,--end-group \
-liomp5 -lpthread
```

To link dynamically with Cluster Sparse Solver for a cluster of systems based on the Intel® 64 architecture, use the following link line:

```
mpicc <user files to link> \
-L$MKL_PATH \
-lmkl_blacs_intelmpi_lp64 \
-lmkl_intel_lp64 \
-lmkl_intel_thread -lmkl_core \
-liomp5 -lpthread
```

See Also

[Linking with Intel® Math Kernel Library Cluster Edition Software Using the Link-line Advisor](#)

Examples for Linking a Fortran Application

These examples illustrate linking of an application under the following conditions:

- Main module is in Fortran.
- You are using the Intel® Fortran Compiler.
- You are using the Intel® MPI Library library.
- Intel® oneAPI Math Kernel Library (oneMKL) functions use LP64 interfaces.
- The `PATH` environment variable contains a directory with the MPI linker scripts.
- `$MKL_PATH` is a user-defined variable containing `<mkl_directory>/lib`.

To link dynamically with ScaLAPACK for a cluster of systems based on the Intel® 64 architecture, use the following link line:

```
mpiifort <user files to link> \
-L$MKL_PATH \
-lmkl_scalapack_lp64 \
-lmkl_blacs_intelmpi_lp64 \
-lmkl_intel_lp64 -lmkl_intel_thread -lmkl_core \
-liomp5 -lpthread
```

To link statically with Cluster FFT for a cluster of systems based on the Intel® 64 architecture, use the following link line:

```
mpiifort <user files to link> \
-Wl,--start-group \
$MKL_PATH/libmkl_cdft_core.a \
$MKL_PATH/libmkl_blacs_intelmpi_lp64.a \
$MKL_PATH/libmkl_intel_lp64.a \
$MKL_PATH/libmkl_intel_thread.a \
$MKL_PATH/libmkl_core.a \
-Wl,--end-group \
-liomp5 -lpthread
```

To link statically with Cluster Sparse Solver for a cluster of systems based on the Intel® 64 architecture, use the following link line:

```
mpiifort <user files to link> \
-Wl,--start-group \
$MKL_PATH/libmkl_blacs_intelmpi_lp64.a \
$MKL_PATH/libmkl_intel_lp64.a \
$MKL_PATH/libmkl_intel_thread.a \
$MKL_PATH/libmkl_core.a \
-Wl,--end-group \
-liomp5 -lpthread
```

See Also

[Linking with Intel® Math Kernel Library Cluster Edition Software](#)
[Using the Link-line Advisor](#)

Managing Behavior of the Intel® oneAPI Math Kernel Library with Environment Variables

See Also

[Intel® oneAPI Math Kernel Library \(oneMKL\)-specific Environment Variables for Threading Control](#)

Specifying the Code Branches

for how to use an environment variable to specify the code branch for Conditional Numerical Reproducibility
[Using Intel® oneAPI Math Kernel Library \(oneMKL\) Verbose Mode](#)
 for how to use an environment variable to set the verbose mode

Managing Behavior of Function Domains with Environment Variables

Setting the Default Mode of Vector Math with an Environment Variable

Intel® oneAPI Math Kernel Library (oneMKL) enables overriding the default setting of the Vector Mathematics (VM) global mode using the `MKL_VML_MODE` environment variable.

Because the mode is set or can be changed in different ways, their precedence determines the actual mode used. The settings and function calls that set or change the VM mode are listed below, with the precedence growing from lowest to highest:

1. The default setting
2. The `MKL_VML_MODE` environment variable
3. A call `vmlSetMode` function
4. A call to any VM function other than a service function

For more details, see the Vector Mathematical Functions section in the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference* and the description of the `vmlSetMode` function in particular.

To set the `MKL_VML_MODE` environment variable, use the following command in your command shell:

- For the bash shell:


```
export MKL_VML_MODE=<mode-string>
```
- For a C shell (csh or tcsh):


```
setenv MKL_VML_MODE <mode-string>
```

In these commands, *<mode-string>* controls error handling behavior and computation accuracy, consists of one or several comma-separated values of the *mode* parameter listed in the table below, and meets these requirements:

- Not more than one accuracy control value is permitted
- Any combination of error control values except `VML_ERRMODE_DEFAULT` is permitted
- No denormalized numbers control values are permitted

Values of the *mode* Parameter

Value of <i>mode</i>	Description
Accuracy Control	
<code>VML_HA</code>	high accuracy versions of VM functions
<code>VML_LA</code>	low accuracy versions of VM functions
<code>VML_EP</code>	enhanced performance accuracy versions of VM functions
Denormalized Numbers Handling Control	
<code>VML_FTZDAZ_ON</code>	Faster processing of denormalized inputs is enabled.
<code>VML_FTZDAZ_OFF</code>	Faster processing of denormalized inputs is disabled.
<code>VML_FTZDAZ_CURRENT</code>	Keep the current CPU settings for denormalized inputs.
Error Mode Control	
<code>VML_ERRMODE_IGNORE</code>	On computation error, VM Error status is updated, but otherwise no action is set. Cannot be combined with other <code>VML_ERRMODE</code> settings.
<code>VML_ERRMODE_NOERR</code>	On computation error, VM Error status is not updated and no action is set. Cannot be combined with other <code>VML_ERRMODE</code> settings.
<code>VML_ERRMODE_STDERR</code>	On error, the error text information is written to <i>stderr</i> .
<code>VML_ERRMODE_EXCEPT</code>	On error, an exception is raised.
<code>VML_ERRMODE_CALLBACK</code>	On error, an additional error handler function is called.
<code>VML_ERRMODE_DEFAULT</code>	On error, an exception is raised and an additional error handler function is called.

These commands provide an example of valid settings for the `MKL_VML_MODE` environment variable in your command shell:

- For the bash shell:

```
export MKL_VML_MODE=VML_LA,VML_ERRMODE_ERRNO,VML_ERRMODE_STDERR
```
- For a C shell (csh or tcsh):

```
setenv MKL_VML_MODE VML_LA,VML_ERRMODE_ERRNO,VML_ERRMODE_STDERR
```

NOTE

VM ignores the `MKL_VML_MODE` environment variable in the case of incorrect or misspelled settings of *mode*.

Managing Performance of the Cluster Fourier Transform Functions

Performance of Intel® oneAPI Math Kernel Library (oneMKL) Cluster FFT (CFFT) in different applications mainly depends on the cluster configuration, performance of message-passing interface (MPI) communications, and configuration of the run. Note that MPI communications usually take approximately 70% of the overall CFFT compute time. For more flexibility of control over time-consuming aspects of CFFT algorithms, Intel® oneAPI Math Kernel Library (oneMKL) provides the `MKL_CDFT` environment variable to set special values that affect CFFT performance. To improve performance of your application that intensively calls CFFT, you can use the environment variable to set optimal values for you cluster, application, MPI, and so on.

The `MKL_CDFT` environment variable has the following syntax, explained in the table below:

```
MKL_CDFT=option1[=value1],option2[=value2],...,optionN[=valueN]
```

Important

While this table explains the settings that usually improve performance under certain conditions, the actual performance highly depends on the configuration of your cluster. Therefore, experiment with the listed values to speed up your computations.

Option	Possible Values	Description
alltoallv	0 (default)	Configures CFFT to use the standard <code>MPI_Alltoallv</code> function to perform global transpositions.
	1	Configures CFFT to use a series of calls to <code>MPI_Isend</code> and <code>MPI_Irecv</code> instead of the <code>MPI_Alltoallv</code> function.
	4	Configures CFFT to merge global transposition with data movements in the local memory. CFFT performs global transpositions by calling <code>MPI_Isend</code> and <code>MPI_Irecv</code> in this case. Use this value in a hybrid case (MPI + OpenMP), especially when the number of processes per node equals one.
wo_omatcopy	0	Configures CFFT to perform local FFT and local transpositions separately. CFFT usually performs faster with this value than with <code>wo_omatcopy = 1</code> if the configuration parameter <code>DFTI_TRANSPOSE</code> has the value of <code>DFTI_ALLOW</code> . See the <i>Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference</i> for details.
	1	Configures CFFT to merge local FFT calls with local transpositions. CFFT usually performs faster with this value than with <code>wo_omatcopy = 0</code> if <code>DFTI_TRANSPOSE</code> has the value of <code>DFTI_NONE</code> .
	-1 (default)	Enables CFFT to decide which of the two above values to use depending on the value of <code>DFTI_TRANSPOSE</code> .
enable_soi	Not applicable	A flag that enables low-communication Segment Of Interest FFT (SOI FFT) algorithm for one-dimensional complex-to-complex CFFT, which requires fewer MPI communications than the standard nine-step (or six-step) algorithm.

Caution

While using fewer MPI communications, the SOI FFT algorithm incurs a minor loss of precision (about one decimal digit).

The following example illustrates usage of the environment variable assuming the bash shell:

```
export MKL_CDFT=wo_omatcopy=1,alltoallv=4,enable_soi
mpirun -ppn 2 -n 16 ./mkl_cdft_app
```

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Managing Invalid Input Checking in LAPACKE Functions

The high-level interface includes an optional, on by default, NaN check on all matrix inputs before calling any LAPACK routine. This option affects all routines. If an input matrix contains any NaNs, the input parameter corresponding to this matrix is flagged with a return value error. For example, if the fifth parameter is found to contain a NaN, the routine returns the value, -5. The middle-level interface does not contain the NaN check.

NaN checking on matrix input can be expensive. By default, NaN checking is turned **on**. LAPACKE provides a way to set it through the environment variable:

- Setting environment variable `LAPACKE_NANCHECK` to 0 turns OFF NaN-checking
- Setting environment variable `LAPACKE_NANCHECK` to 1 turns ON NaN-checking

The other way is the call the `LAPACKE_set_nancheck` function; see the Developer Reference for C's [LAPACK Auxiliary Routines](#) section for more information.

Note that the NaN-checking flag value set by the call to `LAPACKE_set_nancheck` always has higher priority than the environment variable, `LAPACKE_NANCHECK`.

Instruction Set–Specific Dispatching on Intel® Architectures

Intel® oneAPI Math Kernel Library (oneMKL) automatically queries and then dispatches the code path supported on your Intel® processor to the optimal instruction set architecture (ISA) by default. The `MKL_ENABLE_INSTRUCTIONS` environment variable or the `mkl_enable_instructions` support function enables you to dispatch to an ISA-specific code path of your choice. For example, you can run the Intel® Streaming SIMD Extensions 4.2 (Intel® SSE4.2) code path on an Intel processor based on Intel® Advanced Vector Extensions 2 (Intel® AVX2). This feature is not available on non-Intel processors.

In some cases, oneMKL also provides support for upcoming architectures ahead of hardware availability, but the library does not automatically dispatch the code path specific to an upcoming ISA by default. If for your exploratory work you need to enable an ISA for an Intel processor that is not yet released or if you are working in a simulated environment, you can use the `MKL_ENABLE_INSTRUCTIONS` environment variable or `mkl_enable_instructions` support function.

The following table lists possible values of `MKL_ENABLE_INSTRUCTIONS` alongside the corresponding ISA supported by a given processor. `MKL_ENABLE_INSTRUCTIONS` dispatches to the default ISA if the ISA requested is not supported on the particular Intel processor. For example, if you request to run the Intel AVX512 code path on a processor based on Intel AVX2, oneMKL runs the Intel AVX2 code path. The table also explains whether the ISA is dispatched by default on the processor that supports this ISA.

Value of <code>MKL_ENABLE_INSTRUCTIONS</code>	ISA	Dispatched by Default
AVX512	Intel® Advanced Vector Extensions 512 (Intel® AVX-512)	Yes
AVX512_E1	Intel® Advanced Vector Extensions 512 (Intel® AVX-512) with support for Intel® Deep Learning Boost (Intel® DL Boost)	Yes
AVX512_E2	Intel® Advanced Vector Extensions 512 (Intel® AVX-512) with support for Intel® Deep Learning Boost (Intel® DL Boost), EVEX-encoded AES, and Carry-Less Multiplication Quadword instructions	Yes
AVX512_E3	Intel® Advanced Vector Extensions 512 (Intel® AVX-512) with support for Intel® Deep Learning Boost (Intel® DL Boost) and bfloat16	Yes

Value of MKL_ENABLE_INSTRUCTIONS	ISA	Dispatched by Default
AVX512_E4	Intel® Advanced Vector Extensions 512 (Intel® AVX-512) with support for INT8, BF16, FP16 (limited) instructions, and Intel® Advanced Matrix Extensions (Intel® AMX) with INT8 and BF16	Yes
AVX512_E5	Intel® Advanced Vector Extensions 512 (Intel® AVX-512) with support for INT8, BF16, FP16 (limited) instructions, and Intel® Advanced Matrix Extensions (Intel® AMX) with INT8, BF16, and FP16	Yes
AVX2	Intel® Advanced Vector Extensions 2 (Intel® AVX2)	Yes
AVX2_E1	Intel® Advanced Vector Extensions 2 (Intel® AVX-2) with support for Intel® Deep Learning Boost (Intel® DL Boost)	Yes
SSE4_2	Intel® Streaming SIMD Extensions 4.2 (Intel® SSE4.2)	Yes
AVX10	Intel® Advanced Vector Extensions 10 Version 2 (Intel® AVX 10.2)–enabled processors with support for Intel® Advanced Matrix Extensions (Intel® AMX) with INT8, BF16, FP16, BF8, and FP8	No

For more details about the `mkl_enable_instructions` function, including the argument values, see the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference*.

For example:

- To turn on automatic CPU-based dispatching of Intel AVX-512 with support of Intel DL Boost, bfloat16, Intel AMX with bfloat16 and 8-bit integer, and FP16 instruction, do one of the following:
 - Call


```
mkl_enable_instructions(MKL_ENABLE_AVX512_E4)
```
 - Set the environment variable:
 - For the bash shell:


```
export MKL_ENABLE_INSTRUCTIONS=AVX512_E4
```
 - For a C shell (csh or tcsh):


```
setenv MKL_ENABLE_INSTRUCTIONS AVX512_E4
```
- To configure the library not to dispatch more recent architectures than Intel AVX2, do one of the following:
 - Call


```
mkl_enable_instructions(MKL_ENABLE_AVX2)
```
 - Set the environment variable:
 - For the bash shell:


```
export MKL_ENABLE_INSTRUCTIONS=AVX2
```
 - For a C shell (csh or tcsh):


```
setenv MKL_ENABLE_INSTRUCTIONS AVX2
```

NOTE

Settings specified by the `mkl_enable_instructions` function take precedence over the settings specified by the `MKL_ENABLE_INSTRUCTIONS` environment variable.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Configuring Your Integrated Development Environment to Link with Intel® oneAPI Math Kernel Library

Configuring the Eclipse* IDE CDT to Link with oneMKL

This section explains how to configure the Eclipse* Integrated Development Environment (IDE) C/C++ Development Tools (CDT) to link with Intel® oneAPI Math Kernel Library (oneMKL).

Tip

After configuring your CDT, you can benefit from the Eclipse-provided `code assist` feature. See Code/Context Assist description in the CDT Help for details.

To configure your Eclipse IDE CDT to link with Intel® oneAPI Math Kernel Library (oneMKL), you need to perform the steps explained below. The specific instructions for performing these steps depend on your version of the CDT and on the tool-chain/compiler integration. Refer to the CDT Help for more details.

To configure your Eclipse IDE CDT, do the following:

1. Open **Project Properties** for your project.
2. Add the Intel® oneAPI Math Kernel Library (oneMKL) include path, that is, `<mkl_directory>/include`, to the project's include paths.
3. Add the Intel® oneAPI Math Kernel Library (oneMKL) library path for the target architecture to the project's library paths. For example, for the Intel® 64 architecture, add `<mkl_directory>/lib/intel64_lin`.
4. Specify the names of the Intel® oneAPI Math Kernel Library (oneMKL) libraries to link with your application. For example, you may need the following libraries: `mkl_intel_lp64`, `mkl_intel_thread`, `mkl_core`, and `iomp5`.

NOTE

Because compilers typically require library names rather than file names, omit the "lib" prefix and "a" or "so" extension.

See Also

[Intel® oneAPI Math Kernel Library \(oneMKL\) Libraries to Link with Linking in Detail](#)

Intel® Math Kernel Library Benchmarks

Intel Optimized LINPACK Benchmark for Linux*

Intel Optimized LINPACK Benchmark for Linux* is a generalization of the LINPACK 1000 benchmark. It solves a dense ($\text{real} \times 8$) system of linear equations ($Ax=b$), measures the amount of time it takes to factor and solve the system, converts that time into a performance rate, and tests the results for accuracy. The generalization is in the number of equations (N) it can solve, which is not limited to 1000. It uses partial pivoting to assure the accuracy of the results.

Do not use this benchmark to report LINPACK 100 performance because that is a compiled-code only benchmark. This is a shared-memory (SMP) implementation which runs on a single platform. Do not confuse this benchmark with:

- Intel® Distribution for LINPACK* Benchmark, which is a distributed memory version of the same benchmark.
- LINPACK, the library, which has been expanded upon by the LAPACK library.

Intel provides optimized versions of the LINPACK benchmarks to help you obtain high LINPACK benchmark results on your genuine Intel processor systems more easily than with the High Performance Linpack (HPL) benchmark.

Additional information on this software, as well as on other Intel® software performance products, is available at <https://www.intel.com/content/www/us/en/developer/tools/overview.html>.

Acknowledgement

This product includes software developed at the University of Tennessee, Knoxville, Innovative Computing Laboratories.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Contents of the Intel® Optimized LINPACK Benchmark

The Intel Optimized LINPACK Benchmark for Linux* contains the following files, located in the `./share/mkl/benchmarks/linpack/` subdirectory within the Intel® oneAPI Math Kernel Library (oneMKL) directory:

File in <code>./share/mkl/benchmarks/linpack/</code>	Description
<code>xlinpack_xeon64</code>	The 64-bit program executable for a system with Intel Xeon processor using Intel® 64 architecture.
<code>lininput_xeon64</code>	Sample input file for a pre-determined problem.
<code>help.lpk</code>	Simple help file.
<code>xhelp.lpk</code>	Extended help file.
The following file is not available immediately after installation and appears as a result of the execution of the sample script.	
<code>lin_xeon64.txt</code>	Result of the sample script execution.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[High-level Directory Structure](#)

Running the Software

The following sample script can be used to run the benchmark for the pre-determined sample problem sizes. Change it to reflect the correct number of CPUs/threads, problem input files, and so forth.

```
# Setting up affinity for better threading performance
export KMP_AFFINITY=nowarnings,compact,1,0,granularity=fine

# Use numactl for better performance on multi-socket machines.
nnodes=`numactl -H 2>&1 | awk '/available:/ {print $2}'`
cpucore=`cat /proc/cpuinfo | awk '/cpu cores/ {print $4; exit}'`

if [ $nnodes -gt 1 -a $cpucore -gt 8 ]
then
    numacmd="numactl --interleave=all"
else
    numacmd=
fi

arch=xeon64
{
    date
    $numacmd ./xlinpack_$arch lininput_$arch
    echo -n "Done: "
    date
} | tee lin_$arch.txt
```

To run the software for other problem sizes, see the extended help included with the program. You can view extended help by running the program executable with the `-e` option:

```
./xlinpack_xeon64 -e
```

The pre-defined data input file `lininput_xeon64` is an example. Different systems have different numbers of processors or amounts of memory and therefore require new input files. The extended help provides insight into proper ways to change the sample input files.

The input file requires the following minimum amount of memory:

```
lininput_xeon64          16 GB
```

If the system has less memory than the above sample data input requires, you may need to edit or create your own data input files, as explained in the extended help.

The Intel Optimized LINPACK Benchmark determines the optimal number of OpenMP threads to use. To run a different number, you can set the `OMP_NUM_THREADS` or `MKL_NUM_THREADS` environment variable inside a sample script. If you run the Intel Optimized LINPACK Benchmark without setting the number of threads, it defaults to the number of physical cores.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Product and Performance Information

Notice revision #20201201

Known Limitations of the Intel® Optimized LINPACK Benchmark

The following limitations are known for the Intel Optimized LINPACK Benchmark for Linux*:

- Intel Optimized LINPACK Benchmark supports only OpenMP threading
- Intel Optimized LINPACK Benchmark is threaded to effectively use multiple processors. So, in multi-processor systems, best performance will be obtained with the Intel® Hyper-Threading Technology turned off, which ensures that the operating system assigns threads to physical processors only.
- If an incomplete data input file is given, the binaries may either hang or fault. See the sample data input files and/or the extended help for insight into creating a correct data input file.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Intel® Distribution for LINPACK* Benchmark and Intel® Optimized HPL-AI* Benchmark**Overview of the Intel® Distribution for LINPACK* Benchmark**

The Intel® Distribution for LINPACK* Benchmark is based on modifications and additions to High-Performance LINPACK* (HPL) (<http://www.netlib.org/benchmark/hpl/>) from Innovative Computing Laboratories (ICL) at the University of Tennessee, Knoxville. The Intel® Distribution for LINPACK* Benchmark can be used for TOP500 runs (see <http://www.top500.org>) and for benchmarking your cluster. To use the benchmark, you must be familiar with HPL usage. The Intel® Distribution for LINPACK* Benchmark provides some enhancements designed to make the HPL usage more convenient and to use Intel® MPI Library settings to improve performance.

The Intel® Distribution for LINPACK* Benchmark measures the amount of time it takes to factor and solve a random dense system of linear equations ($Ax=b$) in `real*8` precision, converts that time into a performance rate, and tests the results for accuracy. The benchmark uses random number generation and full row pivoting to ensure the accuracy of the results.

Intel provides optimized versions of the LINPACK* benchmarks to help you obtain high LINPACK* benchmark results on your systems based on genuine Intel processors more easily than with the standard HPL benchmark. The prebuilt binaries require Intel® MPI Library to be installed on the cluster. The run-time version of Intel MPI Library is free and can be downloaded from <https://www.intel.com/content/www/us/en/developer/tools/overview.html>.

The Intel package includes software developed at the University of Tennessee, Knoxville, ICL, and neither the University nor ICL endorse or promote this product. Although HPL is redistributable under certain conditions, this particular package is subject to the Intel® oneAPI Math Kernel Library (oneMKL) license.

HPL code is homogeneous by nature: it requires that each MPI process runs in an environment with similar CPU and memory constraints. The Intel® Distribution for LINPACK* Benchmark supports heterogeneity, meaning that the data distribution can be balanced to the performance requirements of each node, provided that there is enough memory on that node to support additional work. For information on how to configure Intel® oneAPI Math Kernel Library (oneMKL) to use the internode heterogeneity, see [Heterogeneous Support in the Intel® Distribution for LINPACK* Benchmark](#).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Overview of the Intel® Optimized HPL-AI* Benchmark

The Intel® Optimized HPL-AI Benchmark is heavily modified based on the High-Performance LINPACK* (HPL) Benchmark (<http://www.netlib.org/benchmark/hpl/>) from Innovative Computing Laboratories (ICL) at the University of Tennessee, Knoxville to meet the requirements of the HPL-AI (also known as HPL-MxP) rules (see <https://hpl-mxp.org/rules.md>). The usage of the Intel® Optimized HPL-AI Benchmark is very similar to that of the Intel® Distribution for LINPACK* Benchmark. Although the Intel® Optimized HPL-AI Benchmark measures the same amount of time as the Intel® Distribution for LINPACK* Benchmark, lower precision is used to obtain an approximate solution and the final solution is calculated by a double-precision-based iterative method.

For more details about the HPL-AI (HPL-MxP) benchmark, see the [HPL-MxP home page](#).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Contents of the Intel® Distribution for LINPACK* Benchmark and the Intel® Optimized HPL-AI* Benchmark

The Intel® Distribution for LINPACK* Benchmark and the Intel® Optimized HPL-AI Benchmark include prebuilt binaries linked with Intel® MPI Library. For a customized MPI implementation, tools are also included to build a binary using Intel® oneAPI Math Kernel Library (oneMKL) MPI wrappers. All the files are located in the ./benchmarks/mp_linpack/subdirectory of the Intel® oneAPI Math Kernel Library (oneMKL) directory.

File in <mkl_directory>/benchmarks/ mp_linpack/	Contents
--	----------

COPYRIGHT	Original Netlib HPL copyright document
readme.txt	Information about the files provided
Prebuilt executables for performance testing	
xhpl_intel64_dynamic	Prebuilt binary of the Intel® Distribution for LINPACK* Benchmark for CPU
xhpl_intel64_dynamic_gpu	Prebuilt binary of the Intel® Distribution for LINPACK* Benchmark for GPU
xhpl-ai_intel64_dynamic	Prebuilt binary of the Intel® Optimized HPL-AI Benchmark for CPU
xhpl-ai_intel64_dynamic_gpu	Prebuilt binary of the Intel® Optimized HPL-AI Benchmark for GPU
Run scripts and an input file example	
runme_intel64_dynamic	Sample run script for the Intel® 64 architecture and binary dynamically linked against Intel® MPI Library
runme_intel64_prv	Script that sets HPL environment variables. It is called by runme_intel64_dynamic
HPL.dat	Example of an HPL configuration file

File in <mk1 directory>/benchmarks/ Contents

mp_linpack/

Prebuilt libraries and utilities for building with a customized MPI implementation

libhpl_intel64.a	Library file required to build Intel® Distribution for LINPACK*
libhpl_intel64_gpu.a	Benchmark for the Intel® 64 architecture with a customized MPI implementation
libhpl-ai_intel64.a	
libhpl-ai_intel64_gpu.a	
HPL_main.c	Source code required to build Intel® Distribution for LINPACK*
	Benchmark for the Intel® 64 architecture with a customized MPI implementation

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[High-level Directory Structure](#)

Building the Intel® Distribution for LINPACK* Benchmark and the Intel® Optimized HPL-AI* Benchmark for a Customized MPI Implementation

To build the binary, follow these steps:

1. Specify the location of Intel® oneAPI Math Kernel Library (oneMKL) to be used (MKLROOT) .
2. Set up your MPI environment.
3. Run the following commands:

```
$> export MKL_DIRS=${MKLROOT}/lib
$> export MKL_LIBS="-L${MKL_DIRS} -Wl,-Bstatic -Wl,--start-group
-lmkl_intel_lp64 -lmkl_sequential
-lmkl_core -Wl,--end-group -Wl,-Bdynamic"
$> mpicc -o xhpl -O2 -I${MKLROOT}/include HPL_main.c
${MKLROOT}/share/mkl/interfaces/mklmpi/mklmpi-impl.c
libhpl_intel64.a ${MKL_LIBS} -ldl -lpthread -lm
$> mpicc -o xhpl_gpu -O2 -I${MKLROOT}/include HPL_main.c
${MKLROOT}/share/mkl/interfaces/mklmpi/mklmpi-impl.c
libhpl_intel64_gpu.a ${MKL_LIBS} -ldl -lpthread -lm
$> mpicc -o xhpl-ai -O2 -I${MKLROOT}/include HPL_main.c
${MKLROOT}/share/mkl/interfaces/mklmpi/mklmpi-impl.c
libhpl-ai_intel64.a ${MKL_LIBS} -ldl -lpthread -lm
$> mpicc -o xhpl-ai_gpu -O2 -I${MKLROOT}/include HPL_main.c
${MKLROOT}/share/mkl/interfaces/mklmpi/mklmpi-impl.c
libhpl-ai_intel64_gpu.a ${MKL_LIBS} -ldl -lpthread -lm
```

See Also

[Contents of the Intel® Distribution for LINPACK* Benchmark](#)

Building the Netlib HPL from Source Code

The source code for Intel® Distribution for LINPACK* Benchmark is not provided. However, you can download reference Netlib HPL source code from <http://www.netlib.org/benchmark/hpl/> . To build the HPL:

1. Download and extract the source code.
2. Copy the makefile:

```
$> cp setup/Make.Linux_Intel64 .
```

3. Edit `Make.Linux_Intel64` as appropriate
4. Build the HPL binary:

```
$> make arch=Linux_Intel64
```
5. Check that the built binary is available in the `bin/Linux_Intel64` directory.

NOTE

The Intel® Distribution for LINPACK* Benchmark may contain additional optimizations compared to the reference Netlib HPL implementation.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Contents of the Intel® Distribution for LINPACK* Benchmark](#)

Configuring Parameters

The most significant parameters in `HPL.dat` are P , Q , NB , and N . Specify them as follows:

- P and Q - the number of rows and columns in the process grid, respectively.
 $P*Q$ must be the number of MPI processes that HPL is using.
Choose $P \leq Q$.
- N - the problem size:
 - For homogeneous runs, choose N divisible by $NB*LCM(P,Q)$, where LCM is the least common multiple of the two numbers.
 - For heterogeneous runs, see [Heterogeneous Support in the Intel® Distribution for LINPACK* Benchmark](#) for how to choose N .

NOTE

Increasing N usually increases performance, but the size of N is bounded by memory. In general, you can compute the memory required to store the matrix (which does not count internal buffers) as $8*N*N/(P*Q)$ bytes, where N is the problem size and P and Q are the process grids in `HPL.dat`. A general rule is to choose a problem size that fills 80% of memory.

- NB - the block size of the data distribution.

The table below shows the recommended values of NB and element sizes for the CPU version:

Processors	Intel® Distribution for LINPACK* Benchmark	Intel® Optimized HPL-AI* Benchmark
Intel® Xeon Processor supporting Intel® Advanced Vector Extensions 2 (Intel® AVX2) instructions	192	192
Intel® Xeon Processor supporting Intel® Advanced Vector Extensions 512 (Intel® AVX-512) instructions	384	384
Intel® Xeon Processor supporting Intel® Advanced Vector Extensions 512 (Intel® AVX-512) instructions with Intel® Deep Learning Boost and bfloat16	384	768

Processors	Intel® Distribution for LINPACK* Benchmark	Intel® Optimized HPL-AI* Benchmark
Intel® Xeon Processor supporting Intel® Advanced Vector Extensions 512 (Intel® AVX-512) instructions with Intel® AMX bfloat16	384	1536
Element size	8 bytes	4 bytes

The table below shows the recommended values of NB and element sizes for the GPU version:

Processors	Intel® Distribution for LINPACK* Benchmark	Intel® Optimized HPL-AI* Benchmark
Intel® Data Center GPU Series	384	1152 or 1536
Element size	8 bytes	2 bytes

Ease-of-use Command-Line Parameters

The Intel® Distribution for LINPACK* Benchmark supports command-line parameters for HPL that help you to avoid making small changes in the `HPL.dat` input file every time you do a new run.

Placeholders in this command line illustrate these parameters:

```
./xhpl -n <problem size> -m <memory size in Mbytes> -b <block size> -p <grid row dimn> -q <grid column dimn>
```

You can also use command-line parameters with the sample runme scripts. For example:

```
./runme_intel64_dynamic -m <memory size in Mbytes> -b <block size> -p <grid row dimn> -q <grid column dimn>
```

For more command-line parameters, see [Heterogeneous Support in the Intel® Distribution for LINPACK* Benchmark](#).

If you want to run for $N=10000$ on a 1x3 grid, execute this command, provided that the other parameters in `HPL.dat` and the script are correct:

```
./runme_intel64_dynamic -n 10000 -p 1 -q 3
```

By using the `m` parameter you can scale by the memory size instead of the problem size. The `m` parameter only refers to the size of the matrix storage. Therefore, to use matrices that fit in 50000 Mbytes with $NB=256$ on 16 nodes, adjust the script to set the total number of MPI processes to 16 and execute this command:

```
./runme_intel64_dynamic -m 50000 -b 256 -p 4 -q 4
```

Running the Intel® Distribution for LINPACK* Benchmark and the Intel® Optimized HPL-AI* Benchmark

To run the Intel® Distribution for LINPACK* Benchmark on multiple nodes or on one node with multiple MPI processes, you need to use MPI and either modify `HPL.dat` or use [Ease-of-use Command-line Parameters](#). The following example describes how to run the dynamically-linked prebuilt Intel® Distribution for LINPACK* Benchmark binary using the script provided. To run other binaries, adjust the steps accordingly; specifically, change line 58 of `runme_intel64_dynamic` to point to the appropriate binary.

1. Load the necessary environment variables by accessing setup scripts.
2. In `HPL.dat`, set the problem size N to 10000. Because this setting is for a test run, the problem size should be small.
3. For better performance, enable non-uniform memory access (NUMA) on your system and configure to run an MPI process for each NUMA socket as explained below.
 - Refer to your BIOS settings to enable NUMA on your system.
 - Set the following variables at the top of the `runme_intel64_dynamic` script according to your cluster configuration:

<code>MPI_PROC_NUM</code>	The total number of MPI processes.
<code>MPI_PER_NODE</code>	The number of MPI processes per each cluster node.
<code>NUMA_PER_MPI</code>	The number of NUMA nodes per each MPI process.
<code>USE_HPL_AI</code>	Uncomment this to enable the Intel® Optimized HPL-AI* Benchmark.
<code>USE_HPL_GPU</code>	Uncomment this to enable GPUs.
<code>HPL_NUMSTACK</code>	The number of stacks on each GPU.
<code>HPL_NUMDEV</code>	The number of GPUs.

- In the `HPL.dat` file, set the parameters P_s and Q_s so that $P_s * Q_s$ equals the number of MPI processes. For example, for two processes, set P_s to 1 and Q_s to 2. Alternatively, leave the `HPL.dat` file as is and launch with the `-p` and `-q` command-line parameters.

4. Execute the `runme_intel64_dynamic` script:

```
./runme_intel64_dynamic
```

5. Rerun the test, increasing the size of the problem until the matrix size uses about 80% of the available memory. To do this, either modify N_s in line 6 of `HPL.dat` or use the `-n` command-line parameter:

- For 16 GB: 40000 N_s
- For 32 GB: 56000 N_s
- For 64 GB: 83000 N_s

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Heterogeneous Support in the Intel® Distribution for LINPACK* Benchmark

Intel® Distribution for LINPACK* Benchmark achieves heterogeneous support by distributing the matrix data unequally between the nodes. The heterogeneous factor command-line parameter `f` controls the amount of work to be assigned to the more powerful nodes, while the command-line parameter `c` controls the number of process columns for the faster nodes:

```
./xhpl -n <problem size> -b <block size> -p <grid row dimn> -q <grid column dimn> -f  
<heterogeneous factor> -c <number of faster processor columns>
```

If the heterogeneous factor is 2.5, roughly 2.5 times the work will be put on the more powerful nodes. The more work you put on the more powerful nodes, the more memory you might be wasting on the other nodes if all nodes have equal amount of memory. If your cluster includes many different types of nodes, you may need multiple heterogeneous factors.

Let P be the number of rows and Q the number of columns in your processor grid ($P \times Q$). The work must be *homogeneous* within each processor column because vertical operations, such as pivoting or panel factorization, are synchronizing operations. When there are two different types of nodes, use MPI to process all the faster nodes first and make sure the "PMAP process mapping" (line 9) of `HPL.dat` is set to 1 for Column-major mapping. Because all the nodes must be the same within a process column, the number of faster nodes must always be a multiple of P , and you can specify the faster nodes by setting the number of process columns C for the faster nodes with the `c` command-line parameter. The `-f 1.0 -c 0` setting corresponds to the default homogeneous behavior.

To understand how to choose the problem size N for a heterogeneous run, first consider a homogeneous system, where you might choose N as follows:

$$N \approx \sqrt{\text{Memory Utilization} * P * Q * \text{Memory Size in Bytes} / 8}$$

Memory Utilization is usually around 0.8 for homogeneous Intel® Xeon® processor systems. On a heterogeneous system, you may apply a different formula for N for each set of nodes that are the same and then choose the minimum N over all sets. Suppose you have a cluster with only one heterogeneous factor F

and the number of processor columns (out of the total Q) in the group with that heterogeneous factor equal to C . That group contains $P \cdot C$ nodes. First compute the sum of the parts: $S = F \cdot P \cdot C + P \cdot (Q - C)$. Note that on a homogeneous system $S = P \cdot Q$, $F = 1$, and $C = Q$. Take N as

$$N \sim \sqrt{\text{Memory Utilization} * P * Q * ((F \cdot P \cdot C) / S) * \text{Memory Size in Bytes} / 8}$$

or simply scale down the value of N for the homogeneous system by $\sqrt{F \cdot P \cdot C / S}$.

Example

Suppose the cluster has 100 nodes each having 64 GB of memory, and 20 of the nodes are 2.7 times as powerful as the other 80. Run one MPI process per node for a total of 100 MPI processes. Assume a square processor grid $P = Q = 10$, which conveniently divides up the faster nodes evenly. Normally, the HPL documentation recommends choosing a matrix size that consumes 80 percent of available memory. If N is the size of the matrix, the matrix consumes $8N^2 / (P \cdot Q)$ bytes. So a homogeneous run might look like:

```
./xhpl -n 820000 -b 256 -p 10 -q 10
```

If you redistribute the matrix and run the heterogeneous Intel® Distribution for LINPACK* Benchmark, you can take advantage of the faster nodes. But because some of the nodes will contain 2.7 times as much data as the other nodes, you must shrink the problem size (unless the faster nodes also happen to have 2.7 times as much memory). Instead of $0.8 \cdot 64\text{GB} \cdot 100$ total memory size, we have only $0.8 \cdot 64\text{GB} \cdot 20 + 0.8 \cdot 64\text{GB} / 2.7 \cdot 80$ total memory size, which is less than half the original space. So the problem size in this case would be 526000. Because $P = 10$ and there are 20 faster nodes, two processor columns are faster. If you arrange MPI to send these nodes first to the application, the command line looks like:

```
./xhpl -n 526000 -b 1024 -p 10 -q 10 -f 2.7 -c 2
```

The `m` parameter may be misleading for heterogeneous calculations because it calculates the problem size assuming all the nodes have the same amount of data.

Warning

The number of faster nodes must be $C \cdot P$. If the number of faster nodes is not divisible by P , you might not be able to take advantage of the extra performance potential by giving the faster nodes extra work.

While it suffices to simply provide `f` and `c` command-line parameters if you need only one heterogeneous factor, you must add lines to the `HPL.dat` input to support multiple heterogeneous factors. For the above example (two processor columns have nodes that are 2.7 times faster), instead of passing `f` and `c` command-line parameters you can modify the `HPL.dat` input file by adding these two lines to the end:

```
1          number of heterogeneous factors
0 1 2.7    [start_column, stop_column, heterogeneous factor for that range]
```

NOTE

Numbering of processor columns starts at 0. The start and stopping numbers must be between 0 and $Q - 1$ (inclusive).

If instead there are three different types of nodes in a cluster and you need at least two heterogeneous factors, change the number in the first row above from 1 to 2 and follow that line with two lines specifying the start column, stopping column, and heterogeneous factor.

When choosing parameters for heterogeneous support in `HPL.dat`, primarily focus on the most powerful nodes. The larger the heterogeneous factor, the more balanced the cluster may be from a performance viewpoint, but the more imbalanced from a memory viewpoint. At some point, further performance balancing might affect the memory too much. If this is the case, try to reduce any changes done for the faster nodes (such as in block sizes). Experiment with values in `HPL.dat` carefully because wrong values may greatly hinder performance.

When tuning on a heterogeneous cluster, do not immediately attempt a heterogeneous run, but do the following:

1. Break the cluster down into multiple homogeneous clusters.
2. Make heterogeneous adjustments for performance balancing. For instance, if you have two different sets of nodes where one is three times as powerful as the other, it must do three times the work.
3. Figure out the approximate size of the problem (per node) that you can run on each piece.
4. Do some homogeneous runs with those problem sizes per node and the final block size needed for the heterogeneous run and find the best parameters.
5. Use these parameters for an initial heterogeneous run.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Environment Variables

The table below lists Intel® oneAPI Math Kernel Library (oneMKL) environment variables to control runs of the Intel Distribution for LINPACK* Benchmark.

Environment Variable	Description	Value
HPL_LARGEPAGE	Defines the memory mapping to be used for the Intel® Xeon® processor.	0 or 1: 0 - normal memory mapping, default. 1 - memory mapping with large pages (2 MB per page mapping). It may increase performance.
HPL_LOG	Controls the level of detail for the HPL output.	0, 1, or 2: 0 - no log is displayed. 1 - only one root node displays a log, exactly the same as the <code>ASYOUGO</code> option provides. 2 - the most detailed log is displayed. All <i>P</i> root nodes in the processor column that owns the current column block display a log.
HPL_HOST_CORE, HPL_HOST_NODE	Specifies cores or Non-Uniform Memory Access (NUMA) nodes to be used. HPL_HOST_NODE requires NUMA mode to be enabled. You can check whether it is enabled by the <code>numactl --hardware</code> command. The default behavior is auto-detection of the core or NUMA node.	A list of integers ranging from 0 to the largest number of a core or NUMA node in the cluster.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Improving Performance of Your Cluster

To improve cluster performance, follow these steps, provided all required software is installed on each node:

1. Reboot all nodes.
2. Ensure all nodes are in identical conditions and no zombie processes are left running from prior HPL runs. To do this, run single-node Stream and Intel® Distribution for LINPACK* Benchmark on every node. Ensure results are within 10% of each other (problem size must be large enough depending on memory size and CPU speed). Investigate nodes with low performance for hardware/software problems.
3. Check that your cluster interconnects are working. Run a test over the complete cluster using an MPI test for bandwidth and latency, such as one found in the Intel® MPI Benchmarks package.
4. Run an Intel® Distribution for LINPACK* Benchmark on pairs of two or four nodes and ensure results are within 10% of each other. The problem size must be large enough depending on the memory size and CPU speed.
5. Run a small problem size over the complete cluster to ensure correctness.
6. Increase the problem size and run the real test load.
7. In case of problems go back to step 2.

Before making a heterogeneous run, always run its homogeneous equivalent first.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[Heterogeneous Support in the Intel® Distribution for LINPACK* Benchmark](#)

Intel® Optimized High Performance Conjugate Gradient Benchmark**Overview of the Intel Optimized HPCG**

The Intel® Optimized High Performance Conjugate Gradient Benchmark (Intel® Optimized HPCG) provides CPU- and GPU-optimized implementations of the HPCG benchmark (<http://hpcg-benchmark.org>). The CPU version is optimized for Intel® Xeon® processors with Intel® Advanced Vector Extensions 2 (Intel® AVX2) and Intel® Advanced Vector Extensions 512 (Intel® AVX-512) support. The GPU version is optimized for the Intel® Data Center GPU Max Series.

The HPCG Benchmark is intended to complement the High Performance LINPACK benchmark used in the TOP500 (<http://www.top500.org>) system ranking by providing a metric that better aligns with a broader set of important cluster applications.

The HPCG benchmark implementation is based on a 3-dimensional (3D) regular 27-point discretization of an elliptic partial differential equation. The implementation calls a 3D domain to fill a 3D virtual process grid for all the available MPI ranks. The HPCG benchmark uses the preconditioned conjugate gradient method (CG) to solve the intermediate systems of equations and incorporates a local and symmetric Gauss-Seidel preconditioning step that requires a triangular forward solve and a backward solve. A synthetic multi-grid V-cycle is used on each preconditioning step to make the benchmark better fit real-world applications. The HPCG benchmark implements matrix multiplication locally, with an initial halo exchange between neighboring processes. The benchmark exhibits irregular accesses to memory and fine-grain recursive computations that dominate many scientific workloads.

Intel® CPU Optimized HPCG Benchmark

The Intel® Optimized HPCG for CPUs benchmark contains source code of the HPCG v3.1 reference implementation with necessary modifications to include:

- Intel® architecture optimizations
- Prebuilt benchmark executables that link to Intel® oneAPI Math Kernel Library (oneMKL)
 - Inspector-executor Sparse BLAS kernels for sparse matrix-vector multiplication (SpMV)
 - Sparse triangular solve (SpTRSV)
 - Symmetric Gauss-Seidel smoother (SYMGS)

that are optimized for Intel AVX2 and Intel AVX-512 instruction sets. Use this package to evaluate the performance of distributed-memory systems based on any generation of the Intel® Xeon® processor E3, Intel® Xeon® processor E5, Intel® Xeon® processor E7, and Intel® Xeon® Scalable processor families.

The Intel® oneAPI Math Kernel Library Inspector-executor Sparse BLAS kernels SpMV, TRSV, and SYMGS are implemented using an inspector-executor model. The inspection step chooses the best algorithm for the input matrix and converts the matrix to a special internal representation to achieve high performance at the execution step.

Intel® GPU Optimized HPCG Benchmark

The Intel® GPU Optimized HPCG benchmark contains source code of the HPCG v3.1 reference implementation with necessary modifications to include:

- Using SYCL and C++ languages for efficient host and device scheduling of kernels
- Intel® GPU architecture optimizations
- A symmetric permutation of the sparse matrix to enable more task parallelism in some of the key computation kernels like the Symmetric Gauss-Seidel smoother.
- Conversion of the sparse matrix to an Ellpack Block Sparse (ESB) matrix format for more efficient vectorizable loads on the GPU hardware.
- Core computation kernels written in SYCL and using the "Explicit SIMD" (ESIMD) SYCL Extension for lower level Intel GPU programming:
 - Sparse matrix-vector multiplication (SpMV)
 - Sparse triangular solve (SpTRSV)
 - Symmetric Gauss-Seidel smoother (SYMGS).

Use this package to evaluate the performance of distributed-memory systems based on the Intel® Data Center GPU Max Series family.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Versions of the Intel® CPU Optimized HPCG

The Intel® CPU Optimized HPCG package includes prebuilt HPCG benchmark for Intel® MPI Library 2019.2.2 or higher. All the files of the benchmark are located in the `/share/mkl/benchmarks/hpcg/hpcg_cpu` subdirectory of the Intel® oneAPI Math Kernel Library (oneMKL) directory. The following versions of the benchmark are available:

File in .../benchmarks/ hpcg/hpcg_cpu/bin	Description
xhpcg_avx2	The Intel AVX2 optimized version of the benchmark, optimized for systems based on the third and later generations of the Intel® Xeon® processor E3 family, Intel® Xeon® processor E5 family, Intel® Xeon® processor E7 family, and future Intel processors with Intel AVX2 support. The prebuilt Intel AVX2 optimized version of the benchmark does not run on systems that do not support Intel AVX2 instructions.
xhpcg_skx and xhpcg_avx512	The Intel® Xeon® Scalable processor (formerly Skylake) optimized version of the benchmark is designed for systems based on Intel® Xeon® Scalable processors and future Intel processors with Intel AVX-512 support. Running the Intel AVX2 optimized versions of the benchmark on an Intel AVX-512-enabled system produces sub-optimal performance. The Intel® Xeon® Scalable processor optimized version of the benchmark does not run on systems that do not support Intel AVX-512.
NOTE The two binaries are the same; however, due to historical reasons, we keep xhpcg_skx but add the more generic (and better named) xhpcg_avx512 binary.	
hpcg.dat	HPCG benchmark input file.

The Intel Optimized HPCG package also includes the source code necessary to build these versions of the benchmark for other MPI implementations, such as MPICH2, or Open MPI: Intel AVX2 optimized version, and Intel AVX-512 optimized version. Build instructions are available in the `QUICKSTART` file included with the package.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

High-level Directory Structure

Versions of the Intel® GPU Optimized HPCG

The Intel® GPU Optimized HPCG package includes prebuilt HPCG benchmark for Intel® MPI Library 2021.11.0 or higher. All the files of the benchmark are located in the `/share/mkl/benchmarks/hpcg/hpcg_gpu` subdirectory of the Intel® oneAPI Math Kernel Library (oneMKL) directory. The following versions of the benchmark are available:

File in .../benchmarks/ hpcg/hpcg_gpu/bin	Description
xhpcg_impi_pvc with libhpcg_kernels_imp i_pvc.so	The Intel® GPU optimized version of the benchmark, optimized for Intel Data Center GPU Max Series family of discrete GPUs. The prebuilt binary and companion shared library are ahead of time (AOT) compiled for "-device pvc", so they may not work on other Intel GPUs.
hpcg.dat	HPCG benchmark input file.

The Intel Optimized HPCG package also includes the source code necessary to build these versions of the benchmark for other MPI implementations, such as MPICH. Build instructions are available in the `README.md` file included with the package.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

See Also

[High-level Directory Structure](#)

Getting Started with Intel® CPU Optimized HPCG

To start working with the benchmark:

1. On a cluster file system, unpack the Intel® CPU Optimized HPCG package to a directory accessible by all nodes. Read and accept the license as indicated in the `readme.txt` file included in the package.
2. Change the directory to `hpcg/hpcg_cpu/bin`.
3. Determine the prebuilt version of the benchmark that is best for your system or follow `QUICKSTART` instructions to build a version of the benchmark for your MPI implementation.
4. Ensure that the Intel® oneAPI Math Kernel Library (oneMKL), Intel C/C++ Compiler, and MPI runtime environments have been set properly. You can do this using the `vars.sh` scripts that are included in those distributions.
5. Run the chosen version of the benchmark.

For Intel® Xeon® Processor families up to the 3rd generation Xeon Scalable servers (previously code-named Ice Lake), the Intel AVX2 and AVX512 optimized versions perform best with one MPI process per socket and one OpenMP* thread per core, skipping simultaneous multithreading (SMT) threads. We recommend setting the affinity as `KMP_AFFINITY=granularity=fine,compact,1,0` when hyperthreading is enabled on the system. Specifically, for a 128-node cluster with two Intel® Xeon® Platinum 8380 processors per node, run the executable as follows:

```
#> export OMP_NUM_THREADS=20; export KMP_AFFINITY=granularity=fine,compact,1,0;
mpiexec.hydra --genval1 -n 256 --ppn 2 -f ${nodefile} ./bin/xhpcg_avx512 --nx=192
--ny=192 --nz=192 --run-real-ref=1
```

For Intel® Xeon® Scalable Processor families from 4th generation Xeon Scalable servers and beyond, the AVX512 optimized versions often perform best with one MPI process per NUMA node (or per die on each processor socket) and two OpenMP* threads per core, utilizing simultaneous multithreading (SMT) threads. We recommend setting the affinity as `KMP_AFFINITY=granularity=fine,compact`. Specifically, for a 128-node cluster with two Intel® Xeon® Platinum 8480+ processors per node, run the executable as follows:

```
#> export OMP_NUM_THREADS=28; export KMP_AFFINITY=granularity=fine,compact;
mpiexec.hydra --genval1 -n 1024 --ppn 8 -f ${nodefile} ./bin/xhpcg_avx512 --nx=192
--ny=192 --nz=192 --run-real-ref=1
```

6. When the benchmark completes execution, which usually takes a few minutes, find the YAML file with official results in the current directory. The performance rating of the benchmarked system is in the last section of the file:

```
HPCG result is VALID with a GFLOP/s rating of: [GFLOP/s]
```

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Getting Started with Intel® GPU Optimized HPCG

To start working with the benchmark:

1. On a cluster file system, unpack the Intel® GPU Optimized HPCG package to a directory accessible by all nodes. Read and accept the license as indicated in the `readme.txt` file included in the package.
2. Change the directory to `hpcg/hpcg_gpu/bin`.
3. Determine the prebuilt version of the benchmark that is best for your system or follow `README.md` instructions to build a version of the benchmark for your MPI implementation, including modifying and utilizing one of the provided `example_*_runscript.sh` files for building and running.
4. Ensure that the Intel® oneAPI Math Kernel Library (oneMKL), Intel SYCL C++ Compiler, and MPI runtime environments have been set properly. You can do this using the `vars.sh` scripts that are included in those distributions or using oneAPI HPCG toolkit `vars.sh` scripts.
5. Run the chosen version of the benchmark.

For Intel® Data Center GPU Max Series, the PVC optimized versions perform best with one MPI process per GPU tile. The reference code (used for verifying correctness and convergence) utilizes OpenMP* threads, which should be divided up evenly between the various ranks on the node. Specifically, for a 128-node cluster with four Intel® Data Center GPU Max Series cards per node each with two sockets on a two-socket fourth-generation Platinum 8480+ with 56 cores per socket and 2 simultaneous multithreading (SMT) threads per core, run the executable as follows:

```
#> export OMP_NUM_THREADS=28; export KMP_AFFINITY=granularity=fine,compact; export
SYCL_QUEUE_THREAD_POOL_SIZE=26; mpiexec.hydra --genvall -np 1024 --ppn 8 -f $
{nodefile} ./bin/xhpcg_impi_pvc --nx=320 --ny=320 --nz=320 --run-real-ref=1 --
affinity-per-node=compact
```

6. When the benchmark completes execution, which usually takes a few minutes, find the YAML file with official results in the current directory. The performance rating of the benchmarked system is in the last section of the file:

```
HPCG result is VALID with a GFLOP/s rating of: [GFLOP/s]
```

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Choosing the Best Configuration and Problem Sizes for CPUs

The performance of the Intel CPU Optimized HPCG depends on many system parameters including (but not limited to) the hardware configuration of the host and MPI implementation used. To get the best performance for a specific system configuration, choose a combination of these parameters:

- The number of MPI processes per host node
- The number of OpenMP* threads per MPI process
- The local problem size

On Intel® Xeon® processor-based clusters, use the Intel AVX2 or Intel AVX-512 optimized version of the benchmark depending on the supported instruction set. For CPUs with one natural NUMA node per socket (up to 3rd generation Xeon® scalable processors), we recommend using one MPI process per CPU socket and one OpenMP* thread per physical CPU core skipping SMT threads. For CPUs from 4th generation Xeon® Scalable processors and beyond, there is often a more natural NUMA like division within each socket, and it is often best to use more MPI processes per socket matching these natural divisions. For instance, there are four dies in each socket of the Intel Xeon® Platinum 8480+ and 9480 processors corresponding to 14 physical cores (and for the 9480 model, an HBM stack attached to that die), and four MPI processes per socket can give top performance. In other processors, there is not a natural subdivision; however, even with an increasing number of cores per socket, it is sometimes worthwhile to increase the number of MPI processes per socket to reduce the number of OpenMP threads per MPI process, leading to better balance and performance. It is worth trying a single MPI process per socket with all OpenMP threads as well as multiple ranks per socket, with the number of OpenMP threads targeted to a range of 10–36 threads assigned per MPI process in order to find the best performance on the system.

For best performance, use a local problem size that is large enough to better utilize available cores, but not too large, so that all tasks fit the available memory. With modern CPUs, the last level cache (LLC) sizes per socket have grown immensely, so to comply with current HPCG benchmark requirements, the local problem size ($n_x \times n_y \times n_z$) should also be chosen large enough so that the combined size of a vector per MPI process on the socket (each vector is $n_x \times n_y \times n_z \times \text{sizeof}(\text{double})$ bytes) should also not fit in the LLC.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Choosing the Best HPCG Configuration for GPUs

The performance of the Intel GPU Optimized HPCG depends on many system parameters including (but not limited to) the hardware configuration of the host node and one or more devices attached to the node as well as the MPI implementation used. To get the best performance for a specific system configuration, choose a combination of these parameters:

- The number of MPI processes per host node (defining work on the host + an attached device)
- The number of OpenMP* threads per MPI process for reference code used in validation of benchmark
- The local problem size

On Intel® Data Center GPU Max Series GPUs, we recommend the use of one MPI process per tile with a large local problem size. With modern GPUs, the last level cache (LLC) sizes per tile can be either extremely large or quite small, and the device memory can be quite limited. So to comply with current HPCG benchmark requirements, the local problem size ($n_x \times n_y \times n_z$) should be large enough so that the size of a vector from the benchmark (each vector is $n_x \times n_y \times n_z \times \text{sizeof}(\text{double})$ bytes) does not completely fit in the LLC of the device, but not too large that the full benchmark system doesn't fit in device memory.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Appendix A: Intel® oneAPI Math Kernel Library Language Interfaces Support

See Also

[Mixed-language Programming with Intel® oneAPI Math Kernel Library \(oneMKL\)](#)

Language Interfaces Support, by Function Domain

The following table shows language interfaces that Intel® oneAPI Math Kernel Library (oneMKL) provides for each function domain. However, Intel® oneAPI Math Kernel Library (oneMKL) routines can be called from other languages using mixed-language programming. See [Mixed-language Programming with the Intel Math Kernel Library](#) for an example of how to call Fortran routines from C/C++.

Function Domain	Fortran interface	C/C++ interface
Basic Linear Algebra Subprograms (BLAS)	Yes	through CBLAS

Function Domain	Fortran interface	C/C++ interface
BLAS-like extension transposition routines	Yes	Yes
Sparse BLAS Level 1	Yes	through CBLAS
Sparse BLAS Level 2 and 3	Yes	Yes
LAPACK routines for solving systems of linear equations	Yes	Yes
LAPACK routines for solving least-squares problems, eigenvalue and singular value problems, and Sylvester's equations	Yes	Yes
Auxiliary and utility LAPACK routines	Yes	Yes
Parallel Basic Linear Algebra Subprograms (PBLAS)	Yes	
ScaLAPACK	Yes	†
Direct Sparse Solvers/ Intel® oneAPI Math Kernel Library (oneMKL) PARDISO, a direct sparse solver based on Parallel Direct Sparse Solver (PARDISO*)	Yes	Yes
Intel® Math Kernel Library Parallel Direct Sparse Solver for Clusters (Intel MKL Parallel Direct Sparse Solver for Clusters)	Yes	Yes
Other Direct and Iterative Sparse Solver routines	Yes	Yes
Vector Mathematics (VM)	Yes	Yes
Vector Statistics (VS)	Yes	Yes
Fast Fourier Transforms (FFT)	Yes	Yes
Cluster FFT	Yes	Yes
Trigonometric Transforms	Yes	Yes
Fast Poisson, Laplace, and Helmholtz Solver (Poisson Library)	Yes	Yes
Optimization (Trust-Region) Solver	Yes	Yes
Data Fitting	Yes	Yes
Extended Eigensolver	Yes	Yes
Support functions (including memory allocation)	Yes	Yes

† Supported using a mixed language programming call. See [Include Files](#) for the respective header file.

Include Files

The table below lists Intel® oneAPI Math Kernel Library (oneMKL) include files.

Function Domain/ Purpose	Fortran Include Files	C/C++ Include Files
All function domains	<code>mkl.fi</code>	<code>mkl.h</code>
BLACS		<code>mkl_blacs.h</code> ††

Function Domain/ Purpose	Fortran Include Files	C/C++ Include Files
BLAS	blas.f90 mkl_blas.fi [†]	mkl_blas.h [‡]
BLAS-like Extension Transposition Routines	mkl_trans.fi [†]	mkl_trans.h [‡]
CBLAS Interface to BLAS		mkl_cblas.h [‡]
Sparse BLAS	mkl_spblas.fi [†]	mkl_spblas.h [‡]
LAPACK	lapack.f90 mkl_lapack.fi [†]	mkl_lapack.h [‡]
C Interface to LAPACK		mkl_lapacke.h [‡]
PBLAS		mkl_pblas.h ^{‡‡}
ScaLAPACK		mkl_scalapack.h ^{‡‡}
Intel® oneAPI Math Kernel Library (oneMKL) PARDISO	mkl_pardiso.f90 mkl_pardiso.fi [†]	mkl_pardiso.h [‡]
Intel® Math Kernel Library Parallel Direct Sparse Solver for Clusters (Intel MKL Parallel Direct Sparse Solver for Clusters)	mkl_cluster_sparse_solve r.f90	mkl_cluster_sparse_solve r.h [‡]
Direct Sparse Solver (DSS)	mkl_dss.f90 mkl_dss.fi [†]	mkl_dss.h [‡]
RCI Iterative Solvers		
ILU Factorization	mkl_rci.f90 mkl_rci.fi [†]	mkl_rci.h [‡]
Optimization Solver	mkl_rci.f90 mkl_rci.fi [†]	mkl_rci.h [‡]
Vector Mathematics	mkl_vml.90 mkl_vml.fi [†]	mkl_vml.h [‡]
Vector Statistics	mkl_vsl.f90 mkl_vsl.fi [†]	mkl_vsl.h [‡]
Fast Fourier Transforms	mkl_dfti.f90	mkl_dfti.h [‡]
Cluster Fast Fourier Transforms	mkl_cdft.f90	mkl_cdft.h ^{‡‡}
Partial Differential Equations Support		
Trigonometric Transforms	mkl_trig_transforms.f90	mkl_trig_transform.h [‡]
Poisson Solvers	mkl_poisson.f90	mkl_poisson.h [‡]
Data Fitting	mkl_df.f90	mkl_df.h [‡]
Extended Eigensolver	mkl_solvers_ee.fi [†]	mkl_solvers_ee.h [‡]
Support functions	mkl_service.f90 mkl_service.fi [†]	mkl_service.h [‡]

Function Domain/ Purpose	Fortran Include Files	C/C++ Include Files
Declarations for replacing memory allocation functions. See Redefining Memory Functions for details.		<code>i_malloc.h</code>
Auxiliary macros to determine the version of Intel® oneAPI Math Kernel Library (oneMKL) at compile time.	<code>mkl_version</code>	<code>mkl_version[†]</code>

[†] You can use the `mkl.fi` include file in your code instead.

[‡] You can include the `mkl.h` header file in your code instead.

^{‡‡} Also include the `mkl.h` header file in your code.

See Also

[Language Interfaces Support, by Function Domain](#)

Appendix B: Support for Third-Party Interfaces

FFTW Interface Support

Intel® oneAPI Math Kernel Library (oneMKL) offers two collections of wrappers for the FFTW interface (www.fftw.org). The wrappers are the superstructure of FFTW to be used for calling the Intel® oneAPI Math Kernel Library (oneMKL) Fourier transform functions. These collections correspond to the FFTW versions 2.x and 3.x and the Intel® oneAPI Math Kernel Library (oneMKL) versions 7.0 and later.

These wrappers enable using Intel® oneAPI Math Kernel Library (oneMKL) Fourier transforms to improve the performance of programs that use FFTW without changing the program source code. See the "FFTW Interface to Intel® oneAPI Math Kernel Library" appendix in the *Intel® oneAPI Math Kernel Library (oneMKL) Developer Reference* for details on the use of the wrappers.

Important

For ease of use, the FFTW3 interface is also integrated in Intel® oneAPI Math Kernel Library (oneMKL).

Caution

The FFTW2 and FFTW3 interfaces are not compatible with each other. Avoid linking to both of them. If you must do so, first modify the wrapper source code for FFTW2:

1. Change every instance of `fftw_destroy_plan` in the `fftw2xc` interface to `fftw2_destroy_plan`.
2. Change all the corresponding file names accordingly.
3. Rebuild the pertinent libraries.

Appendix C: Directory Structure in Detail

Tables in this section show contents of the Intel® oneAPI Math Kernel Library (oneMKL) architecture-specific directories.

See Also

[High-level Directory Structure](#)

Using Language-Specific Interfaces with Intel® oneAPI Math Kernel Library (oneMKL)

Intel® oneAPI Math Kernel Library Benchmarks

Detailed Structure of the Intel® 64 Architecture Directories

Product and Performance Information
Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex .
Notice revision #20201201

Static Libraries in the lib Directory

Some of the libraries in this directory are optional. However, some optional libraries are installed by default, while the rest are not. To get those libraries that are not installed by default, explicitly select the specified optional component during installation.

File	Contents	Optional Component	
		Name	Installed by Default
SYCL API Layer			
mkl_sycl.a	SYCL library for the Intel DPC++ compilers. Required if SYCL-specific functionality is needed. This library includes BLAS, LAPACK, DFT, Data Fitting, Vector Math, Sparse BLAS, RNG and Summary Statistic domains.		
Interface Layer			
libmkl_intel_lp64.a	LP64 interface library for the Intel compilers. Also used for other supported compilers that do not have a specialized Intel® oneAPI Math Kernel Library (oneMKL) interface library. All oneMKL function domains support LP64 programming. Read more about LP64 in Using the ILP64 Interface vs. LP64 Interface..		
libmkl_intel_ilp64.a	ILP64 interface library for the Intel compilers. Also used for other supported compilers		

File	Contents	Optional Component	
		Name	Installed by Default
	that do not have a specialized oneMKL interface library. All oneMKL function domains support ILP64 programming with some limitations for FFT. Read more about ILP64 and limitations in Using the ILP64 Interface vs. LP64 Interface..		
libmkl_blas95_lp64.a	Fortran 95 interface library for BLAS for the Intel® Fortran compiler. Supports the LP64 interface.	Fortran 95 interfaces for BLAS and LAPACK	Yes
libmkl_blas95_ilp64.a	Fortran 95 interface library for BLAS for the Intel® Fortran compiler. Supports the ILP64 interface.	Fortran 95 interfaces for BLAS and LAPACK	Yes
libmkl_lapack95_lp64.a	Fortran 95 interface library for LAPACK for the Intel® Fortran compiler. Supports the LP64 interface.	Fortran 95 interfaces for BLAS and LAPACK	Yes
libmkl_lapack95_ilp64.a	Fortran 95 interface library for LAPACK for the Intel® Fortran compiler. Supports the ILP64 interface.	Fortran 95 interfaces for BLAS and LAPACK	Yes
libmkl_gf_lp64.a	LP64 interface library for the GNU Fortran compilers. All oneMKL function domains support LP64 programming. Read more about LP64 in Using the ILP64 Interface vs. LP64 Interface..	GNU* Compiler Collection support	Yes
libmkl_gf_ilp64.a	ILP64 interface library for the GNU Fortran compilers. All oneMKL function domains support ILP64 programming with some limitations for FFT. Read more about ILP64 and	GNU* Compiler Collection support	Yes

File	Contents	Optional Component	
		Name	Installed by Default
	limitations in Using the ILP64 Interface vs. LP64 Interface..		
Threading Layer			
libmkl_intel_thread.a	OpenMP threading library for the Intel compilers. All oneMKL function domains support Intel OpenMP threading.		
libmkl_tbb_thread.a	Intel TBB threading library for the Intel compilers. All oneMKL function domains support Intel TBB threading.	Intel TBB threading support	Yes
libmkl_gnu_thread.a	OpenMP threading library for the GNU Fortran and C compilers. All oneMKL function domains support GNU OpenMP threading.	GNU* Compiler Collection support	Yes
libmkl_sequential.a	Sequential library. All oneMKL function domains support sequential mode.		
Computational Layer			
libmkl_core.a	All non-architecture-specific implementation and library dispatcher for architecture-specific kernels. Read more about how to work with <code>mkl_enable_instructions</code> and <code>mkl_cbwr_set</code> in the developer reference. <code>mkl_core</code> is required for any link configurations.		
Cluster Libraries			
libmkl_scalapack_lp64.a	ScaLAPACK routine library supporting the LP64 interface	Cluster support	

File	Contents	Optional Component	
		Name	Installed by Default
libmkl_scalapack_ilp64.a	ScaLAPACK routine library supporting the ILP64 interface	Cluster support	
libmkl_cdft_core.a	Cluster version of FFT functions.	Cluster support	
libmkl_blacs_intelmpi_lp64.a	LP64 version of BLACS routines for Intel® MPI Library and MPICH2 or higher.	Cluster support	
libmkl_blacs_intelmpi_ilp64.a	ILP64 version of BLACS routines for Intel MPI Library and MPICH2 or higher.	Cluster support	
libmkl_blacs_openmpi_lp64.a	LP64 version of BLACS routines supporting Open MPI.	Cluster support	
libmkl_blacs_openmpi_ilp64.a	ILP64 version of BLACS routines supporting Open MPI.	Cluster support	
Distributed SYCL API			
libmkl_sycl_distributed_dft.a	SYCL-specific distributed DFT library for the Intel DPC++ compilers.		

NOTE

In case any functionality from a specific library is needed, this library should be added to the link line. For example, if you want to use oneMKL with the GCC compiler + ILP64 interface + GNU OpenMP threading, the link line should contain:

1. mkl_intel_ilp64 for ILP64 interface
2. mkl_gnu_thread for GNU OpenMP threading
3. mkl_core for all other oneMKL functionality

Details about what should be linked in different cases can be found in the [oneMKL Link Line Advisor](#).

Dynamic Libraries in the `lib` Directory

Some of the libraries in this directory are optional. However, some optional libraries are installed by default, while the rest are not. To get those libraries that are not installed by default, explicitly select the specified optional component during installation.

File	Contents	Optional Component	
		Name	Installed by Default
libmkl_rt.so	Single Dynamic Library for C/Fortran functionality.		

File	Contents	Optional Component	
		Name	Installed by Default
	Instead of multiple libraries, the user can link to one mkl_rt library and select interface and threading via function or environment variable.		
SYCL API Layer			
libmkl_sycl.so	Umbrella library (linker script) to link with all domain-specific SYCL libraries.		
	Required if SYCL-specific functionality is needed.		
libmkl_sycl_blas.so	BLAS SYCL library for SYCL compilers		
libmkl_sycl_lapack.so	LAPACK SYCL library for SYCL compilers		
libmkl_sycl_dft.so	Discrete Fourier Transform (DFT) SYCL library for SYCL compilers		
libmkl_sycl_sparse.so	Sparse BLAS SYCL library for SYCL compilers		
libmkl_sycl_vm.so	Vector Mathematics (VM) SYCL library for SYCL compilers		
libmkl_sycl_rng.so	Random Number Generator (RNG) SYCL library for SYCL compilers		
libmkl_sycl_stats.so	Summary Statistic SYCL library for SYCL compilers		
libmkl_sycl_data_fitting.so	Data Fitting SYCL library for SYCL compilers (experimental)		
Interface Layer			
libmkl_intel_lp64.so	LP64 interface library for the Intel compilers. Also use for other supported compilers that do not have a specialized Intel® oneAPI Math Kernel Library (oneMKL) interface library.		

File	Contents	Optional Component	
		Name	Installed by Default
	All Intel® oneAPI Math Kernel Library (oneMKL) function domains support LP64 programming. Read more about LP64 in Using the ILP64 Interface vs. LP64 Interface.		
libmkl_intel_ilp64.so	ILP64 interface library for the Intel compilers. Also use for other supported compilers that do not have a specialized Intel® oneAPI Math Kernel Library (oneMKL) interface library All oneMKL function domains support ILP64 programming with some limitations for FFT. Read more about ILP64 and limitations in Using the ILP64 Interface vs. LP64 Interface..		
libmkl_gf_lp64.so	LP64 interface library for the GNU Fortran compilers All oneMKL function domains support LP64 programming. Read more about LP64 in Using the ILP64 Interface vs. LP64 Interface..	GNU* Compiler Collection support	Yes
libmkl_gf_ilp64.so	ILP64 interface library for the GNU Fortran compilers All oneMKL function domains support ILP64 programming with some limitations for FFT. Read more about ILP64 and limitations in Using the ILP64 Interface vs. LP64 Interface..	GNU* Compiler Collection support	Yes
Threading Layer			
libmkl_intel_thread.so	OpenMP threading library for the Intel compilers.		

File	Contents	Optional Component	
		Name	Installed by Default
	All oneMKL function domains support Intel OpenMP threading.		
libmkl_tbb_thread.so	Intel TBB threading library for the Intel compilers.	Intel TBB threading support	Yes
	All oneMKL function domains support Intel TBB threading.		
libmkl_gnu_thread.so	OpenMP threading library for the GNU Fortran and C compilers.	GNU* Compiler Collection support	Yes
	All oneMKL function domains support GNU OpenMP threading.		
libmkl_sequential.so	Sequential library.		
	All oneMKL function domains support sequential mode.		
Computational Layer			
libmkl_core.so	All non-architecture-specific implementation and library dispatcher for dynamic load of architecture-specific kernels.		
	Read more about how to work with <code>mkl_enable_instructions</code> and <code>mkl_cbwr_set</code> in the developer reference.		
	<code>mkl_core</code> is required for any link configurations. You don't need to link kernel-specific libraries; just link <code>mkl_core</code> .		
libmkl_def.so	Default kernel library.		
	All oneMKL function domains except Vector Mathematics (VM)/ Vector Statistics (VS)/ Data Fitting (DF) are presented in this library.		

File	Contents	Optional Component	
		Name	Installed by Default
libmkl_mc3.so	Kernel library for Intel® Streaming SIMD Extensions 4.2 (Intel® SSE4.2) enabled processors. All oneMKL function domains except VM/VS/DF are presented in this library.		
libmkl_avx2.so	Kernel library for Intel® Advanced Vector Extensions 2 (Intel® AVX2) enabled processors. All oneMKL function domains except VM/VS/DF are presented in this library.		
libmkl_avx512.so	Kernel library for dispatching Intel® Advanced Vector Extensions 512 (Intel® AVX-512) on Intel® Xeon® processors. All oneMKL function domains except VM/VS/DF are presented in this library.		
libmkl_vml_def.so	Vector Mathematics (VM)/Vector Statistics (VS)/Data Fitting (DF) part of default kernels		
libmkl_vml_mc3.so	VM/VS/DF for Intel® SSE4.2 enabled processors		
libmkl_vml_avx2.so	VM/VS/DF optimized for Intel® AVX2 enabled processors		
libmkl_vml_avx512.so	VM/VS/DF optimized for Intel® AVX-512 on Intel® Xeon® processors		
libmkl_vml_cmpt.so	VM/VS/DF library for conditional numerical reproducibility		

Cluster Libraries

File	Contents	Optional Component	
		Name	Installed by Default
libmkl_scalapack_lp64.so	ScaLAPACK routine library supporting the LP64 interface	Cluster support	
libmkl_scalapack_ilp64.so	ScaLAPACK routine library supporting the ILP64 interface	Cluster support	
libmkl_cdft_core.so	Cluster version of FFT functions.	Cluster support	
libmkl_blacs_intelmpi_lp64.so	LP64 version of BLACS routines for Intel® MPI Library and MPICH2 or higher.	Cluster support	
libmkl_blacs_intelmpi_ilp64.so	ILP64 version of BLACS routines for Intel MPI Library and MPICH2 or higher.	Cluster support	
libmkl_blacs_openmpi_lp64.so	LP64 version of BLACS routines for Open MPI.	Cluster support	
libmkl_blacs_openmpi_ilp64.so	ILP64 version of BLACS routines for Open MPI.	Cluster support	
Distributed SYCL API			
libmkl_sycl_distributed_dft.so	Distributed Discrete Fourier Transform (DFT) SYCL library for SYCL compilers.		

NOTE

In case any functionality from a specific library is needed, this library should be added to the link line. For example, if you want to use oneMKL with the GCC compiler + ILP64 interface + GNU OpenMP threading, the link line should contain:

1. mkl_intel_ilp64 for ILP64 interface
2. mkl_gnu_thread for GNU OpenMP threading
3. mkl_core for all other oneMKL functionality

Details about what should be linked in different cases can be found in the [oneMKL Link Line Advisor](#).

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Notices and Disclaimers

Intel technologies may require enabled hardware, software or service activation.

No product or component can be absolutely secure.

Your costs and results may vary.

© Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Product and Performance Information
Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex .
Notice revision #20201201

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Java is a registered trademark of Oracle and/or its affiliates.